3.15 Step Motor

1) 구조

장비에서는 모터의 구동 원리 학습을 위해 스테핑 모터 기본으로 제공합니다. 이 모터는 A, B, /A, /B 4개의 데이터 라인을 가지고 모터 동작을 제어하는 유니폴라 방식의 스텝 모터를 사용하고, 모터로 들어가는 데이터 라인의 주파수 높고 낮음에 따라서 모터의 회전 속도를 제어 할 수 있습니다.



2) 동작

모터 제어를 할 수 있도록 일반적으로 많이 사용하는 스텝 모터가 설치되어 있습니다. 위상 제어를 위해 FPGA I/O가 TR을 통해 연결되어 있으며, 모터의 제어는 여자 방식에 따라 세가지 방식으로 제어가 가능합니다. 1상 여자, 2상 여자, 1-2상 여자 방식이 있고, 이 중 1-2 여자 방식은 고-분해능의 이점을 기지고 있습니다. 다음의 표에서는 모터를 제어하기 위한 데이터의 흐름을 표로 보여주고 있습니다. 각각의 데이터 라인에 다음가 같은 표를 순차적으로 인가하여 모터를 회전 시킬 수 있습니다.

❑ 1상 여자

구분	1	2	3	4	5	6	7	8	9
A	1	0	0	0	1	0	0	0	1
B	0	1	0	0	0	1	0	0	0
/A	0	0	1	0	0	0	1	0	0
/B	0	0	0	1	0	0	0	1	0

1주기

❑ 2상 여자

구분	1	2	3	4	5	6	7	8	9
A	1	0	0	1	1	0	0	1	1
B	1	1	0	0	1	1	0	0	1
/A	0	1	1	0	0	1	1	0	0
/B	0	0	1	1	0	0	1	1	0

1주기

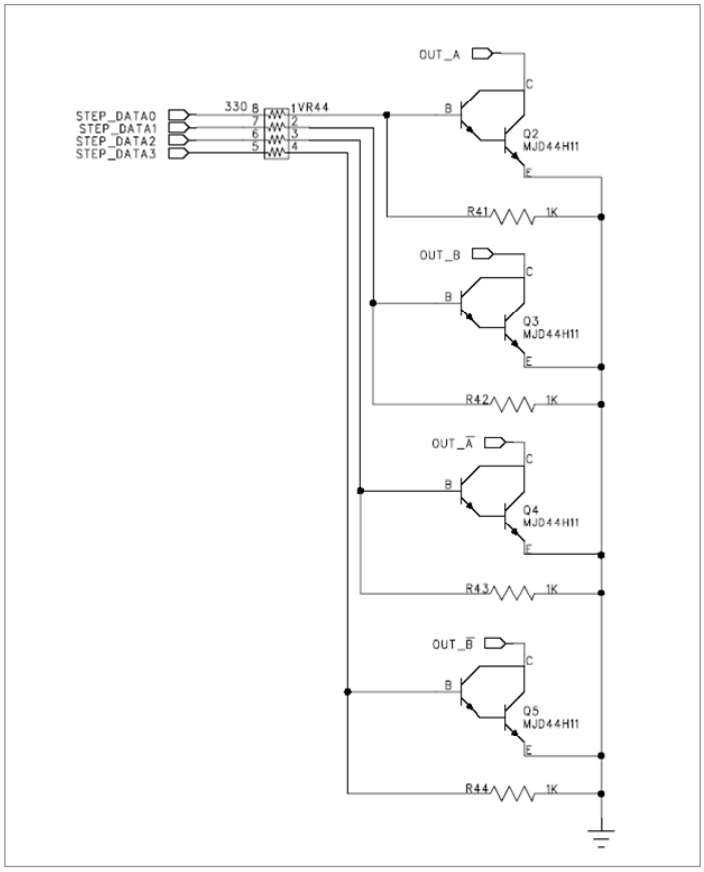
❑ 1-2상 여자

구분	1	2	3	4	5	6	7	8	9
A	1	1	0	0	0	0	0	1	1
B	0	1	1	1	0	0	0	0	0
/A	0	0	0	1	1	1	0	0	0
/B	0	0	0	0	0	1	1	1	0

1주기

FPGA에서의 모터 제어는 위 표 중 하나의 여자 방식을 선택하여 모터에 신호를 제공하여 모터를 동작하게 합니다.

3) 회로



회로에서 보는 것과 같이 A, B, /A, /B 4개의 데이터 라인이 FPGA로부터 모터로 입력되는 모습을 보여주고 있습니다. 모터에서는 FPGA로부터 출력되는 전류가 작기 때문에 중간에 NPN 트랜지스터를 사용, 전류를 증폭하여 모터로 보내게 됩니다. 따라서 모터에 사용할 수 있는 전류로 증폭이 되어 모터의 동작이 이루어 지게 됩니다.

4) 핀 구성표

FPGA Signal	Altera	Xilinx	Description
STEP_DATA[0]	AA23	Y22	Motor Data A
STEP_DATA[1]	AB24	Y25	Motor Data B
STEP_DATA[2]	Y26	Y21	Motor Data /A
STEP_DATA[3]	AB23	Y24	Motor Data /B
Motor_Sensor	AA26	Y26	Magnetic Sensor

핀 구성에 있는 Motor_Sensor는 스텝 모터의 구동에 활용할 수 있도록 자기 센서를 두어 모터의 회전을 카운트 하도록 마련하고 있습니다. 이 자기 센서는 모터의 회전 구조물에 자석이 설치 되어 있고 이 자석의 자기를 감지하여 감지된 값을 FPGA로 전달하게 됩니다. 이 센서는 모터의 구동에 이용할 수 있는 장치로 회전을 감지하여 모터의 구동에 이용할 수 있도록 구성하고 있습니다.

3.16 UART

1) 구성

본 제품에는 PC의 RS232(시리얼)포트와 연결하여 시리얼 데이터 통신을 할 수 있도록 하는 Male타입의 9핀 커넥터를 제공하고 있습니다. 이 장치는 PC에서 장비를 제어할 때 이용할 수 있는 장치로 PC의 하이퍼터미널의 통신 프로그램을 통해 장비와 연동하여 사용할 수 있습니다.

예를 들어 PC에서 시리얼 포트를 통해 명령어를 장비로 전달하면, 장비의 FPGA 디바이스가 미리 설계된 소스를 통해 이를 파악하고 해당하는 동작을 하게 됩니다.



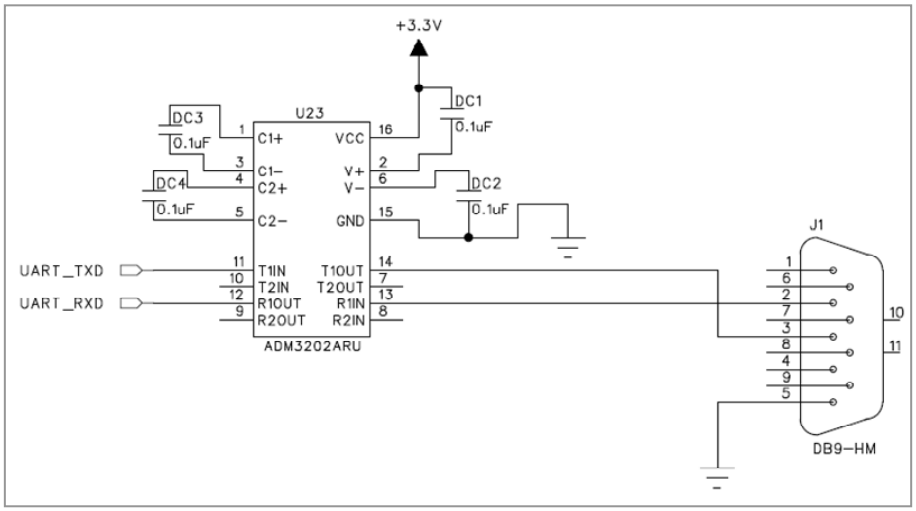
2) 동작

제품에서 사용된 시리얼 인터페이스용 디바이스는 Analog사의 ADM3202를 사용하였습니다. 이 디바이스는 최대 460 kbps의 데이터 전송 속도를 가진 디바이스로 최대 2개의 시리얼 포트를 지원해 주는 디바이스 입니다.

FPGA에서 TTL 레벨로 시리얼 디바이스로 데이터를 보내면 이 디바이스가 시리얼 통신을 위한 전압 레벨로 변환해 줍니다. 입력은 이와 반대로 입력된 전압 레벨을 FPGA가 인식할 수 있는 TTL 레벨로 변환해 주기 때문에 FPGA 및 PC간의 데이터 통신이 이루어 지게 됩니다.

시리얼 통신의 이용은 PC등의 주변 환경과 데이터 통신을 장비 내부의 장치를 제어하거나, 장비에서 얻어지는 센서 등의 값을 다른 장비로 전송하는데 이용할 수 있습니다.

3) 회로



회로에서 보는 것과 같이 +3.3V의 전원으로 시리얼 디바이스가 동작을 하게 됩니다. 따라서 회로 구현이 쉬운 장점이 있습니다. 이 디바이스와 9핀 D-SUB 커넥터를 통해 PC와 장비간의 데이터 통신이 이루어 지게 됩니다.

4) 핀 구성표

FPGA Signal	Altera	Xilinx	Description
UART_TXD	F7	H12	Transmitter Data Input
UART_RXD	H3	K2	Transmitter Data Output

3.17 USB/Serial 포트

1) 구성

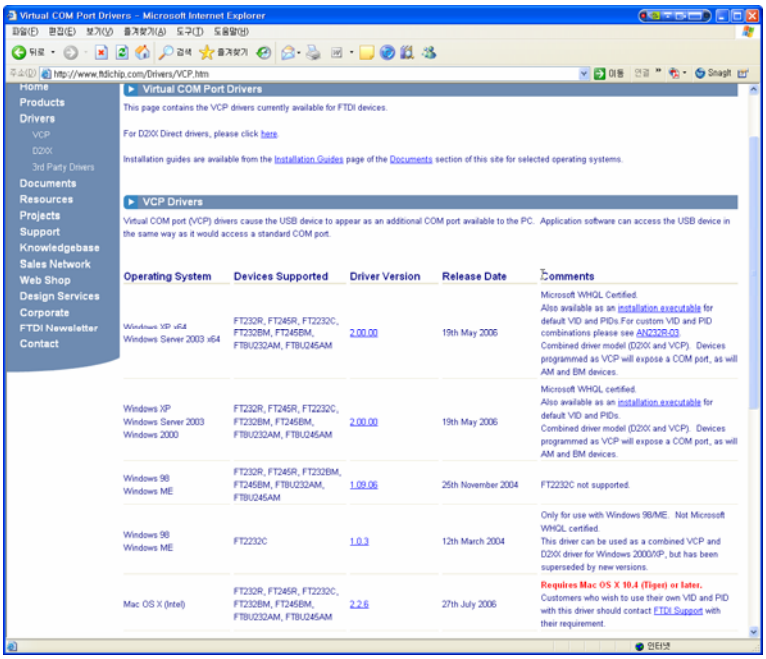
본 장비에서 USB 포트는 시리얼 통신을 할 수 있는 포트 입니다. 따라서 RS232 포트를 통한 통신 실험이 아닌 USB 포트를 이용한 시리얼 통신 실험을 할 수 있는 블록입니다. 이러한 구성은 현재 PC의 RS232포트가 USB 포트로 변화하는 장비에서도 시리얼 통신을 할 수 있도록 하는 USB 포트를 구성하고 있습니다.



2) 동작

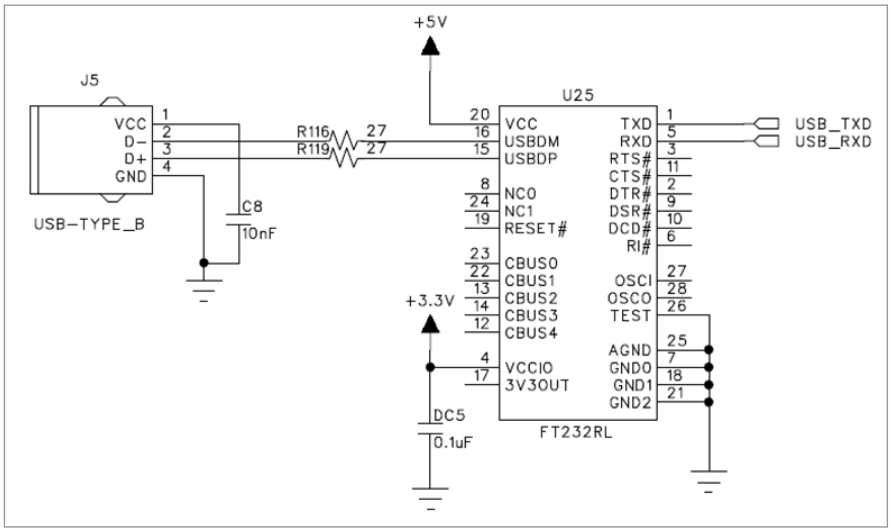
장비에서는 FT232 디바이스를 사용하여 USB를 통한 Serial 데이터 통신 기능을 지원하고 있습니다. UART와 비슷한 핀 구조로 연결 되어 있고, 제어 방법도 UART의 제어 방법과 같은 방법으로 제어 하시면 됩니다.

USB 장치를 PC와 연결하면 드라이버를 설치하라는 메시지가 생기게 됩니다. 이 드라이버를 설치하기 위해서는 먼저 http://www.ftdichip.com/ 에서 FT232R에 대한 드라이버를 먼저 설치를 하거나 CD에서 Windows XP용으로 제공하고 드라이버를 설치하여야 합니다. 아래 그림에서 사이트의 다운로드 센터를 확인할 수 있습니다. 여기에서 사용자의 컴퓨터 환경에 맞는 드라이버를 다운받아 설치해 주시면 됩니다.



드라이버를 설치하면 USB port가 인식이 되며 PC의 드라이버 정보를 통해 현재 연결된 USB to Serial의 등록 상태를 확인할 수 있습니다.

3) 회로



회로에서 보는 것과 같이 USB에서 입력되는 데이터 핀이 FT232 디바이스를 통해 바로 FPGA 디바이스로 연결되어 있습니다.

4) 핀 구성표

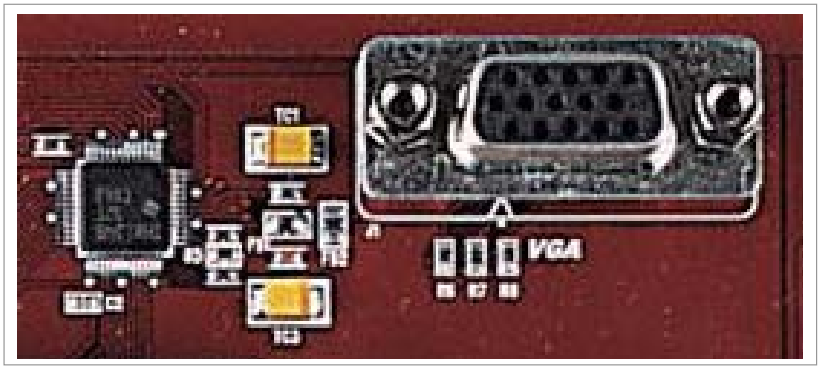
FPGA Signal	Altera	Xilinx	Description
USB_TXD	H1	J11	Transmitter Data Input
USB_RXD	H2	J12	Transmitter Data Output

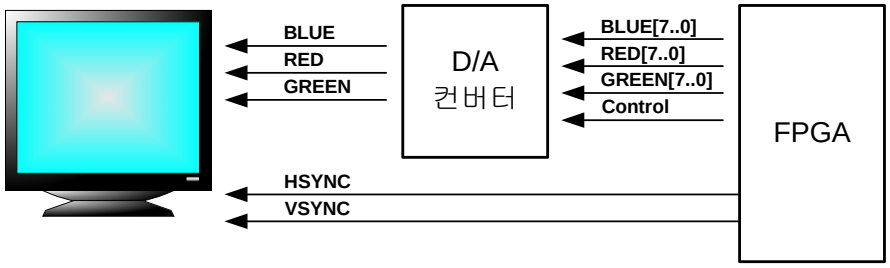
3.18 VGA

1) 구성

HBE-Combo III에서는 VGA port를 Peripheral의 확장 보드에서 지원해 주고 있습니다. 따라서 이 장치를 이용하기 위해서는 해당되는 확장 보드를 장착 후 이용하셔야 합니다.

현재 장비에서 지원하는 VGA 장치는 FPGA I/O에 컬러 VGA 모니터를 구동할 수 있도록 (디지털 RGB를 아날로그 RGB 신호 변환해 줄 수 있는) D/A Converter가 연결되어 있습니다. 다음 그림에서는 데이터의 흐름에 대한 블록도를 보이고 있습니다.





[그림 3-21] VGA 블록 구성

2) 동작

VGA 모니터를 구동하기 위해 FPGA가 디지털 신호를 처리하는 디바이스이기 때문에 신호를 변환하기 위한 D/A 컨버터로 R (빨강), G (녹색), B (파랑) 색깔 별로 8개의 데이터 라인이 연결되어 총 24 bit의 데이터 라인으로 제어하도록 구성되어 있습니다. 이러한 D/A 컨버터를 통해 다양한 색깔을 표현할 수 있고, 1024x768 해상도에 최대 24-비트 RGB 컬러를 구현할 수 있도록 구성되어 있습니다. 사용된 D/A 컨버터의 특징은 다음과 같습니다.

- Triple 8-Bit D/A Converters
- Minimum 80 MSPS Operation
- Direct Drive of Doubly-Terminated 75-W Load Into Standard Video Levels
- 3×8 Bit 4:4:4, 2×8 Bit 4:2:2 or 1×8 Bit 4:2:2 (ITU-BT.656) Multiplexed YPbPr/GBR Input Modes
- Bi-Level (EIA) or Tri-Level (SMPTE) Sync
- Generation With 7:3 Video/Sync Ratio
- Integrated Insertion of Sync-On-Green/Luminance or Sync-On-All Channels
- Configurable Blanking Level
- Internal Voltage Reference
- High-Definition Television (HDTV) Set-Top
- Boxes/Receivers
- High-Resolution Image Processing
- Desktop Publishing
- Direct Digital Synthesis/I-Q Modulation

디바이스에 대한 보다 자세한 설명은 데이터시트를 참고로 하도록 합니다.

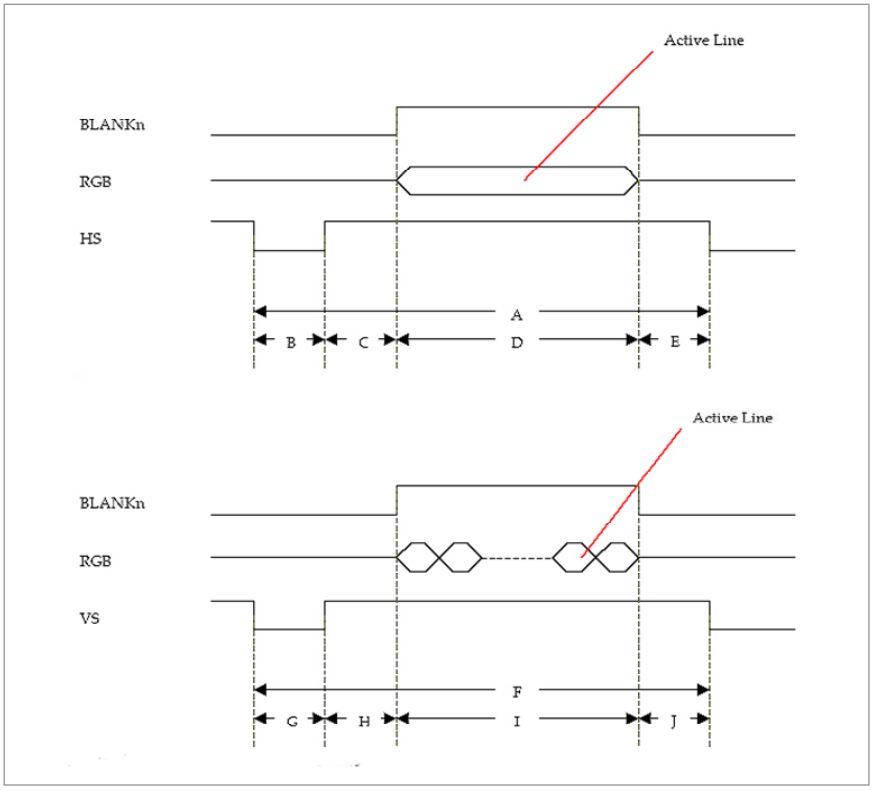
VGA 블록은 FPGA에서 VGA D/A Driver를 제어하는 회로와 VGA 모니터 동기 신호(H-sync/V-sync)를 생성해 주는 회로를 설계해 주어야 하며, 동시에 이와 관련된 I/O 핀을 적절히 지정해 주어야 합니다. 모니터 동기 신호는 구현하고자 하는 해상도에 따라 그 출력 주파수도 달라지며, 동시에 D/A 컨버터 입력 클럭도 수정되어야 합니다. 1024x768 해상도를 구현하기 위해서는 구동 클럭이 65MHz가 되어야 합니다.

다음 표는 해상도에 따른 VGA 동작 타이밍과 기타 유용한 정보를 보이고 있습니다.

〈표 3-6〉 VGA 동작 타이밍

구분	해상도	640 X 480	800 X 600	1024 X 768
A	Line Period	32.8us	26.4us	20.7us
B	Hsync Sync Period	3.8us	3.2us	2.1us
C	Hsync Back Porch	1.9us	2.2us	2.5us
D	Active Video	25.4us	20us	15.7us
E	Hsync Front Porch	0.6us	1us	0.4us
F	Frame Period	16.7ms	16.58ms	16.67ms
G	Vsync Sync Period	0.05ms	0.1ms	0.12ms
H	Vsync Back Porch	1ms	0.6ms	0.6ms
I	Active Frame	15.3ms	15.84ms	15.88ms
J	Vsync Front Porch	0.3ms	0.02ms	0.06ms

비디오 신호는 라인 수평 동기 신호와 수직 동기 신호로 구성됩니다. 해상도에 따라 수평 동기 신호는 640, 800, 1024 픽셀로 구성되며, 수직 동기 신호는 480, 600, 768 픽셀이 됩니다. Blankn 신호가 비디오 DAC 출력 (R, G, B)가 0 임을 출력합니다. 수평 동기 신호 (HS)는 새로운 라인의 시작을 의미하고, 수직 동기 신호 (VS)는 새로운 프레임의 시작을 나타냅니다.



[그림 3-22] 수평 동기 및 수직 동기

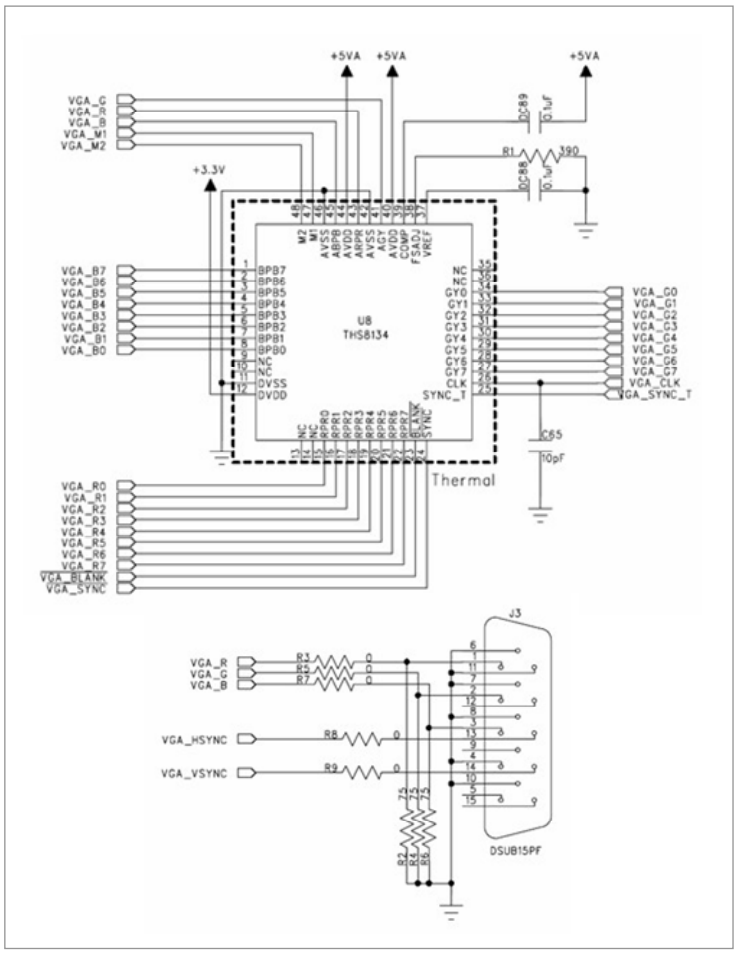
〈표 3-7〉 Video Settings

해상도	640 X 480	800 X 600	1024 X 768
Video clock	25.2MHz	40MHz	65MHz
Horizontal Resolution	640	800	1024
Vertical Resolution	480	600	768
Hsync Pulse Width	95	128	136
Hsync Back Porch width	40	88	160
Hsync Front Porch width	25	40	24

Vsync Pulse Width	2	4	6
Vsync Back Porch Width	22	23	29
Vsync Front Porch Width	10	1	3

그리고 H.Sync (수평동기)와 V.Sync (수직동기)는 각각 주사선의 수평, 수직동기를 맞추기 위한 신호로 이 동기신호가 적절해야만 정상적인 화면을 표현할 수 있습니다. 따라서, H.Sync와 V.Sync에 적절히 동기 시켜 RGB 데이터를 입력함으로 모니터 내의 원하는 위치에 데이터를 표현할 수 있습니다.

3) 회로



회로에서 보는 것과 같이 THS8134의 D/A 컨버터를 이용하여 모니터를 제어합니다.

FPGA 디바이스로부터 R (빨강), G (녹색), B (파랑) 세가지 색깔에 대한 8-bit 데이터를 THS8134 디바이스로 전달하고 이러한 데이터 값을 변환하여 모니터의 색깔을 제어하고 있습니다.

4) 핀 구성표

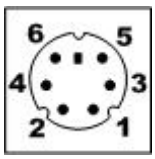
FPGA Signal	Altera	Xilinx	Description
VGA_BLUE[0]	G17	F25	Blue/Pr pixel data 0
VGA_BLUE[1]	G16	F24	Blue/Pr pixel data 1
VGA_BLUE[2]	B20	C17	Blue/Pr pixel data 2
VGA_BLUE[3]	B19	C16	Blue/Pr pixel data 3
VGA_BLUE[4]	J22	L22	Blue/Pr pixel data 4
VGA_BLUE[5]	K19	M21	Blue/Pr pixel data 5
VGA_BLUE[6]	B18	C15	Blue/Pr pixel data 6
VGA_BLUE[7]	B17	B23	Blue/Pr pixel data 7
VGA_RED[0]	A20	A22	Red/Pr pixel data 0
VGA_RED[1]	A19	A20	Red/Pr pixel data 1
VGA_RED[2]	A18	A19	Red/Pr pixel data 2
VGA_RED[3]	A17	A18	Red/Pr pixel data 3
VGA_RED[4]	G15	F23	Red/Pr pixel data 4
VGA_RED[5]	G14	F22	Red/Pr pixel data 5
VGA_RED[6]	A14	A15	Red/Pr pixel data 6
VGA_RED[7]	B15	B20	Red/Pr pixel data 7
VGA_GREEN[0]	G13	F20	Green/Pr pixel data 0
VGA_GREEN[1]	F18	F19	Green/Pr pixel data 1
VGA_GREEN[2]	B14	B19	Green/Pr pixel data 2
VGA_GREEN[3]	A23	B18	Green/Pr pixel data 3
VGA_GREEN[4]	F17	F17	Green/Pr pixel data 4
VGA_GREEN[5]	G18	G15	Green/Pr pixel data 5
VGA_GREEN[6]	B18	B17	Green/Pr pixel data 6
VGA_GREEN[7]	B17	B15	Green/Pr pixel data 7
VGA_BLANK	B16	B21	Blanking control input
VGA_SYNC	C16	C23	Sync control input
VGA_CLK	F23	J21	VGA clock
VGA_SYNC_T	J24	M18	Sync tri-level control
VGA_HSYNC	J23	L24	Horizontal Sync
VGA_VSYNC	C15	C22	Vertical Sync
VGA_M1	J26	M20	Operation mode control 1
VGA_M2	J25	M19	Operation mode control 2

3.19 PS/2 포트 사용하기

1) 구성

PS/2 포트는 주로 PC 키보드나 마우스의 입력장치로 사용되는 직렬 통신방식 입니다. 따라서 키보드 컨트롤러나 마우스 컨트롤러를 설계해 볼 수 있는 PS/2 커넥터를 제공하고 있습니다. PS/2는 데이터 선과 클럭 선 두 개만 가지고 데이터의 전송이 이루어집니다.

데이터 라인이 하나이기 때문에 데이터는 클럭에 동기 되어 시리얼 형태로 전달 됩니다. PS/2의 클럭은 키보드나 마우스 (슬레이브)가 어떤 동작 (키를 눌렀을 경우)을 할 때만 발생됩니다.



PS/2 PORT

〈표 3-8〉 PS/2 Pin Description

Pin	Name	Description
1	Data	Key Data
2	N/C	Not Connect
3	GND	Ground
4	VCC	+5V DC
5	CLK	Clock
6	N/C	Not Connect

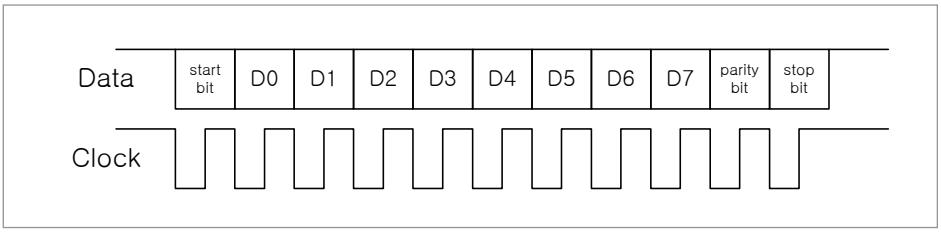
데이터 형태는 시작(1-bit), 정지 비트(1-bit), 패리티 비트(1-bit), 데이터 비트(8-bit)로 구성되며, 모두 11-bit로 구성되어 있습니다. 키보드를 누르면 키보드는 해당 키의 스캔코드 값을 출력합니다.

이 코드는 처음 눌렀을 때 출력할 뿐 만 아니라 키를 뗐을 때에도 출력이 되기 때문에 키가 계속 눌려져 있을 상태를 알 수 있도록 해 줍니다. 결국 키보드는 키를 누를 때 키 값에 해당하는 마크 코드와 키를 뗐을 때 브레이크 코드의 마크 코드를 출력하므로 그 사이를 구분을 할 수 있습니다.



2) 동작

아래의 그림과 같이 키보드 통신에 사용되는 데이터 형태는 start bit, data bit 8개, parity bit, stop bit등의 총 11비트로 구성되어 있습니다. 데이터 신호는 클럭 신호로 동기를 취하는 것이 전제 되어있기 때문에 타이밍만 일치하면 데이터를 보내는 속도에 오차가 많더라도 문제가 되지 않으며 규격으로는 1비트당 60 s에서 100 s면 됩니다.



키보드의 키가 눌려지면 키보드 내부 컨트롤러에 의해 키에 대응하는 스캔 코드를 데이터 선을 통해 보냅니다. 스캔코드는 키가 떨어질 때에도 코드를 발생하기 때문에 이를 통해 키가 계속 눌러지는 것을 인식 할 수 있게 됩니다. 키를 누를 때는 make code가 떨어져 break code가 발생합니다. Make code는 1 Byte로 이루어져 있으며 break code는 make code 앞부분에 F0가 더해진 2 Byte로 이루어 집니다.

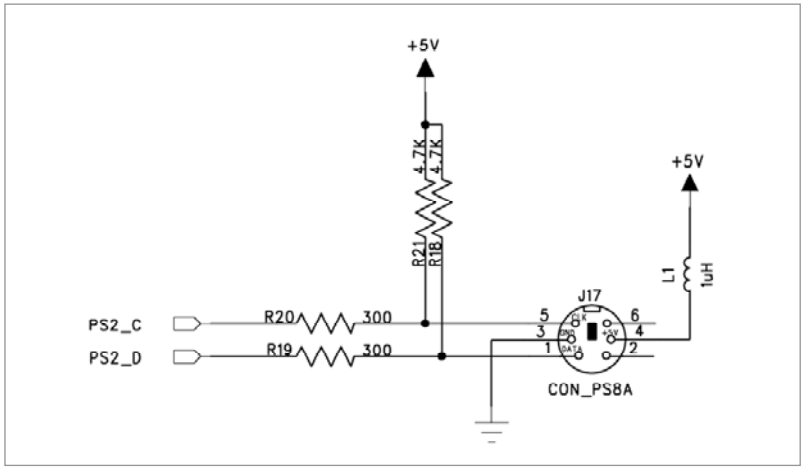
아래의 표는 해당 알파벳 및 숫자에 대한 스캔코드 입니다.

〈표 3-9〉 알파벳 숫자에 대한 스캔 코드

입력 키	Make 코드	Break 코드	입력키	Make 코드	Break 코드	입력키	Make 코드	Break 코드
1	16	F0 16	C	21	F0 21	Q	15	F0 15
2	1E	F0 1E	D	23	F0 23	R	2D	F0 2D
3	26	F0 26	E	24	F0 24	S	1B	F0 1B
4	25	F0 25	F	2B	F0 2B	T	2C	F0 2C
5	2E	F0 2E	G	34	F0 34	U	3C	F0 3C
6	36	F0 36	H	33	F0 33	V	2A	F0 2A
7	3D	F0 3D	I	43	F0 43	W	1D	F0 1D
8	3E	F0 3E	J	3B	F0 3B	X	22	F0 22
9	46	F0 46	K	42	F0 42	Y	35	F0 35
0	45	F0 45	L	4B	F0 4B	Z	1A	F0 1A
-	4E	F0 4E	M	3A	F0 3A	Enter	5A	F0 5A

=	55	F0 55	N	31	F0 31
A	1C	F0 1C	O	44	F0 44
B	32	F0 32	P	4D	F0 4D

3) 회로



회로에서 보는 것과 같이 PS/2에 해당되는 클럭과 데이터 라인이 FPGA 디바이스와 연결 된 모습을 보여주고 있습니다. 이 데이터 라인으로부터 FPGA 디바이스로 스캔 코드를 읽어 들여 키보드에서 어떠한 값이 들어 왔는지 분석하여 인식하게 되는 것 입니다.

4) 핀 구성표

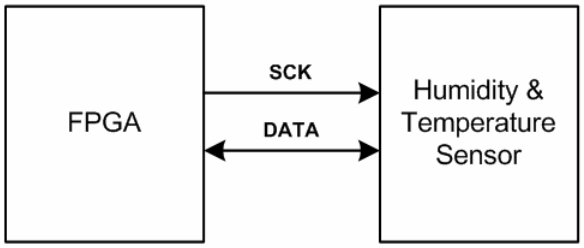
FPGA Signal	Altera	Xilinx	Description
PS2_CLK	G6	J9	PS/2 Clock
PS2_DATA	G5	J8	PS/2 Key data input

3.20 Sensor

1) 구성

현재 장비에 장착되어 있는 센서는 장비가 있는 환경의 온도 및 습도를 측정 할 수 있는 장치로 디지털 방식의 측정 장치입니다. 이 장치는 I2C 방식으로 데이터를 주고 받으며, 측정할 수 있는 온도는 -40 ~ 124℃, 측정할 수 있는 습도의 범위는 0 ~ 100%의 범위를 가지고 있습니다.





[그림 3-23] Sensor 구성

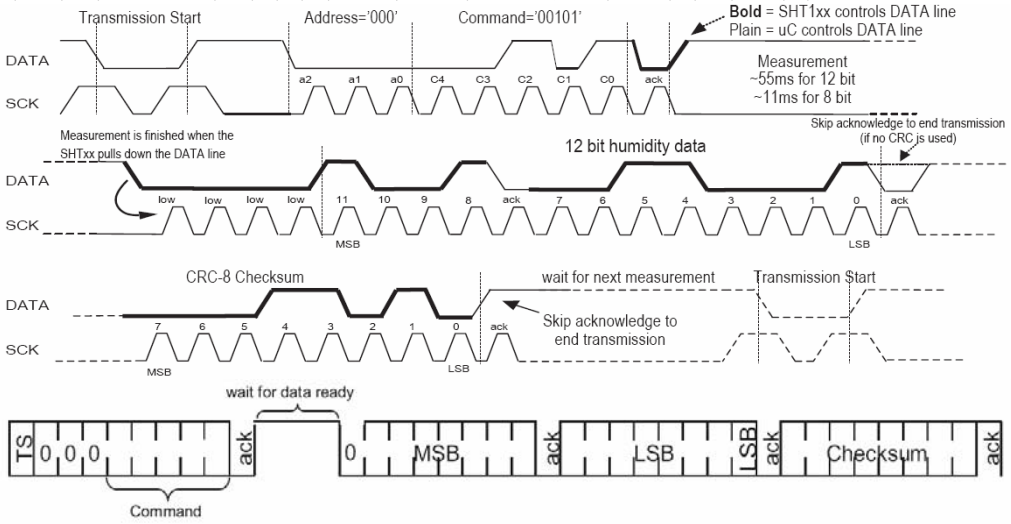
2) 동작

장치를 통해 현재 센서 값을 받기 위해서는 컨트롤러에서 해당하는 Command를 I2C로 보내주어야 합니다. 따라서 센서가 요청한 Command에 따라서 어떠한 동작을 할 것인지 결정하게 됩니다. 다음에 이러한 Command에 대한 내용을 보여주고 있습니다.

Command	Code
Reserved	0000x
Measure Temperature	**00011**
Measure Humidity	**00101**
Read Status Register	00111
Write Status Register	00110
Reserved	0101x-1110x
Soft reset, resets the interface, clears the status register to default values wait minimum 11 ms before next command	**11110**

[그림 3-24] Command list

위의 표에서 보이듯이 Command에 대한 값을 I2C를 통해 센서로 전달하면 데이터 값이 바로 나오는 것이 아니라 얼마간의 Delay 시간을 두고 해당하는 정보 값을 FPGA로 전달하게 됩니다. 이러한 타이밍 정보는 다음의 그림을 통해 참고해 보시기 바랍니다. 이 파형을 통해 데이터의 송 수신 시점을 확인해 보시기 바랍니다.



[그림 3-25] Data Interface Timing

사용자의 위의 그림과 같은 방법으로 온도 및 습도에 대한 값을 요청하고 해당되는 데이터 값을 센서로부터 전달 받을 수 있습니다.
위와 같은 방법으로 데이터를 전달 받으면 전달 받은 데이터 값을 기준으로 현재 온도 및 습도에 대한 값을 계산해 주어야 합니다. 다음의 자료를 통해 계산 공식을 참고 하시기 바랍니다.

❑ Temperature

$$Temperature = d_1 + d_2 \bullet SO_T$$

VDD	d_1 [°C]	d_1 [°F]
5V	-40.00	-40.00
4V	-39.75	-39.50
3.5V	-39.66	-39.35
3V	-39.60	-39.28
2.5V	-39.55	-39.23

SO_T	d_2 [°C]	d_2 [°F]
14bit	0.01	0.018
12bit	0.04	0.072

❑ Humidity

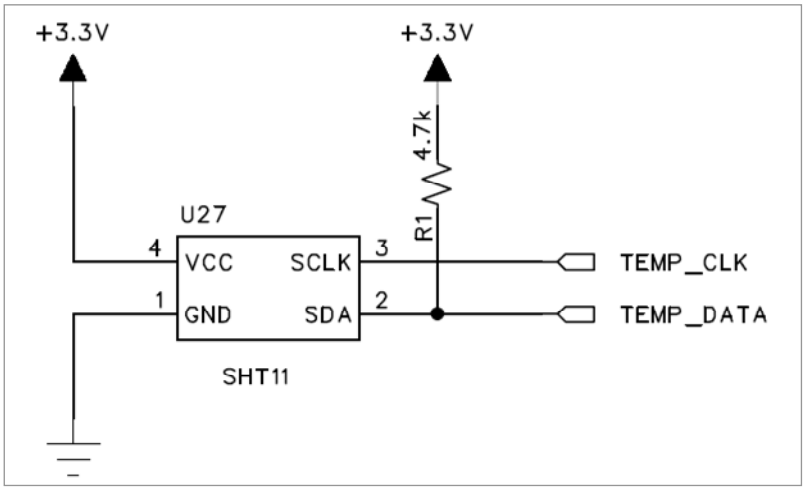
$$RH_{linear} = c_1 + c_2 \bullet SO_{RH} + c_3 \bullet SO_{RH}^2$$

SO_{RH}	c_1	c_2	c_3
12 bit	-4	0.0405	$-2.8 * 10^{-6}$
8 bit	-4	0.648	$-7.2 * 10^{-4}$

$$RH_{true} = (T_{°C} - 25) \bullet (t_1 + t_2 \bullet SO_{RH}) + RH_{linear}$$

SO_{RH}	t_1	t_2
12 bit	0.01	0.00008
8 bit	0.01	0.00128

3) 회로도



FPGA와 센서간의 I2C를 이용한 데이터 통신을 하기 때문에 Clock 및 Data 라인으로 연결되어 있습니다. 또한 센서 전원은 3.3V를 사용하고 있습니다.

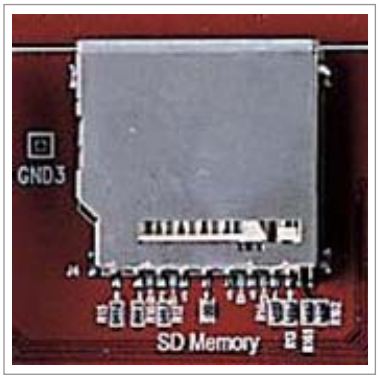
4) 핀 구성표

FPGA Signal	Altera	Xilinx	Description
SENSOR_CLK	J7	K12	Sensor Clock line
SENSOR_DATA	J6	K11	Sensor data line

3.21 SD Connector

1) 구성

본 장비에서는 외부 메모리의 한 형태로 SD Card를 지원하기 위한 커넥터를 구성하고 있습니다. 따라서 이 커넥터를 통해 SD Card를 장착하여 데이터를 읽고 쓰는 환경을 지원해주고 있습니다.

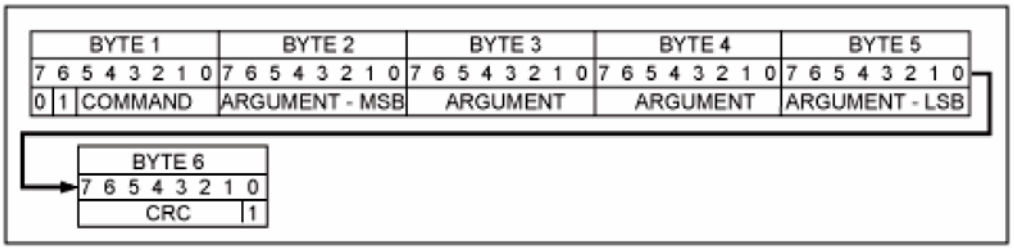


2) 동작

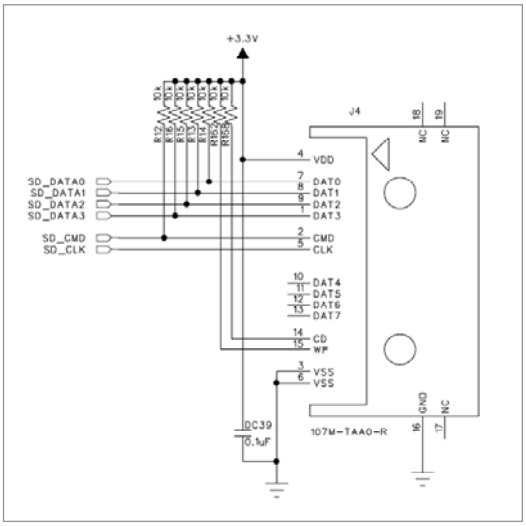
SD 카드는 SPI 통신 방식을 통해 데이터를 주고 받는 형식을 가지고 있습니다. 그리고 온 습도 센서와 같은 방법으로 SD Memory가 어떠한 작업을 할 것인지 Command를 보내면 SD 카드 해당하는 Command에 맞는 데이터를 FPGA로 전달하게 됩니다.

다음에 Command에 대한 자료를 보여주고 있습니다.

Command	Mnemonic	Argument	Reply	Description
0 (0x00)	GO_IDLE_STATE	none	R1	Resets the SD card.
9 (0x09)	SEND_CSD	none	R1	Sends card-specific data.
10 (0x0a)	SEND_CID	none	R1	Sends card identification.
17 (0x11)	READ_SINGLE_BLOCK	address	R1	Reads a block at byte address.
24 (0x18)	WRITE_BLOCK	address	R1	Writes a block at byte address.
55 (0x37)	APP_CMD	none	R1	Prefix for application command.
59 (0x3b)	CRC_ON_OFF	Only Bit 0	R1	Argument sets CRC on (1) or off (0).
41 (0x29)	SEND_OP_COND	none	R1	Starts card initialization.



2) 회로도



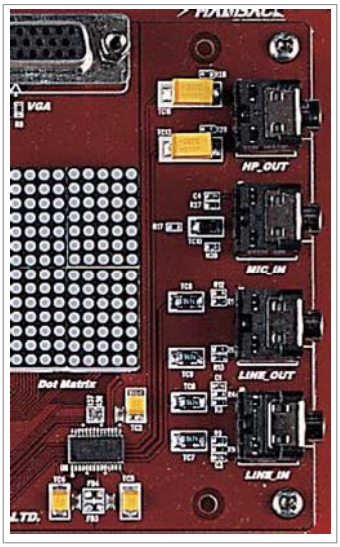
3) 핀 구성표

FPGA Signal	Altera	Xilinx	Description
SD_DATA[]	J1	K5	SD data 0
SD_DATA[]	H6	K4	SD data 1
SD_DATA[]	H4	K3	SD data 2
SD_DATA[]	J8	L3	SD data 3
SD_CMD	J3	K7	SD CMD
SD_CLK	J2	K6	SD Clock

4) Audio Codec

5) 구성

본 장치는 Audio Codec 디바이스를 통해 마이크와 스피커 및 다른 장비와 연결할 수 있도록 지원해 주고 있습니다. 사용하고 있는 디바이스는 8 ~ 96 KHz의 Stereo Audio Codec 디바이스로 높은 성능의 Audio 변환 작업을 지원해 주고 있습니다.



Audio Codec 디바이스의 사양은 다음과 같습니다.

❑ Stereo Codec

- High-Performance Stereo Codec
  - 90dB SNR Multibit Sigma-Delta ADC (A-weighted at 48 KHz)
  - 100-dB SNR Multibit Sigma-Delta DAC (A-weighted at 48 KHz)
  - 8KHz ~ 96KHz Sampling Frequency Support
- Stereo Line Inputs
- Volume Control With Mute on Input and Output
- Audio-Data Input/Output Via TI McBSP Compatible programmable Audio Interface
  - ADC Multiplexed Input for Stereo Line Inputs and Microphone
  - Ideally Suitable for Portable Solid-State Audio Players and Recorders

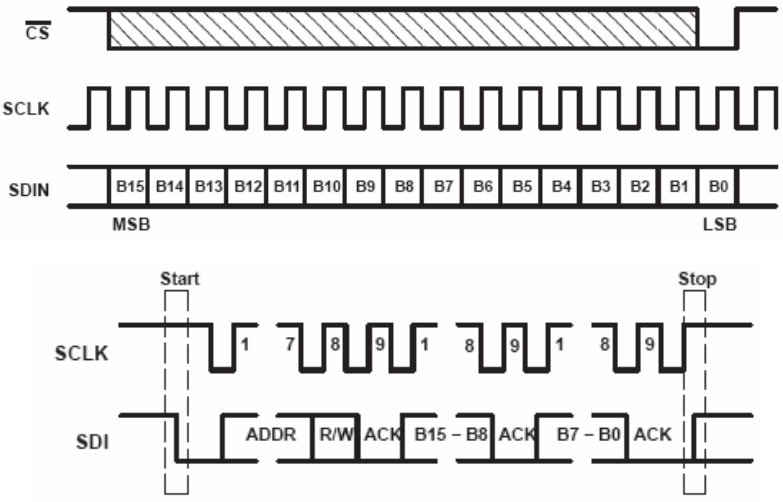
6) 동작

장비에 사용하고 있는 Audio 디바이스는 Stereo Audio Codec 디바이스로써, 스테레오 잭을 통해 입력되는 음원 데이터를 FPGA 디바이스에서 처리할 수 있도록 디지털로 변경하거나 FPGA에서 처리된 음원 데이터를 스피커나 다른 음원 장치에서 사용할 수 있도록 아날로그 값으로 변경해 주는 작업을 합니다.

여기에서 사용하는 TLV320AIC23B는 내부에 레지스터를 통해 디바이스의 데이터 흐름을 제어할 수 있습니다. 다음에서 내부 레지스터의 정보를 보여주고 있습니다.

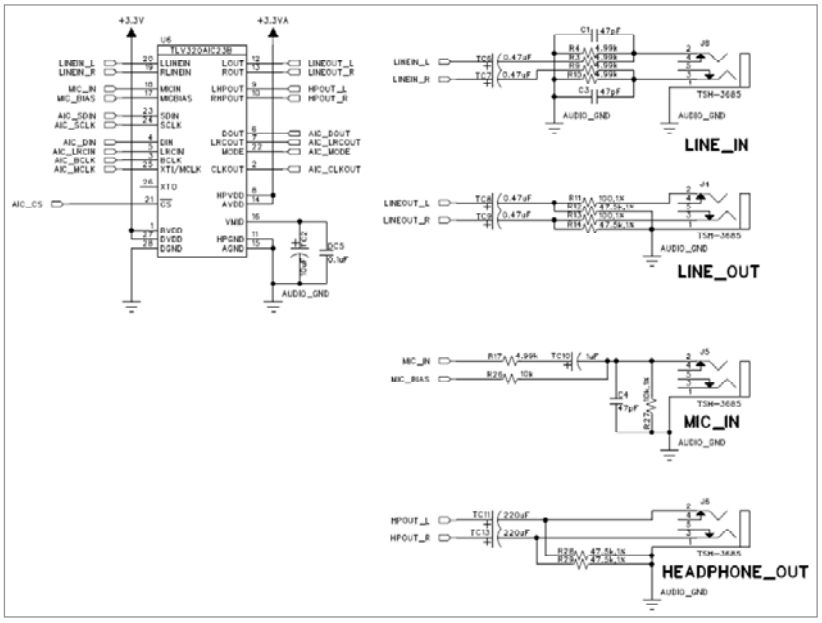
ADDRESS	REGISTER
0000000	Left line input channel volume control
0000001	Right line input channel volume control
0000010	Left channel headphone volume control
0000011	Right channel headphone volume control
0000100	Analog audio path control
0000101	Digital audio path control
0000110	Power down control
0000111	Digital audio interface format
0001000	Sample rate control
0001001	Digital interface activation
0001111	Reset register

이러한 레지스터를 설정하기 위해서는 SPI 모드를 통해 할 수 있습니다. 이 작업을 위한 핀의 타이밍 정보를 다음에서 보여주고 있습니다.



[그림 3-26] Audio Codec Register Setting

7) 회로도



8) 핀 구성표

FPGA Signal	Altera	Xilinx	Description
AIC_CS	M23	P26	Latch/Address Select
AIC_MODE	M22	P25	Serial Interface Mode
AIC_SDI	F15	F14	Serial Data input
AIC_SCLK	F14	E26	Serial Data clock
AIC_MCLK	F21	J20	External clock
AIC_LRCOUT	N23	R21	ADC Clock signal
AIC_DOUT	F13	E24	Serial data output
AIC_LRCIN	E20	E21	DAC Clock signal
AIC_DIN	N24	R22	Serial Data input
AIC_BCLK	V21	V17	Serial bit clock

3.22 확장 포트

1) 구성

HBE-Combo III 장비에서는 기본으로 보드에 장착되어 제공하는 장치 이외에 별도의 확장 모듈을 장착하여 사용할 수 있는 확장 커넥터를 두고 있습니다. 이 커넥터를 HBE-Combo III에 대한 전용 확장 모듈의 장착 및 다른 별도의 장치와 연결하기 용이한 50 핀 2.54 피치의 커넥터를 이용하였습니다. 따라서 50핀 플렛 케이블을 통한 연결에 용이하고, 양면 기판에 커넥터를 조립하여 장비에 장착할 수 있도록 지원해 주고 있습니다. 또한 이 커넥터를 통해 외부로 전원도 공급할 수 있습니다. 확장 커넥터를 통해 5V 및 12V의 전원을 외부로 공급해 주기 때문에 확장해서 사용시 따로 전원을 공급 받을 필요가 없습니다.

다음의 표를 통해 커넥터를 핀 정보를 확인할 수 있습니다.

2) 핀 구성표

❑ Expansion port #1

FPGA Signal	Altera	Xilinx	Description
EXT1[1]	-	-	+5V
EXT1[2]	-	-	+5V
EXT1[3]	-	-	+12V
EXT1[4]	-	-	+12V
EXT1[5]	A20	A22	FPGA I/O
EXT1[6]	A19	A20	FPGA I/O
EXT1[7]	A18	A19	FPGA I/O
EXT1[8]	A17	A18	FPGA I/O
EXT1[9]	A14	A15	FPGA I/O
EXT1[10]	B15	B20	FPGA I/O
EXT1[11]	B14	B19	FPGA I/O
EXT1[12]	A23	B18	FPGA I/O
EXT1[13]	A22	B17	FPGA I/O
EXT1[14]	A21	B15	FPGA I/O
EXT1[15]	B20	C17	FPGA I/O
EXT1[16]	B19	C16	FPGA I/O
EXT1[17]	B18	C15	FPGA I/O
EXT1[18]	B17	B23	FPGA I/O
EXT1[19]	B16	B21	FPGA I/O
EXT1[20]	C16	C23	FPGA I/O

EXT1[21]	C15	C22	FPGA I/O
EXT1[22]	B23	C21	FPGA I/O
EXT1[23]	B22	C20	FPGA I/O
EXT1[24]	B21	C18	FPGA I/O
EXT1[25]	C23	D18	FPGA I/O
EXT1[26]	C22	D17	FPGA I/O
EXT1[27]	C21	D16	FPGA I/O
EXT1[28]	C19	C26	FPGA I/O
EXT1[29]	C17	C25	FPGA I/O
EXT1[30]	D18	D24	FPGA I/O
EXT1[31]	D17	D23	FPGA I/O
EXT1[32]	D16	D22	FPGA I/O
EXT1[33]	D15	D21	FPGA I/O
EXT1[34]	D14	D20	FPGA I/O
EXT1[35]	E18	E17	FPGA I/O
EXT1[36]	E15	E15	FPGA I/O
EXT1[37]	D21	E14	FPGA I/O
EXT1[38]	D20	D26	FPGA I/O
EXT1[39]	D19	D25	FPGA I/O
EXT1[40]	F16	F15	FPGA I/O
EXT1[41]	F15	F14	FPGA I/O
EXT1[42]	F14	E26	FPGA I/O
EXT1[43]	F13	E24	FPGA I/O
EXT1[44]	E20	E21	FPGA I/O
EXT1[45]	-	-	GND
EXT1[46]	-	-	GND
EXT1[47]	-	-	GND
EXT1[48]	-	-	GND
EXT1[48]	-	-	GND
EXT1[50]	-	-	GND

❑ Expansion port #2

FPGA Signal	Altera	Xilinx	Description
EXT2[1]	-	-	+5V
EXT2[2]	-	-	+5V
EXT2[3]	-	-	+12V
EXT2[4]	-	-	+12V

EXT2[5]	F17	F23	FPGA I/O
EXT2[6]	G18	F22	FPGA I/O
EXT2[7]	G17	F20	FPGA I/O
EXT2[8]	G16	F19	FPGA I/O
EXT2[9]	F23	F17	FPGA I/O
EXT2[10]	F21	G15	FPGA I/O
EXT2[11]	J22	F25	FPGA I/O
EXT2[12]	K19	F24	FPGA I/O
EXT2[13]	J26	J21	FPGA I/O
EXT2[14]	J25	J20	FPGA I/O
EXT2[15]	J24	L22	FPGA I/O
EXT2[16]	J23	M21	FPGA I/O
EXT2[17]	K25	M20	FPGA I/O
EXT2[18]	K24	M19	FPGA I/O
EXT2[19]	K23	M18	FPGA I/O
EXT2[20]	K22	L24	FPGA I/O
EXT2[21]	K21	N17	FPGA I/O
EXT2[22]	L23	M26	FPGA I/O
EXT2[23]	L21	M25	FPGA I/O
EXT2[24]	L20	M23	FPGA I/O
EXT2[25]	L19	M22	FPGA I/O
EXT2[26]	K26	N24	FPGA I/O
EXT2[27]	M21	N21	FPGA I/O
EXT2[28]	M20	N20	FPGA I/O
EXT2[29]	M19	N19	FPGA I/O
EXT2[30]	L25	N18	FPGA I/O
EXT2[31]	L24	P23	FPGA I/O
EXT2[32]	N20	P22	FPGA I/O
EXT2[33]	M25	P21	FPGA I/O
EXT2[34]	M24	P20	FPGA I/O
EXT2[35]	M23	P18	FPGA I/O
EXT2[36]	M22	R19	FPGA I/O
EXT2[37]	N24	R18	FPGA I/O
EXT2[38]	N23	R17	FPGA I/O
EXT2[39]	N21	P26	FPGA I/O
EXT2[40]	V21	P25	FPGA I/O
EXT2[41]	V20	R22	FPGA I/O
EXT2[42]	AA23	R21	FPGA I/O
EXT2[43]	Y26	R20	FPGA I/O

EXT2[44]	Y25	V17	FPGA I/O
EXT2[45]	-	-	GND
EXT2[46]	-	-	GND
EXT2[47]	-	-	GND
EXT2[48]	-	-	GND
EXT2[48]	-	-	GND
EXT2[50]	-	-	GND

3.23 AMBA Connector

장비에서는 ARM Processor와 FPGA 디바이스간의 AMBA 버스를 통한 인터페이스를 통해 ARM Processor에서 HBE-Combo III의 내부 장치를 제어할 수 있도록 할 수 있습니다. 현재 이 AMBA 버스 지원을 위해 FPGA는 ARM Processor간의 핀이 할당 되어 있고 여기에서 이 핀에 대한 정보를 확인해 보도록 하겠습니다.

FPGA Signal	Altera	Xilinx	Description
H_ADDR[0]	A5	A4	
H_ADDR[1]	B8	B7	
H_ADDR[2]	B9	B8	
H_ADDR[3]	A10	B1	
H_ADDR[4]	AF21	AD1	
H_ADDR[5]	B12	B12	
H_ADDR[6]	A4	A3	
H_ADDR[7]	AF22	AD2	
H_ADDR[8]	A9	A12	
H_ADDR[9]	AF17	AC8	
H_ADDR[10]	AE17	AF20	
H_ADDR[11]	AF20	AC12	
H_ADDR[12]	AE22	AC3	
H_ADDR[13]	B7	B6	
H_ADDR[14]	AE18	AF23	
H_ADDR[15]	AD21	AE23	
H_ADDR[16]	AE23	AC6	
H_ADDR[17]	AF19	AC11	
H_ADDR[18]	AE21	AC2	
H_ADDR[19]	AF18	AC9	
H_ADDR[20]	AC22	AE17	
H_ADDR[21]	AD22	AE25	
H_ADDR[22]	AE19	AF25	
H_ADDR[23]	AE16	AF19	
H_ADDR[24]	AE20	AC1	
H_ADDR[25]	AC17	AD20	
H_ADDR[26]	AD15	AE18	
H_ADDR[27]	AD19	AE21	
H_ADDR[28]	AE15	AF18	
H_ADDR[29]	AD23	AE26	
H_ADDR[30]	AB20	AC26	

H_ADDR[31]	AC21	AD26	
H_DATA[0]	G10	F7	
H_DATA[1]	G12	F12	
H_DATA[2]	F2	H6	
H_DATA[3]	D10	D13	
H_DATA[4]	F11	F3	
H_DATA[5]	D2	G9	
H_DATA[6]	D9	D11	
H_DATA[7]	F9	E12	
H_DATA[8]	F10	F2	
H_DATA[9]	C12	D3	
H_DATA[10]	D5	D6	
H_DATA[11]	D12	E3	
H_DATA[12]	E8	E4	
H_DATA[13]	C8	C8	
H_DATA[14]	C7	C7	
H_DATA[15]	D11	E1	
H_DATA[16]	D8	D10	
H_DATA[17]	B11	B10	
H_DATA[18]	D7	D9	
H_DATA[19]	B5	B3	
H_DATA[20]	B10	B9	
H_DATA[21]	C10	C11	
H_DATA[22]	B6	B4	
H_DATA[23]	D6	D8	
H_DATA[24]	C11	C12	
H_DATA[25]	C9	C10	
H_DATA[26]	A7	A9	
H_DATA[27]	A8	A10	
H_DATA[28]	C4	C5	
H_DATA[29]	C6	C6	
H_DATA[30]	A6	B2	
H_DATA[31]	B4	A8	
H_RESET	F6	H10	
DMA_REQ0	G3	J6	
DMA_CLR0	G4	J7	
DMA_TC0	E5	H1	
DMA_REQ1	F4	H9	
DMA_CLR1	C3	G4	

DMA_TC1	F1	H2	
DMA_REQ2	E2	G12	
DMA_CLR2	G9	F5	
DMA_TC2	F3	H7	
DMA_REQ3	D1	G6	
DMA_CLR3	C2	G3	
DMA_TC3	F12	F4	
INT0	E1	G10	
INT1	G11	F8	
INT2	E12	E10	
INT3	E10	E7	
SPLIT0	Y16	AB26	
SPLIT1	AB25	AA17	
SPLIT2	AD24	AB15	
SPLIT3	AB26	AA18	
MASTER0	AA16	AC20	
MASTER1	AC26	AA25	
MASTER2	AA15	AC19	
MASTER3	Y15	AB24	
H_SEL	Y14	AB23	
SLAVE_READYIN	AB18	AC25	
SLAVE_READYOUT	AA14	AC16	
MASTER_READY	AC25	AA24	
GRANT	AA20	AC23	
BUSREQ	Y13	AB18	
MASTLOCK	AC23	AA22	
LOCK	AC14	AD15	
PROT0	AA18	AC22	
PROT1	AB15	AC24	
PROT2	AC24	AA23	
PROT3	AA13	AC15	
TRANS0	AD25	AB16	
TRANS1	AC19	AD22	
RESP0	Y18	AC14	
RESP1	AC15	AD17	
BURST0	AD16	AE19	
BURST1	AC20	AD25	
BURST2	AB21	AD14	
WRITE	AA17	AC21	

SIZE0	AD17	AE20	
SIZE1	AC18	AD21	
SIZE2	AC16	AD19	
AHB_CLK	AF14		
EASIC_PI_CLK	AA25		

04

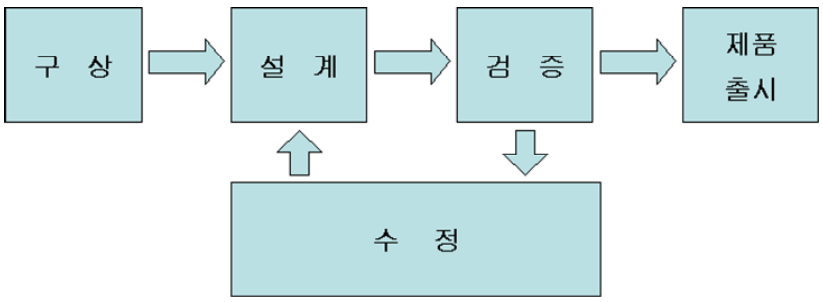
HBE-COMBO II
User's Manual &
Lab Guide

설계 S/W 기본사용법

4. 설계S/W 기본사용법

4.1 Design Flow

일반적으로 어떤 제품을 개발할 때, 아래 그림과 같은 과정으로 진행합니다.

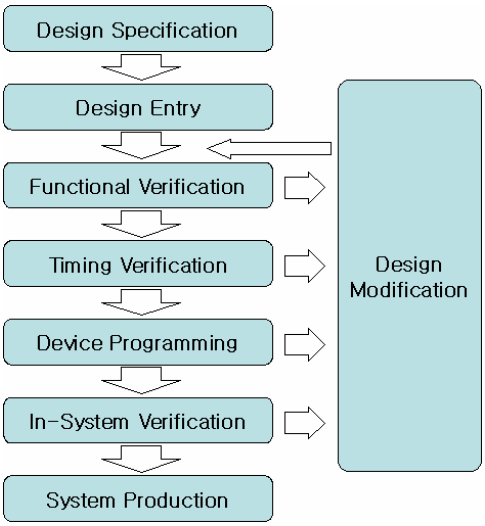


[그림 4-1] 제품 생산의 흐름

어떤 제품을 만들 것인지를 생각하고 (구상), 그 물건을 만들고 (설계), 그 물건이 의도한 대로 만들어 졌는지 테스트 (검증)합니다. 만약에 이상이 발견되면 다시 수정하여 검증하고, 이상이 없다면 제품을 출시하는 과정을 거칩니다.

하나의 시스템을 설계 하는 것도 마찬가지인데, [그림 4-2]는 설계 소프트웨어를 이용해서 시스템을 설계하는 과정입니다.

먼저 Design Specification에서는 어떤 논리 회로를 설계할 것인가를 구상합니다. 구체적으로 어떤 입력을 받아 어떻게 동작하여 어떤 핀으로 출력을 할 것 인지와 같은 내용을 구상하는 것입니다.



[그림 4-2] 시스템 설계 시 설계 흐름

그리고, Design Entry에서 설계하고자 하는 논리 회로를 어떤 방법으로 어떻게 설계할 것인가를 결정하여 논리 회로를 설계합니다. 여기에서 어떤 방법에 해당하는 논리 회로를 설계하는 방법은 여러 가지 있는데, Graphic Design (Schematic Capture에 의한 방법)으로 설계하는 방법, HDL (Hardware Description Language: 하드웨어 기술 언어)를 사용하여 설계하는 방법, 파형으로 설계하는 방법이 그것입니다.

위에서 설계한 내용에 대해 문법을 검사하여 이상이 있다면 그 이상에 대한 메시지 등을 알려주고, 이상이 없다면 설계한 논리 회로를 하드웨어적으로 만들거나 시뮬레이션 할 수 있는 파일 또는 프로그래밍 할 수 있는 파일 등을 생성하는 부분이 Compilation입니다.

이렇게 문법 오류까지 검증된 논리 회로는 설계한 의도대로 동작이 잘 되는지에 대한 검증 과정을 거치게 됩니다. 먼저 기능적으로 문제가 없는지를 살펴보는 Function Verification, 그 후에 직접 디바이스의 특성을 이용하여 설계한 회로가 사용하고자 하는 디바이스나 클럭에 따른 오동작을 하지 않는지를 살펴보는 Timing Verification 과정을 진행합니다.

위의 Function Verification과 Timing Verification은 PC상에서 소프트웨어로 검증하는 방법이고, 이렇게 검증된 논리 회로가 직접 하드웨어 (시스템)에 연결되었을 때 정확하게 동작이 되는지를 검증해야 할 필요가 있습니다. 이 부분이 Device Programming과 In-System Verification입니다. 전자는 말 그대로 디바이스에 설계한 논리 회로를 프로그래밍 한다는 말이고, 후자는 시스템이 꾸며진 상태에서 하드웨어 테스트를 한다는 것입니다.

이렇게 모든 검증이 끝난 후에 비로서 제품이 완성되어 제조됩니다.

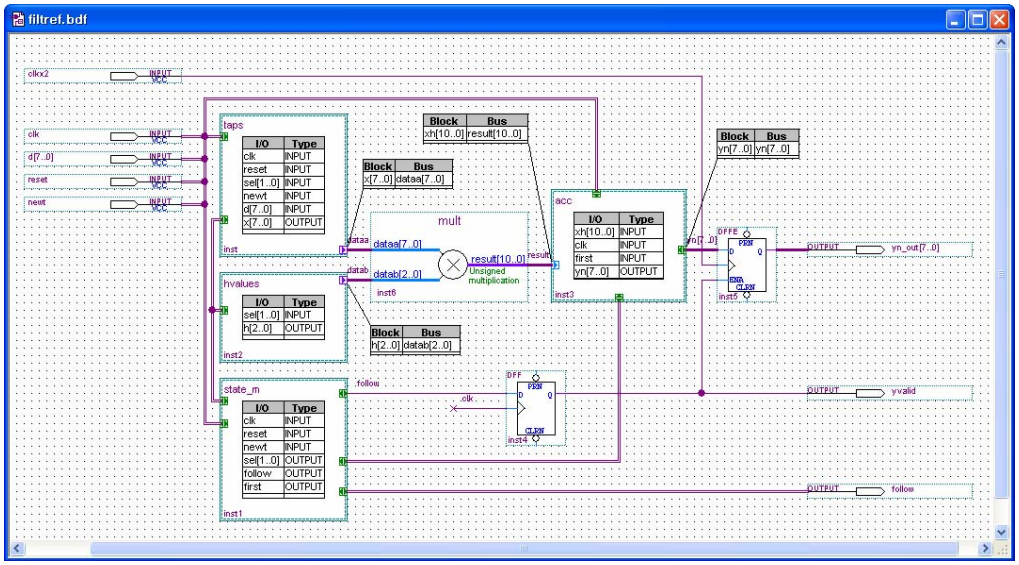
4.2 설계 소프트웨어

4.2.1 Quartus II를 이용한 설계

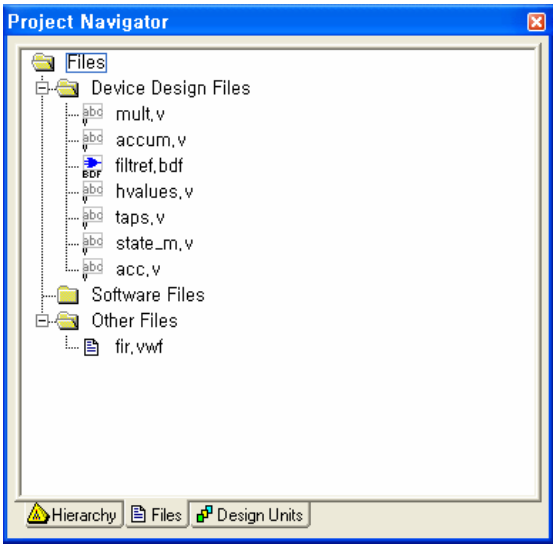
1) 역할

Altera Quartus II 프로그램은 모든 설계를 프로젝트 중심으로 관리합니다. 프로젝트에 포함된 설계 파일 및 컴파일 한 파일, 환경저장 파일, 시뮬레이션 파일, 프로그래밍 파일이 모두 같은 프로젝트 이름으로 관리가 됩니다.

또한 모든 설계를 프로젝트화 하였기 때문에 전체 프로젝트에 대한 관리도 각각의 프로젝트 별로 관리하게 되어 있습니다. [그림 4-3]은 Quartus II에서 기본으로 제공해 주는 예제 중 Fir-filter라는 예제이고, [그림 4-4]는 이 Fir-filter 프로젝트에 대한 Hierarchy Display로, Fir-filter에 대한 하위 블록들을 계층구조로 보여주는 것입니다. [그림 4-4]와 같이 프로젝트에 포함된 설계 파일 및 하위 파일들은 쉽게 파악할 수 있습니다. 또한 이러한 파일들은 Setting 메뉴를 통해 파일 뿐만 아니라 프로젝트에 대한 전반적인 관리를 쉽게 하도록 구성되어 있습니다.



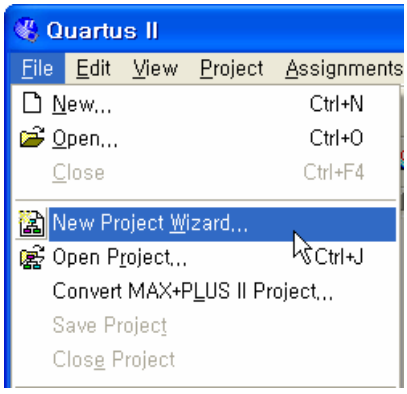
[그림 4-3] Fir-filter Project



[그림 4-4] Hierarchy Display

2) Project 선언

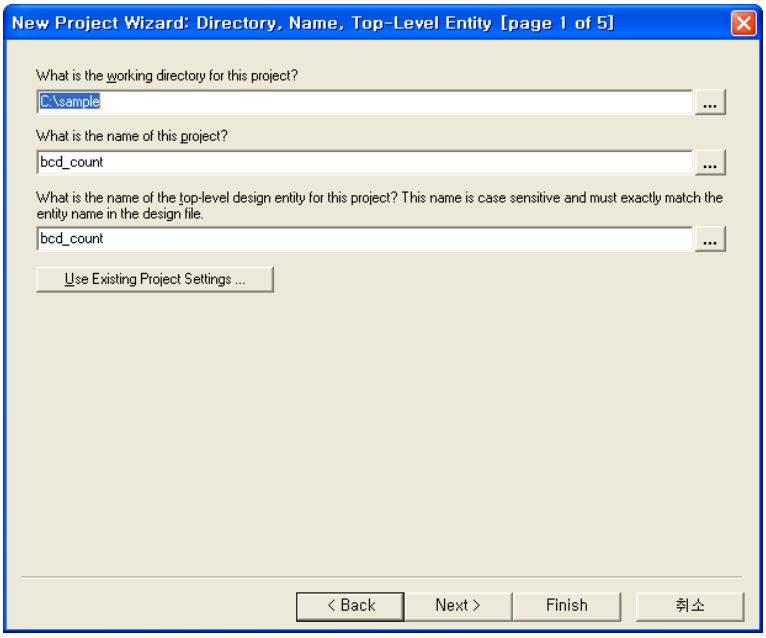
[그림 4-5]와 같이 Quartus II 프로그램은 File -> New Project Wizard 메뉴를 선택해서 현재 작업하려는 프로젝트를 생성할 수 있습니다. 프로젝트 선언은 프로젝트 명 선언에서부터 디바이스 선택 등의 총 5단계로 이루어져 있는데, 이 과정을 거쳐야 실제 코딩을 위한 프로젝트 선언 과정이 완료됩니다.



[그림 4-5] New Project Wizard

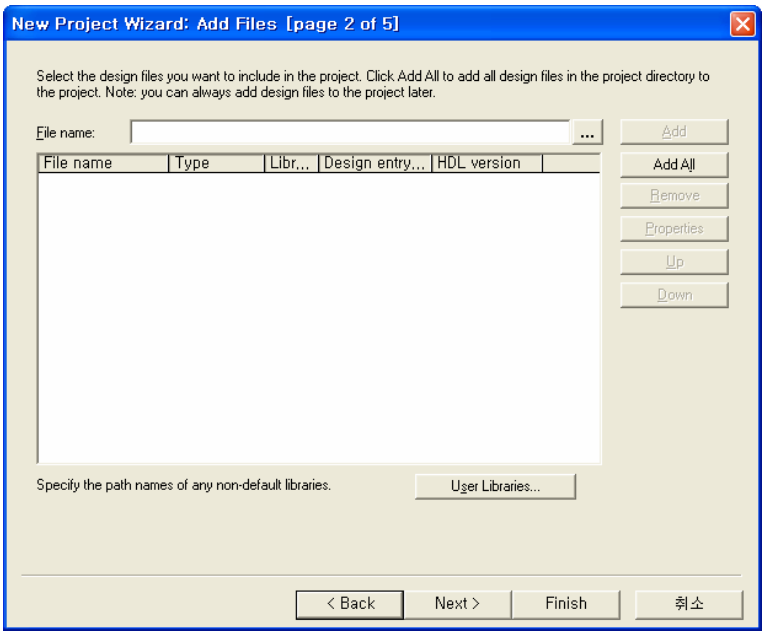
각 단계에서 다음 단계로 진행하기 위해서는 Next 버튼을 클릭해야 합니다. 프로젝트 선언을 시작하면 프로젝트에 대한 전반적인 진행 사항을 설명하는 창이 활성화되고 Next 버튼에 의해 다음 프로젝트 선언을 시작하게 됩니다.

첫 단계로 프로젝트 관련 파일들이 저장될 Directory, Project, Top-Level의 Entity 이름을 설정 합니다. 이때 주의할 사항은 VHDL등의 Language로 작성할 경우 코드의 Entity 이름과 현재 저장하는 Top-Level의 Entity의 이름을 같게 해 주어야 합니다. [그림 4-6]에서는 프로젝트의 이름과 Entity의 이름을 동일하게 counter로 설정해 주었고, 이에 관련된 파일들이 C:₩sample에 저장되도록 설정했습니다.



[그림 4-6] Directory, Name, Top-Level Entity 설정

다음으로 다른 프로젝트 파일에서 필요한 파일들을 추가하거나, 그 프로젝트의 모든 파일들을 참조하기 위한 Library Path를 설정해 주는 부분입니다. 이 부분은 여러 사람들이 함께 작업하는 설계과정에서 유용하게 사용됩니다. 따라서 여러 사람들이 각각의 프로젝트를 나누어 설계하고, 최종 top 프로젝트에서 하나로 모아 설계할 때, [그림 4-7]은 파일들을 Add 버튼에 의해서 필요한 파일들을 프로젝트에 추가하게 됩니다.



[그림 4-7] Add Files

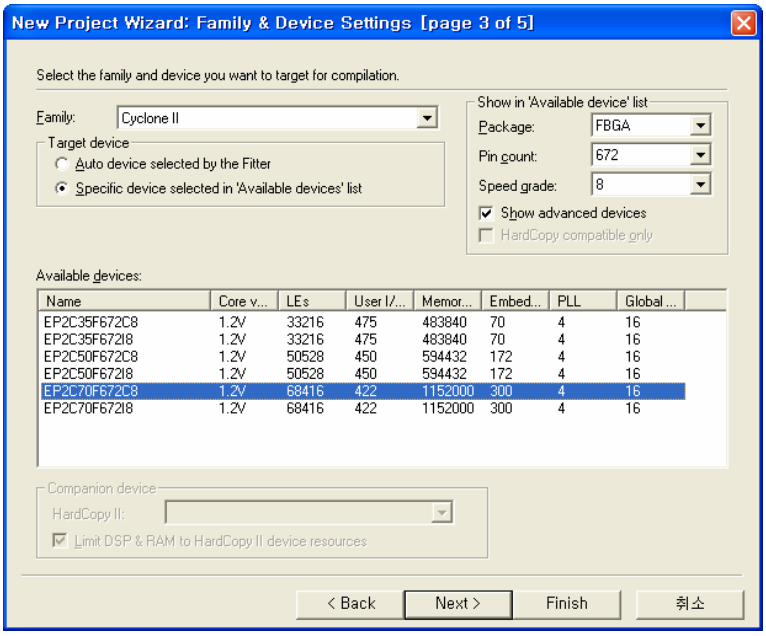
다음으로 Family & Device Settings 설정 부분입니다. Device Family는 사용하는 디바이스가 RAM 또는 ROM 타입이냐에 따라 FPGA, CPLD로 구분 지을 수 있고, 세부적으로 Device 용량 및 디바이스가 가지고 있는 핀 수, 처리 속도에 따라 나누어 집니다. 종류를 보면, Stratix II, Stratix, Cyclone II, Cyclone, APEX II, APEX 20K, & FLEX 10K의 수백만 게이트 단계까지 설계가 가능하고, SoC라는 하나의 디바이스에서 모든 구현을 해 볼 수 있는 제품도 있습니다.

또한 이러한 디바이스에서 주로 사용하는 기능만으로 구성되어 있는 Cyclone II, Cyclone, APEX 1K 가 있습니다. 또한 고속 통신이 가능한 Stratix GX, Mercury가 있고, CPLD로 MAX Series가 있습니다. 이렇게 다양한 Device Family를 Tool에서 선택할 수 있습니다. [그림 4-8]에서는 현재 HBE-Combo III 장비에서 사용하는 Family 군인 cyclone II를 선택한 것을 볼 수 있습니다.

그리고 밑에 보이는 Target device에서의 체크하는 부분은 현재 결정한 Device Family 내에 있는 여러 디바이스 중 타겟 디바이스를 선택해서 프로젝트에 적용할 것인지, 아니면 컴파일러가 디바이스 중 적당한 디바이스를 자동으로 선택해서 프로젝트에 적용할 것인지를 선택하는 작업입니다.

예를 들어 사용자가 타겟 디바이스를 가지고 설계를 할 때, Target device는 Specific device selected in Available device list 에 체크를 해 주고 가지고 있

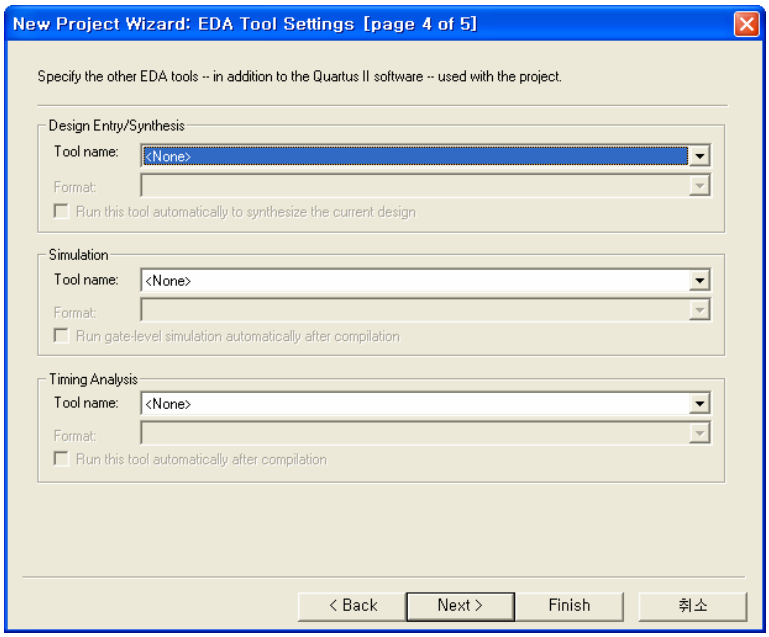
는 디바이스를 선택해 주면 됩니다. 또한 오른 쪽 화면에 있는 Filters 부분은 Family 내에 있는 여러 디바이스 중, 디바이스의 Package, Pin count, Speed grade에 따라서 디바이스를 검색해 주는 작업을 하는 부분입니다. 따라서 타겟 디바이스를 쉽게 찾아 선택할 수 있게 도와주는 부분입니다. 아래의 [그림 4-8]에서는 현재 HBE-Combo III 에서 사용하는 EP2C70F672C8을 선택한 모습을 보여주고 있습니다.



[그림 4-8] Family & Device Settings

Quartus II는 기본적으로 설계, 합성 및 시뮬레이션 작업을 할 수 있도록 기능을 포함하고 있습니다. 하지만 Quartus II가 PLD 지원을 목적으로 만들어진 설계 프로그램이기 때문에 이러한 개개의 작업에서의 최대의 성능을 지원해 준다고는 말하기 힘듭니다.

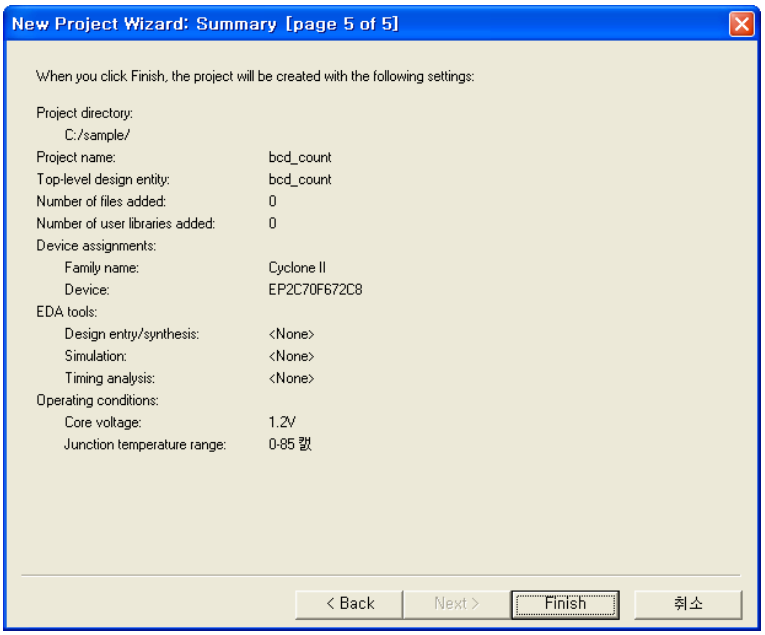
따라서 여기에서는 이러한 점을 보완하기 위해 다른 전문적인 툴과 연동해서 작업이 가능합니다. 이러한 작업을 위해 [그림 4-9]의 창에서 이러한 툴의 연동을 시킬 수 있습니다. 따라서 현재 사용자의 컴퓨터에서 연동하려는 다른 프로그램이 설치되어 있다면 여기에서 설정을 해 주면 됩니다.



[그림 4-9] EDA Tool Settings

여기에서는 현재까지의 프로젝트 설정 단계에서 각 단계에 대한 설정이 어떻게 되었는지 모두 보여주게 됩니다. 따라서 사용자가 이러한 정보를 보고 프로젝트 선언 단계에서 다르게 설정된 부분이 없는지 다시 한번 확인하는 단계가 되겠습니다.

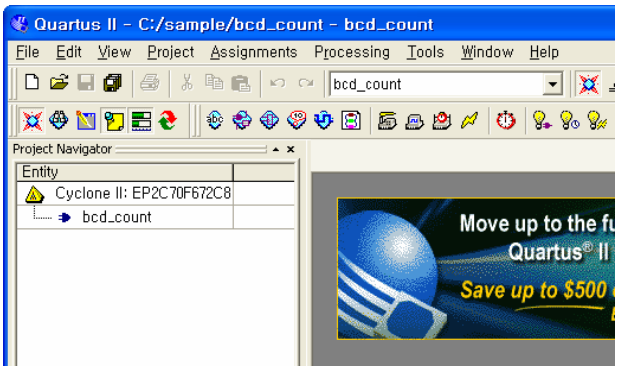
따라서 이러한 확인 절차에 의해 잘못된 부분은 Back 버튼에 의해 그 단계로 가서 수정을 하고, 이상이 없을 시 Finish 버튼에 의해 프로젝트 설정 단계를 마치면 됩니다. 이러한 설정 부분은 프로젝트 설정을 마치더라도 Setting 메뉴를 통해 재 수정이 가능합니다. 아래 [그림 4-10]에서는 Summary 창을 보여주고 있습니다.



[그림 4-10] Summary

이전까지는 작업으로 프로젝트가 선언되었습니다. 이로써 설계할 수 있는 환경이 만들어진 것입니다.

이렇게 프로젝트를 선언하면 [그림 4-11]과 같이 Quartus II 프로그램의 왼쪽 상단의 타이틀 바에 현재 프로젝트가 선언된 경로명 및 프로젝트의 이름이 표시됩니다.

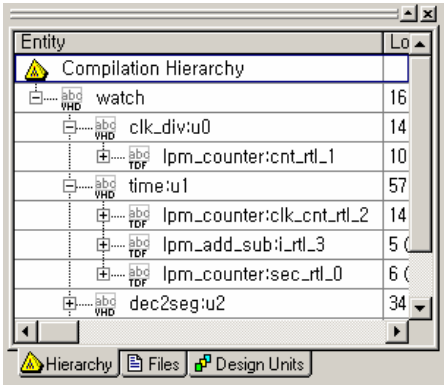


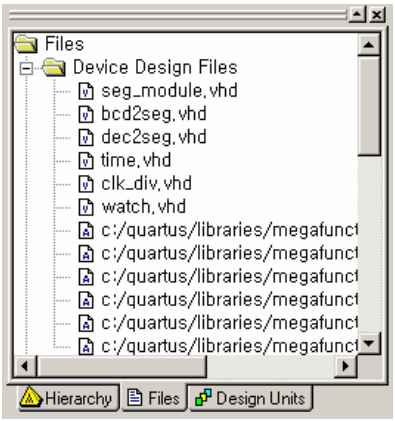
[그림 4-11] 타이틀에 표시된 프로젝트 이름

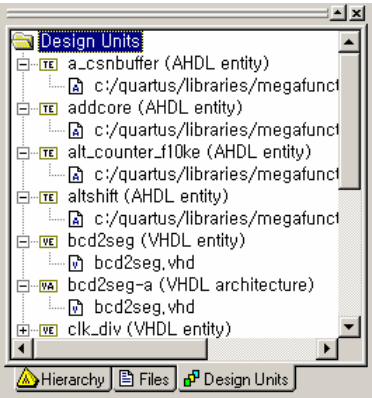
작업을 진행하는 중에 Project Navigator라고 해서 프로젝트 내에 어떤 형태로 구성되어 있는지 알아볼 수 있는데 이것은 크게 3가지로 구분됩니다. 프로젝트의 계층구조를 보여주고, Top-Level Design File이라고 해서 최종 Design을 확인할 수 있는 Hierarchy라는 것이 있습니다. 또한 포함되어있는 파일들(시뮬레이션 파일, 프로그래

밍 파일, 하위 블록 및 각 블록에 포함된 파일)을 쉽게 확인하고 관리할 수 있는 것이 Files입니다. 마지막으로 Design Unit 의 type 세부사항을 Design Units에서 확인해볼 수 있습니다.

[그림 4-12]에서 맨 위에 있는 것이 Hierarchy 이고, 중앙에 Files, 아래에 Design Units 을 보여주고 있습니다. 이것을 통해 프로젝트의 계층구조를 파악해볼 수 있습니다.







[그림 4-12] Project Navigator

3) Design Entry

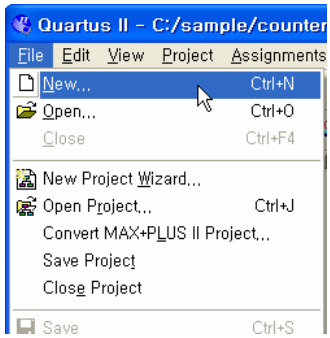
❑ Graphic Editor (Schematic Capture에 의한 설계)

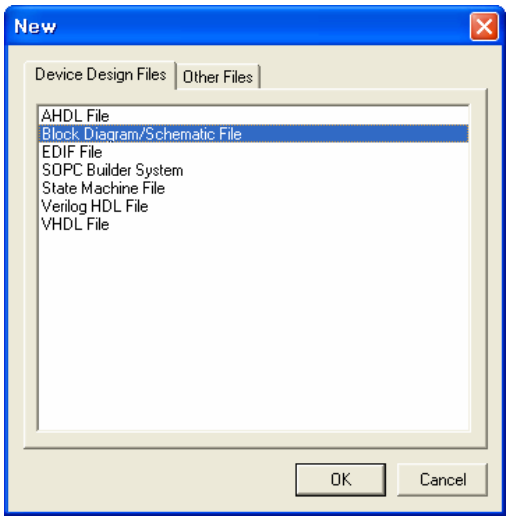
① 특징

특정 기능을 가진 심볼 라이브러리를 생성하여, 그 심볼 라이브러리의 입 출력 데이터 라인을 선 (wire) 등으로 각 심볼 등의 입 출력 데이터 신호를 연결하여 논리 회로를 설계하는 방법입니다.

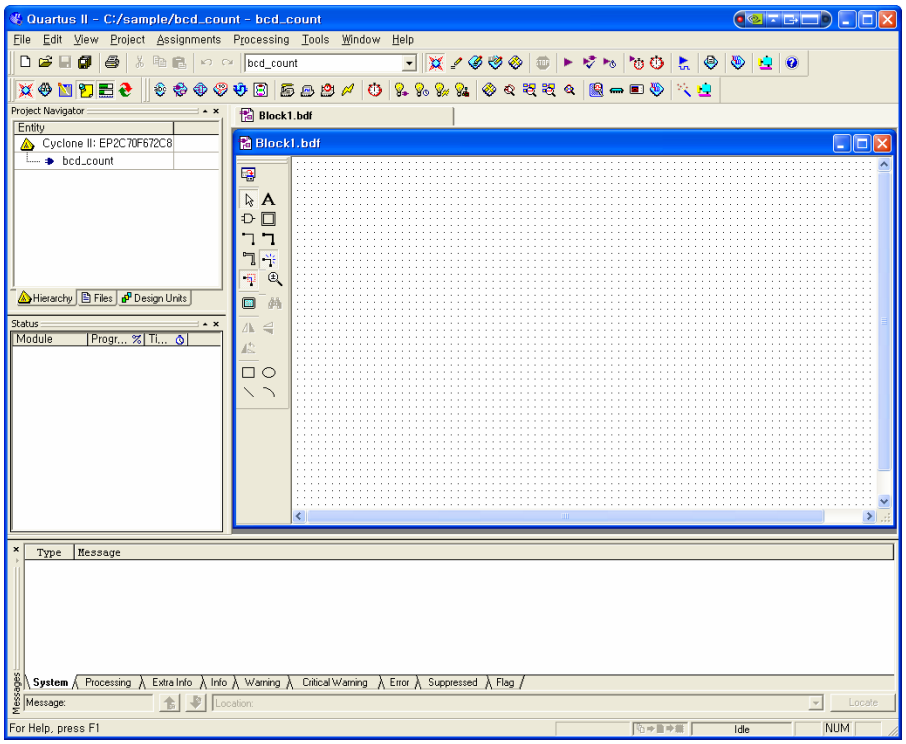
② Graphic Editor 선택하기

Graphic Editor를 이용해 하나의 논리 회로를 설계하기 위해서는 Graphic Editor 창을 활성화 시켜야 하는데, [그림 4-13]과 같이 File -> New 메뉴를 선택하여 New 창을 활성화 한 후 Block Diagram/Schematic File이라는 항목을 선택하여 OK 버튼을 누르면 [그림 4-14]와 같이 Graphic Editor 창이 활성화 됩니다.





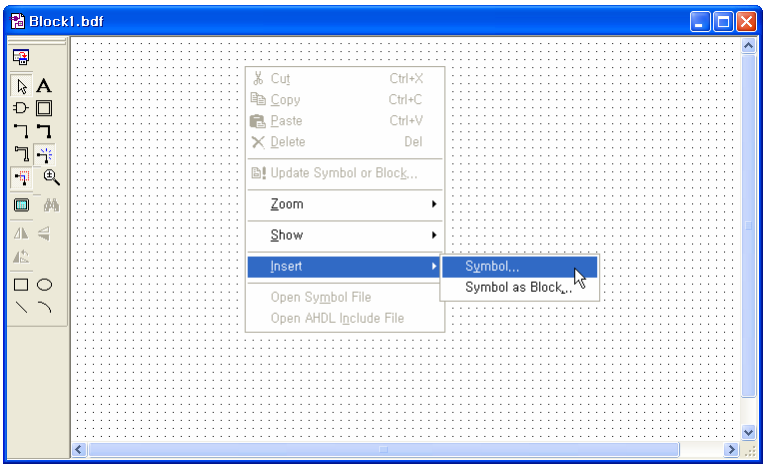
[그림 4-13] 설계 메뉴에서 Block Diagram/Schematic File 선택



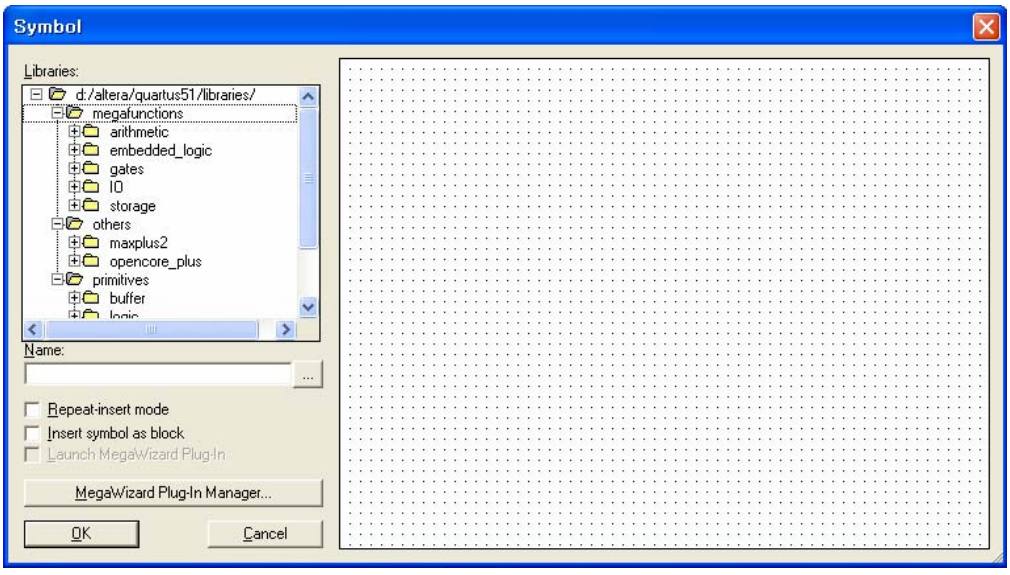
[그림 4-14] Block Diagram/Schematic File을 활성화 한 상태

③ 심볼 라이브러리

Quartus II의 Block Diagram/Schematic File에서는 도면을 마우스 왼쪽 버튼으로 더블-클릭하는 방법 또는 마우스의 오른쪽 버튼을 눌러 Pop-Up 메뉴의 Insert -> Symbol 항목을 선택하는 방법과 Quartus II 메뉴의 Edit -> Insert Symbol 을 통해 라이브러리를 불러오는 방법이 있습니다.



[그림 4-15] Pop-up 메뉴에서 "Insert Symbol" 선택



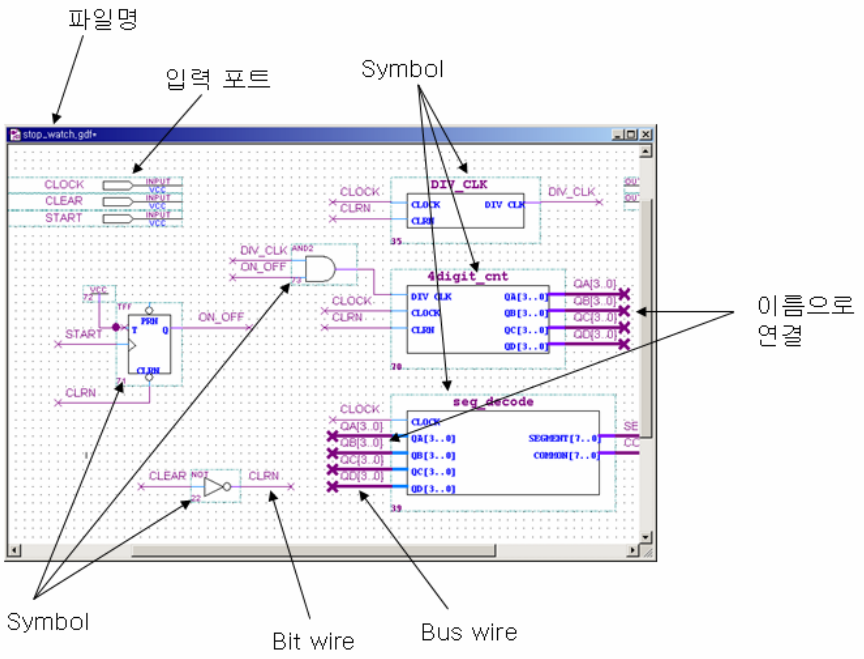
[그림 4-16] Insert Symbol 창

위의 Enter Symbol 창에서 Name에 직접 심볼 명을 쓰거나, 왼쪽의 Libraries에서 해

당 심볼 라이브러리가 들어 있는 폴더에서 찾는 방법으로 Symbol들을 검색 할 수 있습니다. 여기에서는 AND 또는 OR 게이트의 간단한 Symbol에서 약간의 논리가 포함된 74 시리즈, Megafunctions라고 하여 기본 설정과 Parameter 값만 변화시켜 줌으로써 쉽게 설계할 수 있도록 해주는 라이브러리입니다. 이처럼 일부 항목을 바꾸어서 범위가 큰 논리 회로를 쉽게 설계할 수 있도록 해줍니다.

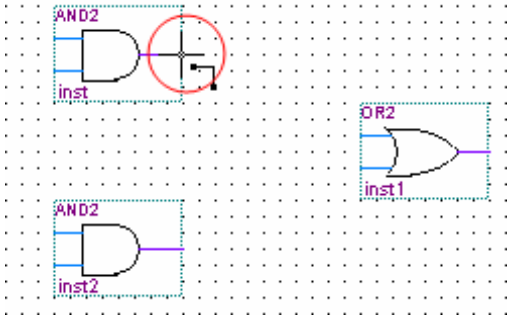
④ **각 Symbol의 연결**

각 Symbol 간의 데이터 연결은 wire 또는 이름으로 연결해 줄 수 있습니다. [그림 4-17]은 stop-watch 예제를 보여주고 있다. 각 Symbol 라이브러리 간의 bus 데이터를 연결할 때는 선 굵기가 굵은 bus wire를 사용하고, bit 데이터를 연결할 때는 굵기가 가는 bit wire를 사용합니다.

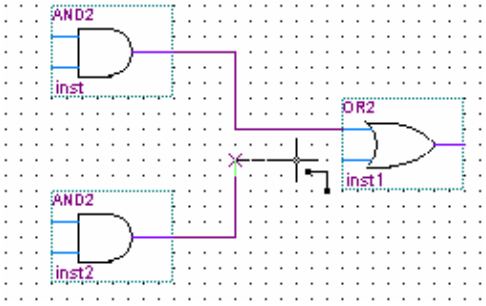


[그림 4-17] Stop-watch 예제에서의 연결

Symbol을 연결하는 방법은 [그림 4-18]과 같이 각 Symbol의 데이터 라인의 연결점을 마우스로 가져가면 마우스의 포인터가 십자 모양으로 바뀝니다. 이 상태에서 마우스의 왼쪽 버튼을 누른 채 드래그 하면 wire가 생성이 되는 데, [그림 4-19]와 같이 wire를 연결하고자 하는 곳까지 드래그 하여 연결하면 됩니다.

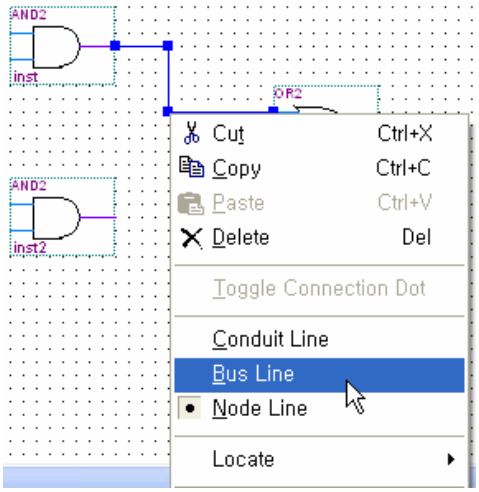


[그림 4-18] 마우스 포인터의 변화



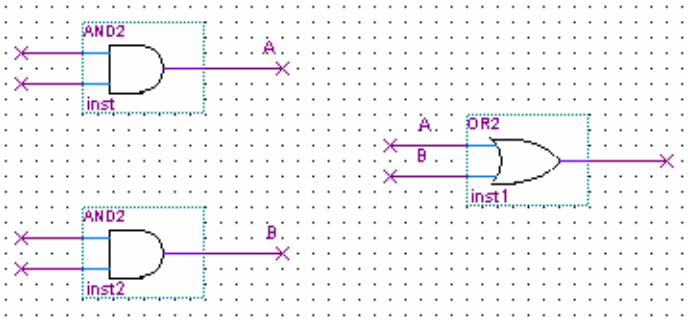
[그림 4-19] wire 연결

bus wire의 경우에는 [그림 4-20] 위에서처럼 wire을 연결한 후, 마우스 왼쪽 버튼의 더블 클릭으로 그 bus wire 전체를 선택 할 수 있습니다. 이 때, 마우스의 오른 쪽 버튼을 눌러 pop-up 메뉴의 Bus Line을 선택하면 됩니다.



[그림 4-20] bus wire로 변환

이름으로 연결할 경우에는 연결하고자 하는 Node를 [그림 4-21]과 같이 Symbol에서 wire을 일정 부분 연결한 후 마우스의 왼쪽 버튼을 이용하여 wire를 선택(클릭)하고 Node 명을 적어 주면 됩니다. 버스 데이터의 경우도 bus wire을 그린 후 이름을 버스의 형태로 적어 줍니다. 이름을 적을 때 4bit의 A라는 이름이 있다면 A[3..0] 또는 A[0..3]로 적어 줍니다. 이 차이는 MSB(상위 비트)의 번호가 4인지 또는 0인지의 차이입니다.

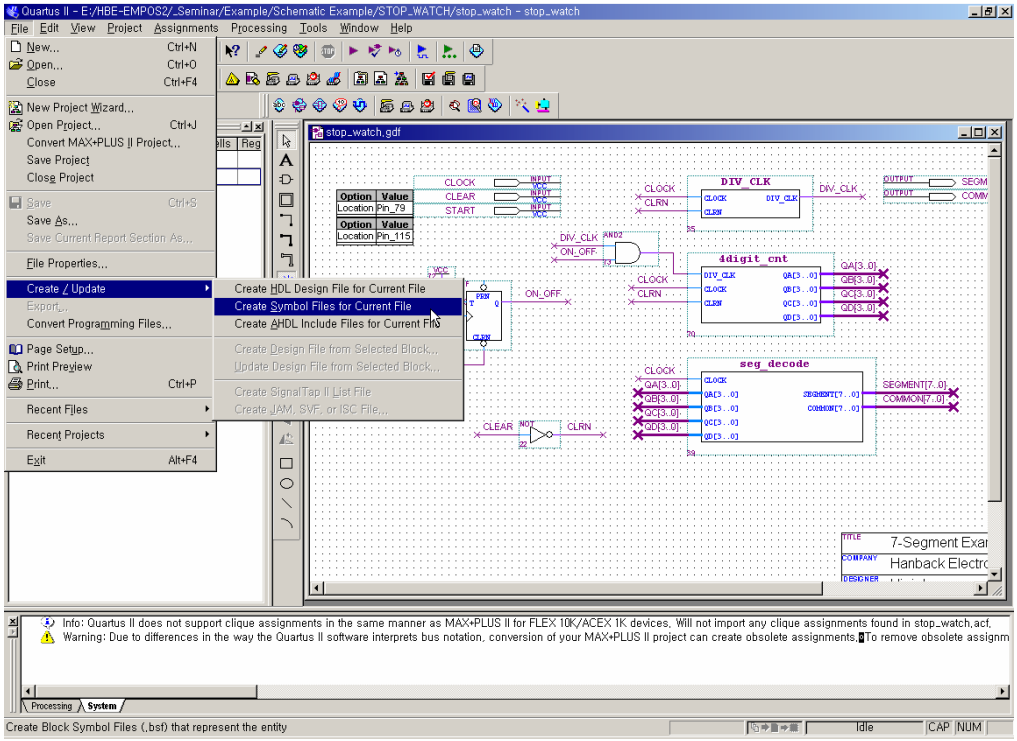


[그림 4-21] 이름으로 연결

⑤ 심볼 생성

사용자가 설계하여 만든 논리 회로를 하나의 블록으로 하여 더 큰 프로젝트에 넣을 수 있습니다. 이렇게 생성한 것은 Graphic Editor 상태에서는 Symbol 형태로 불러오게 됩니다. 이 Symbol 형태를 만들기 위해서는 Graphic Editor를 활성화 시킨 상태에

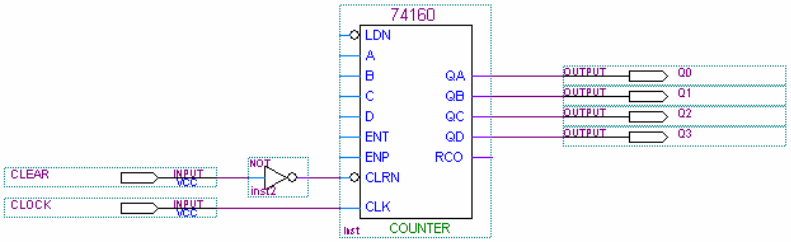
서 File –〉 Create / Update –〉 Create Symbol Files for Current File 메뉴를 선택하여 변화시킵니다. 이 진행 과정은 왼쪽의 상태표시 창에서 확인해볼 수 있습니다.



[그림 4-22] 심볼 생성

⑥ 설계 완성

위의 심블을 이용한 설계 방법을 이용하여 BCD_COUNTER에 대한 설계를 완성할 수 있습니다. 심블은 74160, INPUT, OUTPUT, NOT의 심블을 이용하였습니다.



[그림 4-23] BCD COUNTER

❑ Text Editor (HDL에 의한 설계)

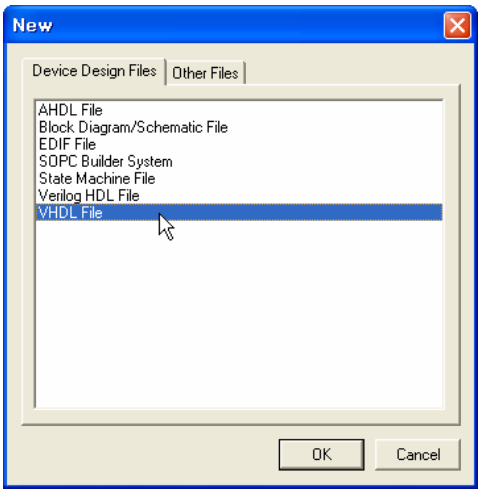
HDL을 이용하여 논리 회로를 설계하는 방법은 각 HDL에 따라 문법을 알아야 하기 때문에 설계 언어에 대한 지식이 필요합니다. 따라서 이 사용자 설명서에서는 가장 기본적으로 Editor 창을 활성화 시키는 방법을 간단한 예제를 보여드리겠습니다.

① 특징

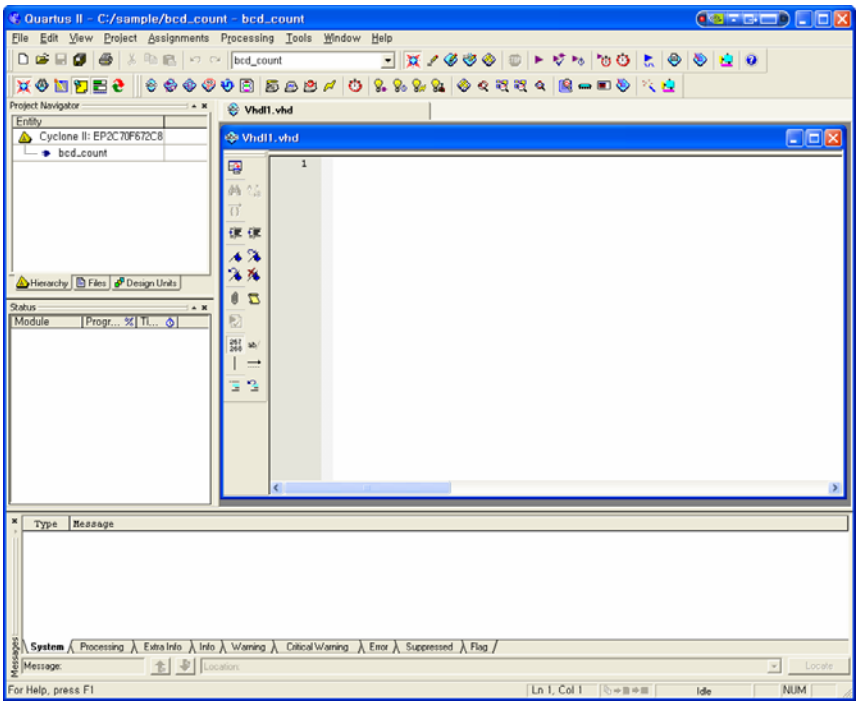
언어를 이용해서 논리 회로를 설계하는 것으로 보통 HDL(Hardware Description Language) 언어를 사용하여 설계합니다. HDL의 종류는 VHDL(Very High-speed integrated circuit HDL), Verilog HDL등이 있는데, Quartus II에서는 Text Editor를 이용하여 HDL을 설계합니다.

② Text Editor 선택하기

Text Editor를 이용하여 하나의 논리 회로를 설계하기 위해서는 Text Editor 창을 활성화 시켜야 하는데, File -> New 메뉴를 선택한 후 [그림 4-24]와 같은 그림이 활성화 되며, 여기에서는 VHDL File을 선택한 것을 보여주고 있다. 이렇게 HDL의 종류를 선택했으면 OK 버튼을 눌러서 Text Editor 창을 활성화 시킨다.



[그림 4-24] New 메뉴에서 Text Editor File 선택



[그림 4-25] Text Editor File을 활성화 한 상태

③ 설계하기

HDL 코드를 사용하여 논리 회로를 설계하기 위해서는 각 HDL (AHDL, VHDL, Verilog HDL 등)의 문법 등에 대해 알아야 할 것이 많기 때문에 간단히 설계한 예만 보여드리겠습니다. 참고로 각각의 설계파일은 다른 확장자(VHDL 파일은 .vhd, AHDL 파일은 .tdf, Verilog HDL 파일은 .v)를 갖는다는 것을 유념해야 합니다.

C:/qdesigns/chiptrip/speed_ch.tdf

```
SUBDESIGN speed_ch
(
    Accel_in, reset, clk    : INPUT;    % File inputs   %
    get_ticket              : OUTPUT    % File output   %
    % File outputs default to GND unless otherwise specified    %
)

VARIABLE
    speed   :   MACHINE             % Create state machine called   %
                OF BITS (q1,q0)     % speed with bits q1 & q0       %
                WITH STATES (

    legal,          % You are driving at a legal speed      %
    warning,        % Police are giving you a warning!      %
    ticket          % Police are giving you a ticket!       %

    );

    gt  :   dff;    % Define flip-flop called gt %
    spd_out :   node;   % Define node called spd_out %

BEGIN
    speed.clk   = clk;      % Connect clock input of speed to INPUT clk     %
    speed.reset = reset;    % Connect reset input of speed to INPUT reset   %
    gt.clk      = clk;      % Connect clock input of gt to INPUT clk        %
    gt.d        = spd_out;  % Connect d input of gt to node spd_out         %
    get_ticket  = gt.q;     % Connect OUTPUT get_ticket to q output of gt   %

    CASE speed IS

    WHEN legal =>

        IF accel_in THEN
            speed = warning;

        ELSE
```

[그림 4-26] AHDL로 설계한 예

G:/jung1/bin/jung1/counter.vhd

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

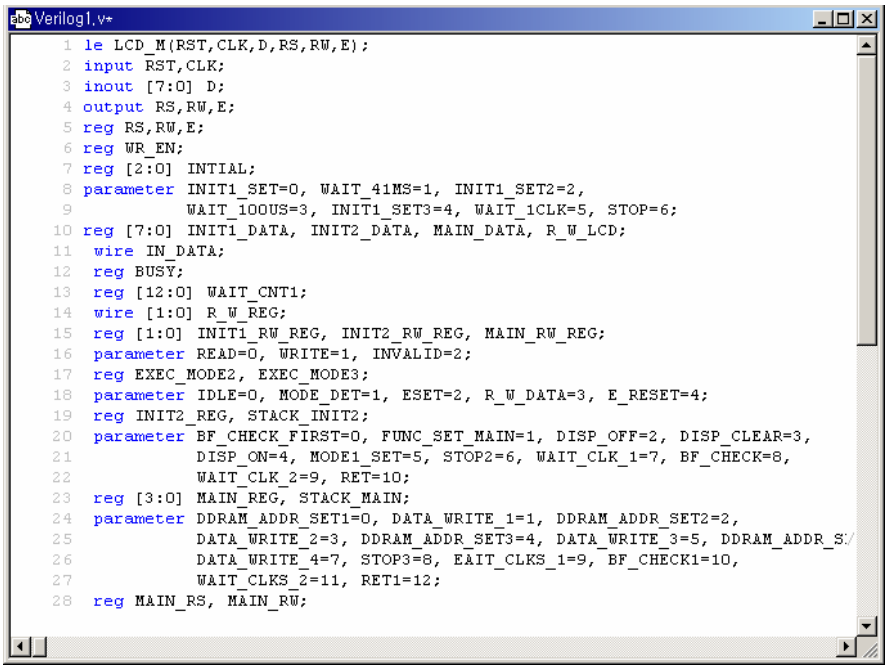
ENTITY COUNTER IS
        PORT(
        CLK     : IN    STD_LOGIC;
        CLEAR   : IN    STD_LOGIC;
        G       : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        SEG7    : OUT STD_LOGIC_VECTOR(7 DOWNTO 0));
END COUNTER;

ARCHITECTURE ARC OF COUNTER IS
    SIGNAL CNT      : INTEGER RANGE 0 TO 5;
    SIGNAL NUM      : INTEGER RANGE 0 TO 15;
    SIGNAL HOUR     : INTEGER RANGE 0 TO 59;
    SIGNAL MIN, SEC : INTEGER RANGE 0 TO 59;
    SIGNAL H10, H1  : INTEGER RANGE 0 TO 9;
    SIGNAL M10, M1  : INTEGER RANGE 0 TO 9;
    SIGNAL S10, S1  : INTEGER RANGE 0 TO 9;
    SIGNAL CNTS     : INTEGER RANGE 0 TO 499999;
    SIGNAL S_CLK    : STD_LOGIC;
    SIGNAL M_CLK    : STD_LOGIC;
    SIGNAL H_CLK    : STD_LOGIC;

COMPONENT SEP
    PORT(
            A       :IN   INTEGER RANGE 0 TO 63;
            TEN, ONE:OUT  INTEGER RANGE 0 TO 15);
END COMPONENT;

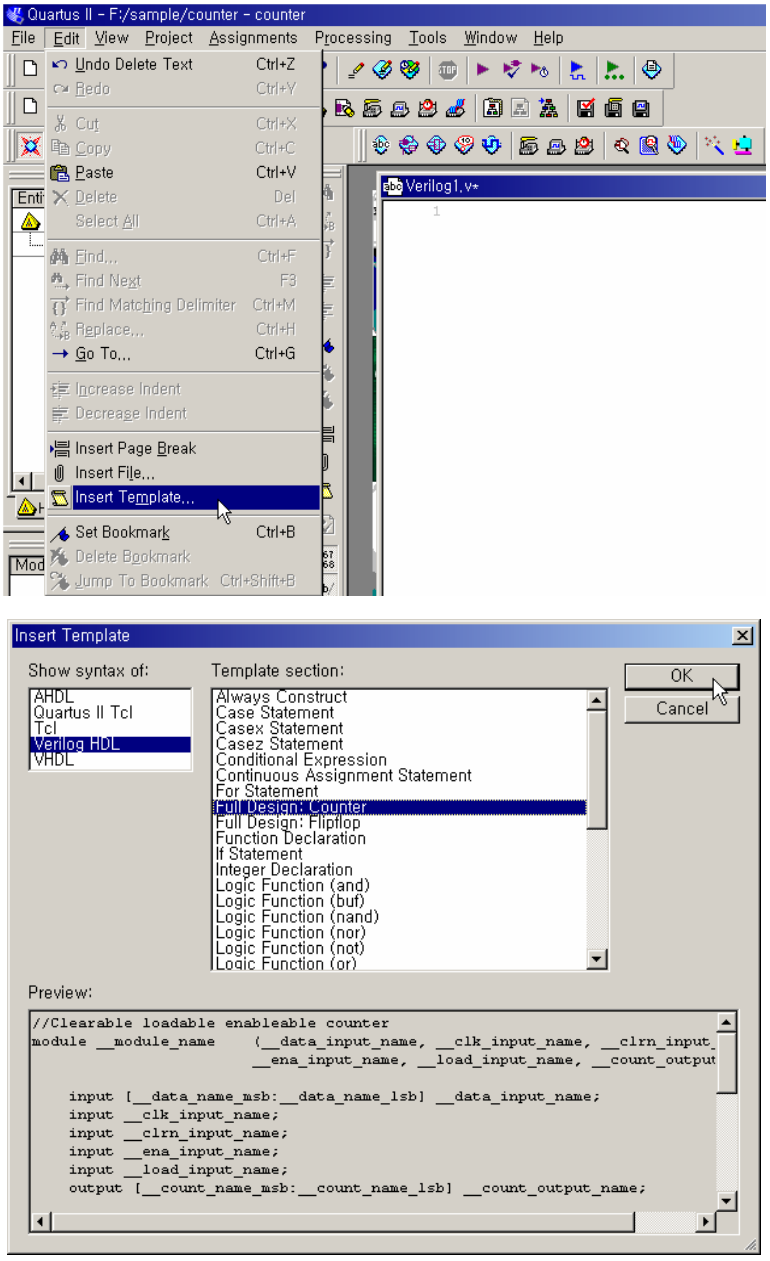
COMPONENT DEC7
    PORT(
            BCD       :IN   INTEGER RANGE 0 TO 15;
            D         :IN   STD_LOGIC;
            SEG       :OUT  STD_LOGIC_VECTOR(7 DOWNTO 0));
```

[그림 4-27] VHDL로 설계한 예



[그림 4-28] Verilog HDL로 설계한 예

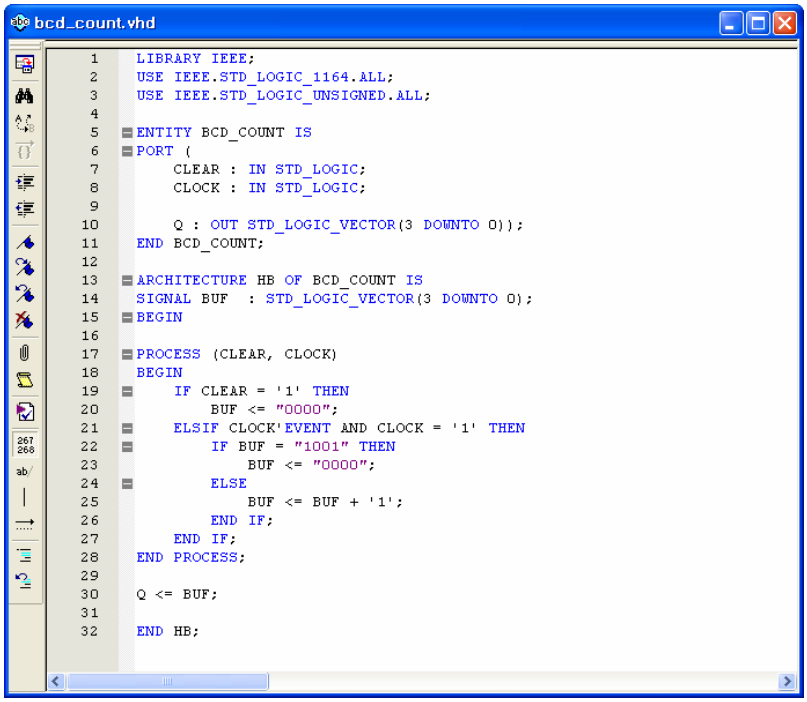
참고. 설계하는 문법은 Text Editor 창을 활성화 시킨 상태에서 [그림 4-29]과 같이 Template 메뉴의 각 언어 - VHDL, Verilog HDL등 - 의 Template 창을 선택하여 설계하는 구문의 문법을 알 수 있습니다.



[그림 4-29] Template 선택

④ **설계 완성**

이전에 설명한 것과 같이 다양한 종류의 Text 언어로 설계가 가능하고 Template를 통해 설계 형태에 대한 도움을 받아 설계가 가능합니다. 다음의 그림은 BCD COUNTER에 대한 예제를 보여주고 있습니다.



[그림 4-30] BCD COUNT(VHDL)

❑ **설계 시 Block Diagram/Schematic File로 설계하는 방법과 HDL로 설계하는 방법은 설계 방법에서 차이가 있습니다. 또한 설계가 완성된 결과도 차이를 보여주고 있습니다. 위에서 예로 보여준 두 가지 방식의 BCD_COUNT는 설계 방법에서만 다르고 이 후로 진행되는 Compile부터의 진행 과정은 동일하게 진행 됩니다.**

4) Compile

① **역할**

컴파일러는 프로젝트에 포함된 모든 디자인 파일을 통합하는 과정과 문법 오류와 일반적인 설계의 오류를 찾는 과정을 하게 됩니다. 또한 논리 회로 합성 및 설계한 논리 회로가 하드웨어에서 이루어지는 가장 기본 단위인 로직 셀로 나누고, 각 셀의 위치를 결정하는 과정을 하게 되고, 시뮬레이션과 시간 측정을 할 수 있도록 하는 파일을 생성하고, 마지막으로 타겟 디바이스에 프로그래밍 할 수 있는 파일을 만드는 과정을 진행합니다.

컴파일 과정은 보통 2번의 과정을 수행하게 되는데, 처음의 컴파일 과정은 현재 작성된 설계 파일에 대한 오류를 검증하는 단계입니다. 따라서 이 과정을 통해 컴파일 오류를 찾아서 수정이 가능합니다. 다음의 컴파일 과정은 설계 파일에 대한 핀 할당 정보 등의 프로젝트 선언 다음의 수정 내용을 현재 프로젝트에 적용해 주는 단계라 할 수 있습니다.

② Compiler

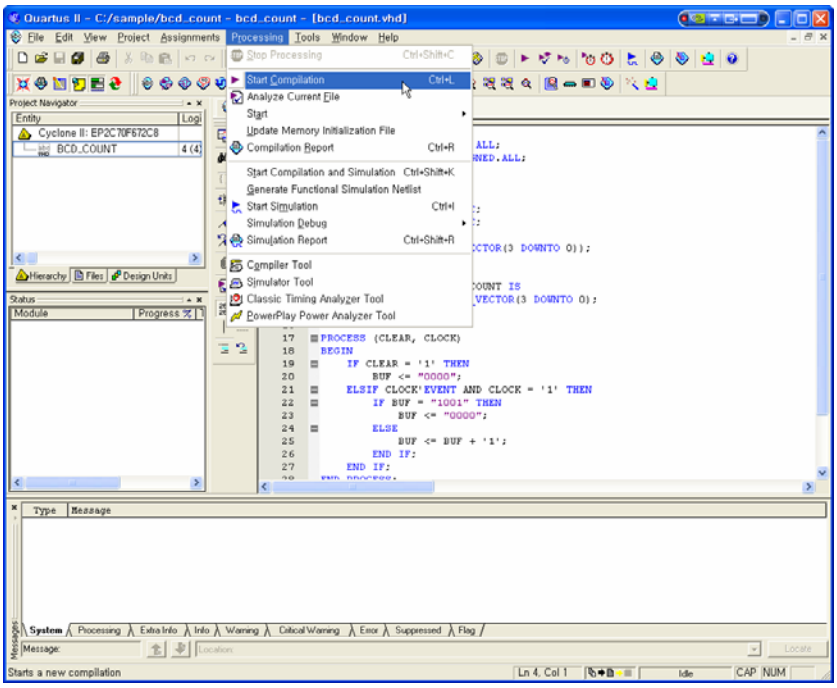
[그림 4-31]과 같이 메뉴의 Processing -〉 Start Compilation 항목을 선택하면 컴파일 과정을 수행하게 됩니다. 이 컴파일 과정은 4단계로 나뉘어 수행하게 되는데, 왼쪽의 상태 표시 창에서 진행 상황을 확인해 볼 수 있습니다.

Analysis & Synthesis 에서는 프로젝트 내에 있는 파일들을 분석하고 통합하는 과정에서 발생하는 기본적인 오류를 체크하는 과정입니다. 따라서 회로를 분석하는 과정에서 오류가 발생하면 컴파일을 중단하고 메시지 창에 에러를 찾아 표시해 줌으로써 회로를 쉽게 수정할 수 있게 해줍니다.그리고 회로를 분석하는 과정에서 디바이스의 내부구조를 효율적으로 사용하기 위해 불필요한 논리를 제거합니다.

Fitter 에서는 사용자가 디바이스를 정해주지 않았을 때는 Quartus II에서 회로에 최적인 디바이스를 자동적으로 찾아서 디바이스의 로직 셀에 배치해 줍니다. 디바이스를 정해 주었을 때는 그 디바이스에 맞게 회로를 적당한 로직 셀에 배치합니다.먼저 각각의 논리 함수를 가장 적합한 위치의 로직 셀에 할당하고 상호 연결이나 핀 등을 적합하게 할당시킵니다.

Assembler에서는 이전의 작업을 디바이스에 프로그램 할 수 있는 파일을 생성시킵니다.

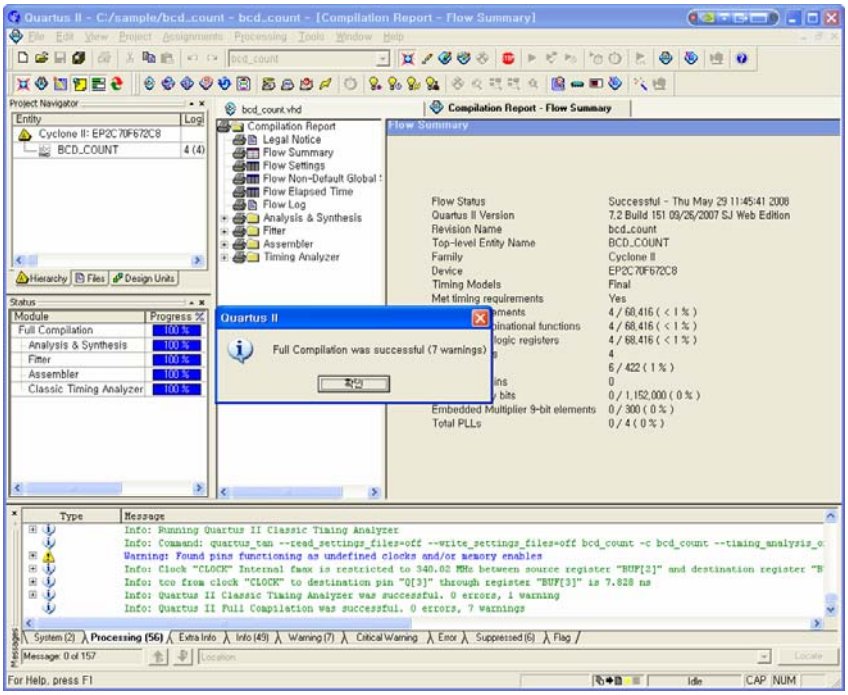
마지막으로 Classic Timing Analyzer에서는 최종적으로 디바이스에 할당된 정보를 가지고 핀과 핀 또는 로직 셀 사이의 데이터 전달 시간을 검사하는 과정입니다.



[그림 4-31] Processing -〉 Start Compilation 항목 선택

[그림 4-32]에서는 컴파일 과정이 끝난 화면을 보여주고 있습니다. 따라서 왼쪽의 상태 표시 창에서 모든 과정이 100%로 완료된 것을 볼 수 있고, 이전에서 말한 4가지의 컴파일 과정을 오른쪽 메인 화면 창에서 리포트 형태로 볼 수 있습니다. 따라서 여기에서 어떠한 형태로 작업이 수행되었는지 나중에 확인해 볼 수 있습니다.

또한 밑에 메시지 창에서 컴파일 과정에 생긴 간단한 정보나, 에러, 또는 경고 등을 표시해 줍니다. 에러 메시지의 발생 시 컴파일이 중단되고 어떠한 내용의 에러 메시지인지 표시해 주게 됩니다. 따라서 에러 메시지를 클릭하면 에러가 난 지점으로 커서가 이동하게 되고 사용자는 에러 메시지와 더불어 에러에 대한 수정을 쉽게 할 수 있습니다.



[그림 4-32] Compiler 과정 완료

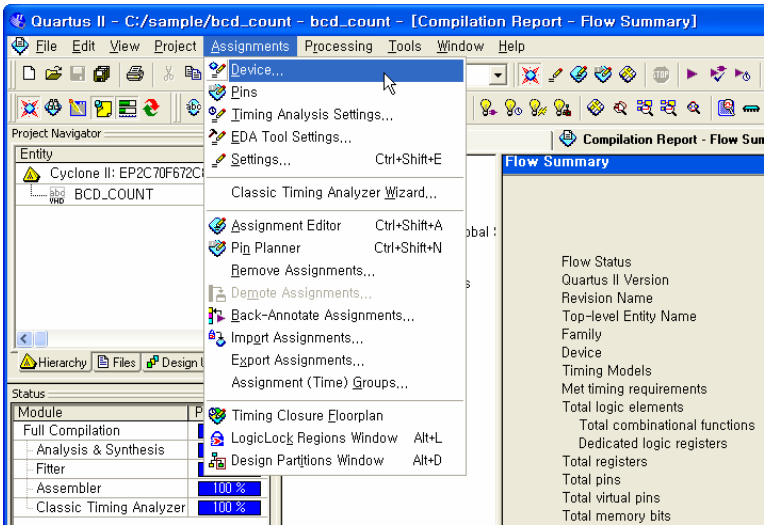
5) Assignments

① 역할

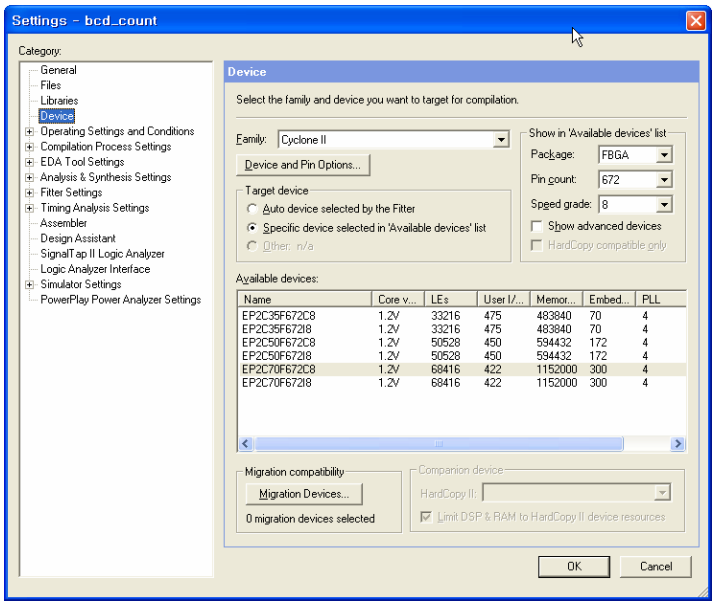
이 작업은 현재 작업하는 프로젝트의 디바이스 설정과 선택한 디바이스의 I/O 핀 할당에 대한 작업을 수행하게 됩니다. 디바이스 선택은 사전에 프로젝트 선언 단계에서 현재 사용하는 디바이스를 선택했습니다. 하지만 프로젝트 선언 후에 이 메뉴를 통한 디바이스 변경 작업을 해 줄 수 있습니다. 지금은 Assignments라는 작업에서의 크게 할 작업은 설계 파일과 디바이스에 있는 I/O핀과의 연결을 해 주는 작업입니다. 따라서 사용자가 어떠한 핀을 이용하여 사용할지를 프로젝트에 선언해 주는 작업이 되겠습니다.

②Device

디바이스는 [그림 4-33]과 같이 Assignments -〉 Device 메뉴를 선택하여 설정합니다. 이 메뉴를 선택하면 [그림 4-34]와 같은 Settings 창이 활성화 됩니다. 이 부분에서 사용하려는 디바이스의 선택을 하거나 다른 디바이스를 찾아 재 설정을 해 줄 수 있습니다.



[그림 4-33] Assignments -> Device 항목 선택



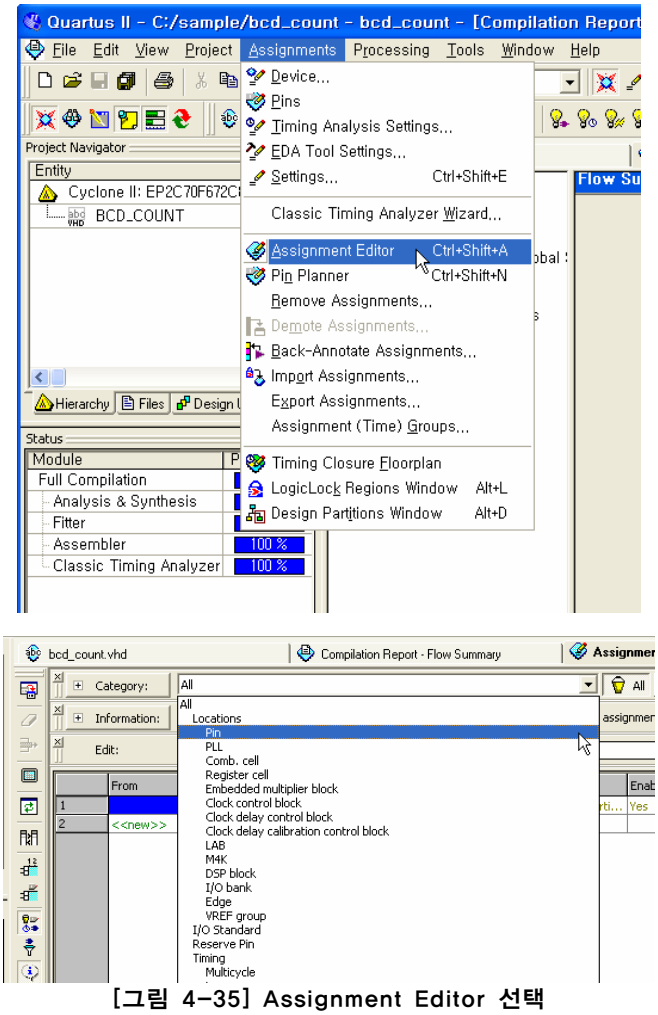
[그림 4-34] Device Settings

작업 창은 프로젝트 설정에서의 디바이스 설정 과정과 비슷한 구조로 되어 있으며, 동일한 방법을 통해 디바이스를 선택해 줍니다. 여기에 있는 Device & Pin Options은 디바이스 내부에 있는 특수 핀, 사용하지 않는 핀, 전원 관리 등의 설정을 해 주는 작업을 하게 됩니다. 따라서 디바이스 관리에 대한 전반적인 작업을 설정해 줄 수 있는 메뉴가 되겠습니다.

활성화 된 창에서는 디바이스 선택 이외의 현재 프로젝트에서 사용자가 설정하려는 모든 작업을 할 수 있습니다. 따라서 프로젝트에서 관리하는 파일, 연동해서 사용하는 툴 설정, 컴파일 및 시뮬레이션 과정의 설정 등의 작업을 Settings 창에서 할 수 있습니다.

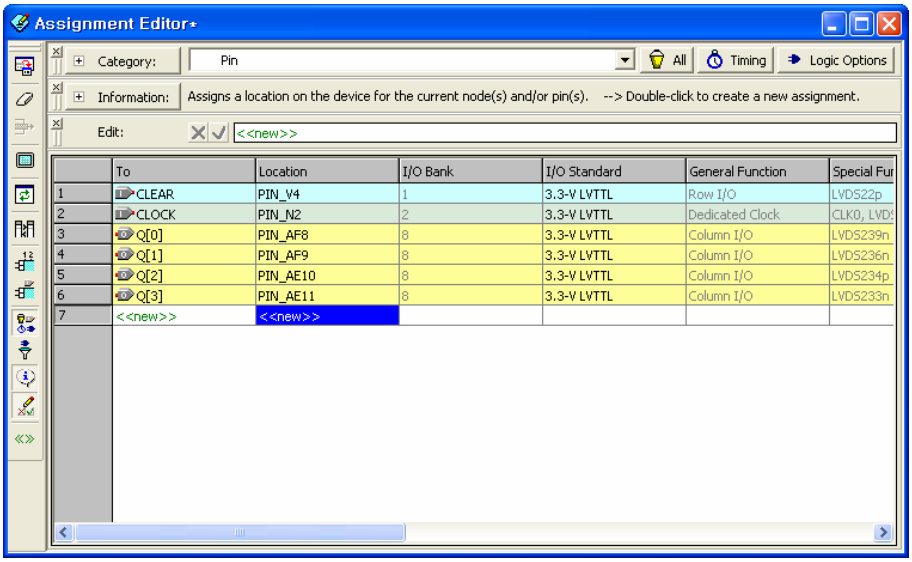
③ Pin Assignments

핀 설정은 [그림 4-35]에서와 같이 메뉴에서 Assignments -> Assignment Editor에서 핀 설정을 해 줄 수 있습니다.



[그림 4-35] Assignment Editor 선택

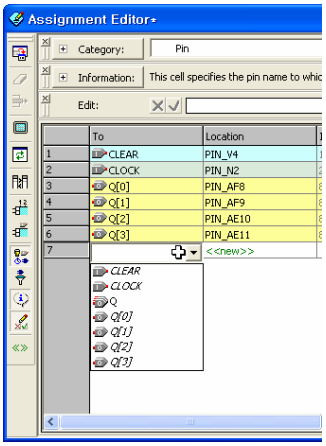
위의 그림에서는 위쪽 그림은 Assignment Editor에 들어가는 메뉴를 보여주고 있습니다. 따라서 이렇게 메뉴에 들어가서 핀 할당을 위한 작업으로 오른 쪽 그림과 같이 Pin 이라는 버튼을 눌러 핀 할당을 위한 환경을 만들어 주어야 합니다.

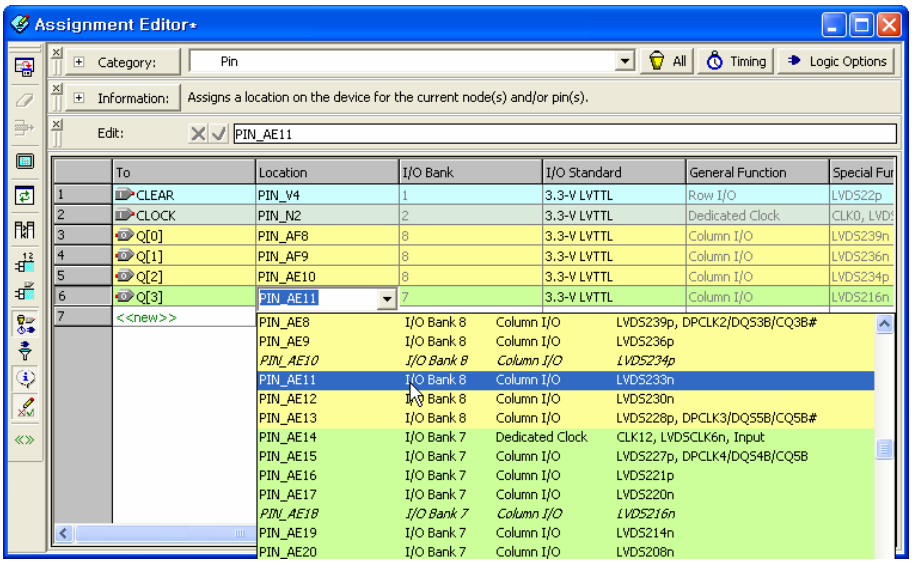


[그림 4-36] Assignment Editor

위의 [그림 4-36]에서는 Assignment Editor창을 보여주고 있습니다. 여기에서는 할당하려는 핀 이름은 설계 파일 작성 시 미리 지정을 해 놓았습니다. 따라서 핀 이름에 대해서는 따로 타이핑 할 필요 없이 찾아서 Assign해 주면 됩니다.

핀 할당 방법은 [그림 4-37]과 같이 To에서는 핀 이름이 들어가는 부분이고, Location 부분에서는 디바이스의 핀 번호가 들어가게 됩니다. 따라서 To와 Location 부분 하위에 〈〈new〉〉 라는 녹색의 부분에 마우스로 클릭하면, To에서는 설계 파일의 포트 이름이 보이게 되고 Location에서는 선택된 디바이스의 핀 번호 모두가 보이게 됩니다. 따라서 이러한 방식으로 포트 이름과 디바이스 핀 번호를 찾아서 할당해 주면 됩니다. [그림 4-35]에서는 핀 할당에 대한 작업을 자세히 보여 주고 있습니다.





[그림 4-37] 핀 할당 작업

위와 같은 방법으로 6개의 입 출력 핀 번호를 할당하면 됩니다. 현재 핀이 연결되어 있는 것은 CLEAR은 버튼 스위치, CLOCK는 메인 클럭, Q0 ~ Q3은 LED를 통해 할당되어 있습니다. 위의 그림 및 매뉴얼에 있는 핀 정보를 통해 핀을 할당해 주시기 바랍니다.

6) Simulation

① 역할

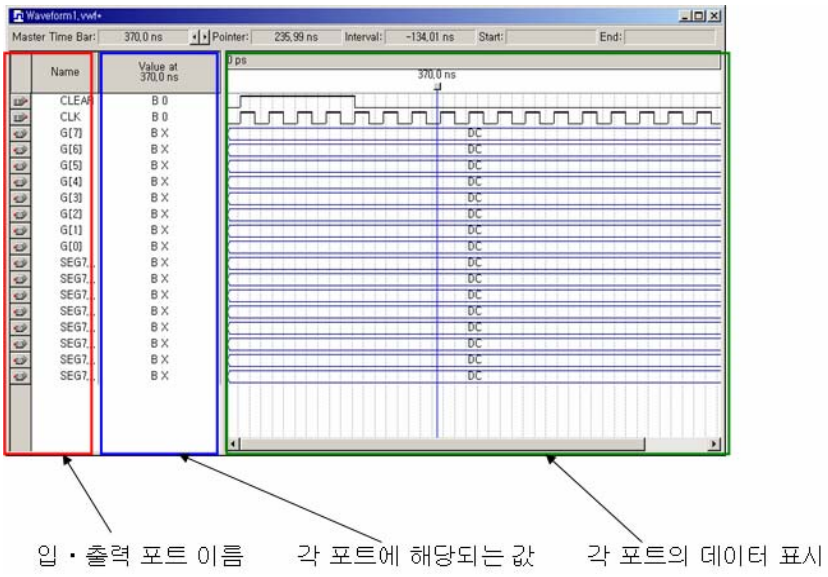
설계한 논리 회로를 하드웨어를 이용해서 검증하는 경우에, 처리하는 데이터량이 많거나 속도가 빠를 경우에는 검증하는데 시간이 많이 걸리고 그 원인을 파악하지 못하는 경우가 있기 때문에 소프트웨어 적으로 먼저 검증하는 것이 필요합니다. Quartus II에서는 이를 위해 파형으로 검증하는 시뮬레이션을 제공하는데, 이 시뮬레이션에서 동작하는데 필요한 입력 파형의 조건을 설정해 주면 그에 따른 출력 결과의 파형이 출력되어 쉽게 확인해 볼 수 있습니다. 따라서 설계 로직에 대한 시뮬레이션 결과가 사용자가 원하는 동작이 되는지 검증 해 볼 수 있습니다.

② 구성

시뮬레이션을 하기 위해서는 입력 파형의 조건을 설정해 주어야 한다고 했습니다. [그림 4-38] Segment에 대한 예제를 시뮬레이션으로 구동한 모습을 보여주고 있는데, 구성으로 왼쪽에 보이는 창에는 입,출력 포트의 이름을 보여줍니다. 이곳에서는 현재 프로젝트의 설계 파일에서 선언한 포트들이 여기에 나타나는 것입니다. 다음으로 가운데 창에서 각 포트에서 사용자가 선택한 시뮬레이션 시간에 나타나는 출력 값을 표시

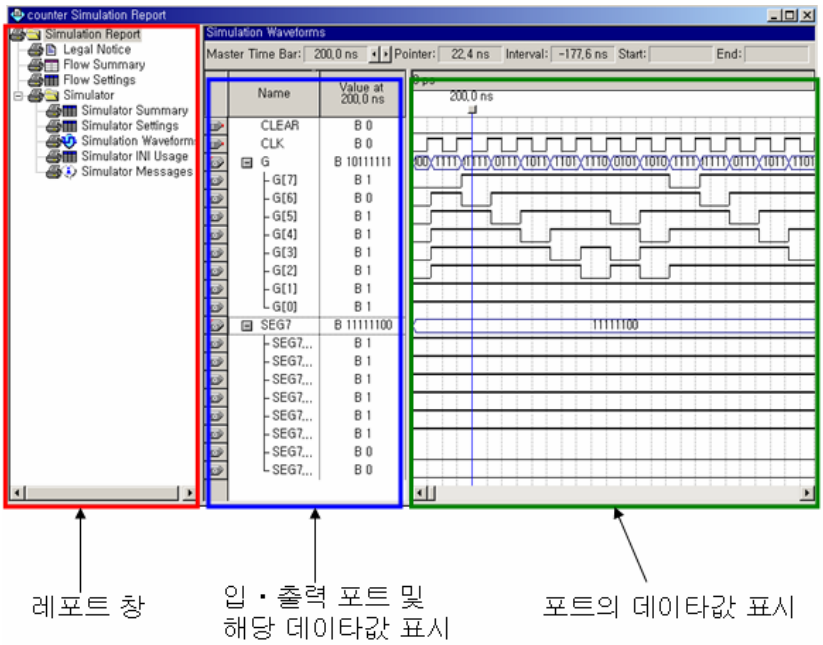
해줍니다.

따라서 파형을 일일이 보면서 값을 알아내는 것과는 달리, 상태 표시 바를 이동하여 사용자가 측정 하고 싶은 시뮬레이션 시간에 출력되는 포트 값을 이곳에서 표시해줍니다. 마지막으로 오른쪽 창에서 각 포트에 대한 실제 데이터 값을 시간의 따라 파형으로 표시해줍니다. 따라서 이 시뮬레이션 창을 통해 포트들이 어떻게 동작했는지를 여기에서 파형으로 한 눈에 파악할 수 있습니다.



[그림 4-38] Simulation 입력 창

다음으로 [그림 4-39]에서는 시뮬레이션 결과값이 나타나는 창을 보여주고 있습니다. Quartus II 에서는 입력 포트의 입력 값을 설정해 주는 창에서 바로 그 결과값이 나타나지 않고, 새로운 창에서 시뮬레이션 결과 값이 어떻게 나타나는지를 확인할 수 있습니다. 그림과 같이 시뮬레이션 결과 파형이 나오는 창의 구성도 리포트 구간이 더 추가된 것을 볼 수 있습니다.



[그림 4-39] Simulation 결과 창

리포트 창에서는 시뮬레이션 결과에 대한 내용을 Text 형식으로 보여주고 있습니다. 여기에서는 시뮬레이션 과정과 설정 정보 등을 확인해 볼 수 있습니다. 그리고 나머지 두 개의 영역에서 시뮬레이션 결과에 대한 것을 파형으로 검증해 볼 수 있습니다.

③ Simulator

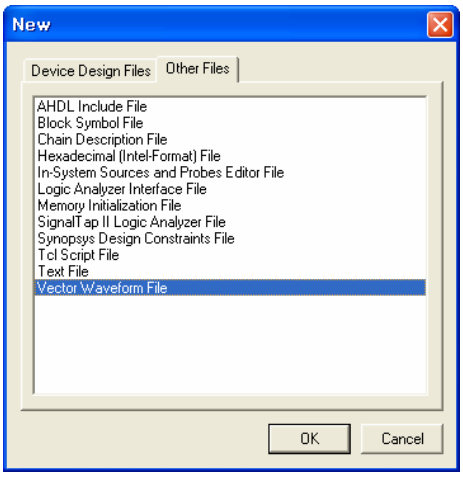
시뮬레이션은 결과 값의 형태에 따라 크게 두 가지 방법으로 나뉘어 집니다. 기능적인 시뮬레이션(Functional Simulation)과 시간 특성을 지닌 시뮬레이션(Timing Simulation)입니다. 기능적인 시뮬레이션은 설계한 회로에 대한 논리적으로 분석하여 이 사항을 시뮬레이션 결과로 보여주게 됩니다. 위의 기능적인 시뮬레이션이 단지 설계한 회로를 논리적으로 검증한다면, 타이밍 시뮬레이션 (시간 특성을 지닌 시뮬레이션 : Timing Simulation)은 설계자가 결정한 하드웨어의 상태 즉, 사용하려고 하는 디바이스와 핀 번호 등의 지연 시간을 고려한 파형으로 검증하는 것입니다.

시뮬레이션은 기본 설정이 타이밍 시뮬레이션으로 동작하게끔 되어 있습니다. 따라서 아무 설정 없이 시뮬레이션을 시키면 타이밍 시뮬레이션으로 동작을 하게 됩니다.

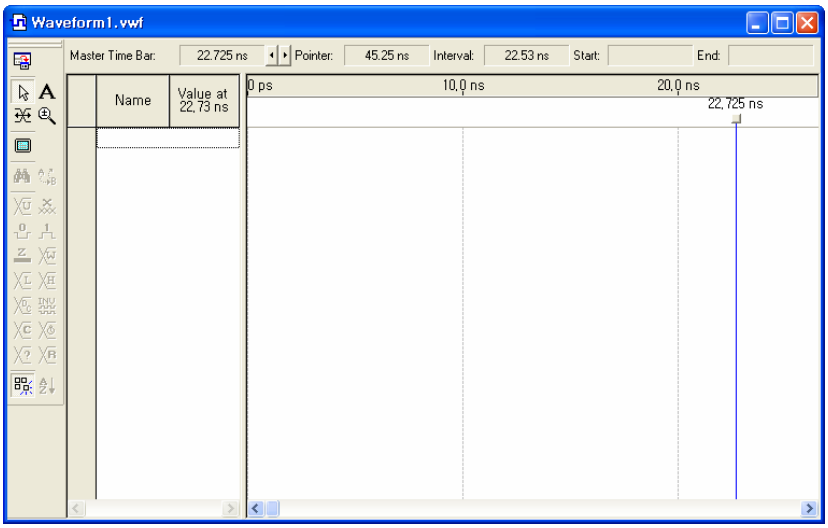
초반에 타이밍 시뮬레이션을 통한 검증에서 타이밍 정보와 논리적인 결과값이 동시에 나옵니다. 또한 포트의 수가 늘어날 때 시뮬레이션에 대한 검증 작업에 어려움이 있을 수 있습니다. 시뮬레이션 분석 방법으로 처음 설계 파일의 검증을 위한 것이므로 기능적인 시뮬레이션을 통해 먼저 설계된 파일의 문제가 없는지 먼저 분석하고 이러한 분

석이 완료가 되었을 때, 타이밍 시뮬레이션을 통해 타겟 디바이스를 적용한 시뮬레이션 검증을 하는 방법으로 이루어 져야 합니다.

다음으로 시뮬레이션 과정을 하나씩 해 보도록 하겠습니다. 먼저 메뉴 창에서 File -> New 항목을 선택하면 [그림 4-40]과 같이 New 창이 활성화 되는데 여기서 Other Files 에서 Vector Waveform File을 선택합니다. 그러면 [그림 4-41]과 같이 시뮬레이션을 할 수 있는 Waveform 창이 활성화 됩니다. 여기에서 입력 핀에 파형을 인가해, 출력되는 파형을 검증해 볼 수 있습니다.



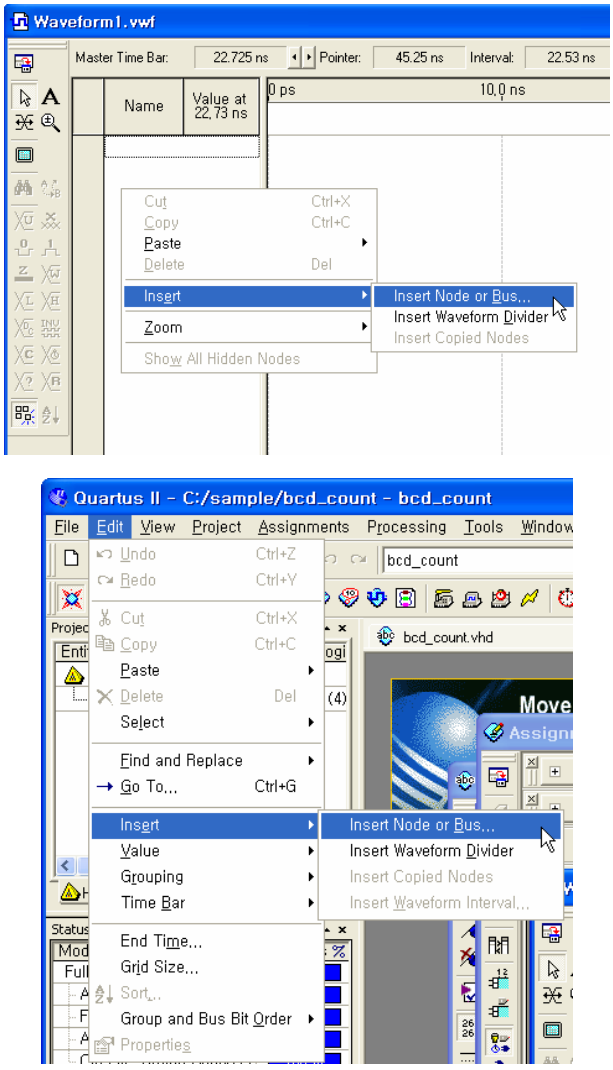
[그림 4-40] Simulation 메뉴 선택



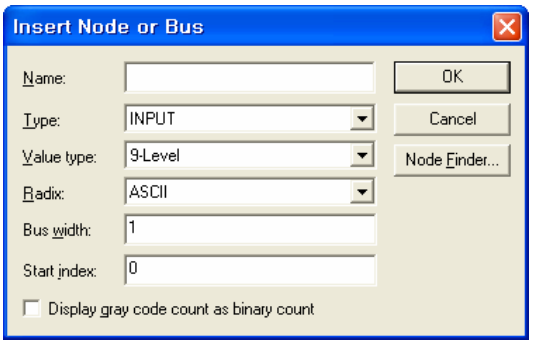
[그림 4-41] Simulation 창 활성화

위의 그림과 같이 시뮬레이션 창을 활성화 시켰으면, 여기에 입,출력 핀을 불러와야

합니다. 불러오는 방법은 [그림 4-42]의 위쪽 그림과 같이 시뮬레이션 창의 왼쪽 편에 있는 Name 창에서 마우스 오른쪽 키에 의한 pop-up메뉴에서 Insert -> Insert Node or Bus를 선택하거나, [그림 4-42]의 아래 쪽 그림과 같이 Quartus II 메뉴의 Edit -> Insert -> Insert Node or Bus 를 선택합니다. 그러면 [그림 4-43]과 같이 입, 출력 포트를 넣는 새로운 창이 생깁니다. 여기서 포트를 검색하기 위해 오른쪽에 있는 Node Finder 버튼을 클릭하여 입,출력 포트를 찾을 수가 있습니다.

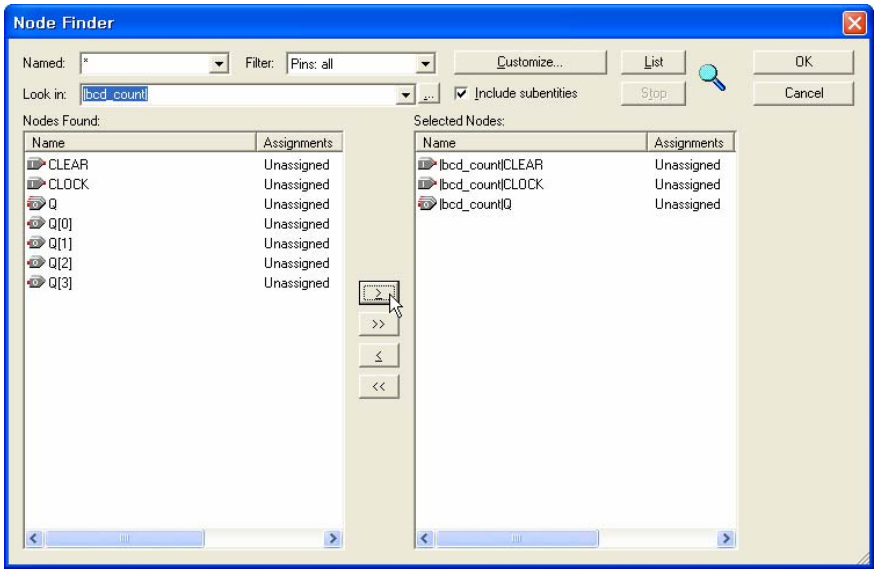


[그림 4-42] Simulation 입 · 출력 포트 불러오기



[그림 4-43] Insert Node or Bus

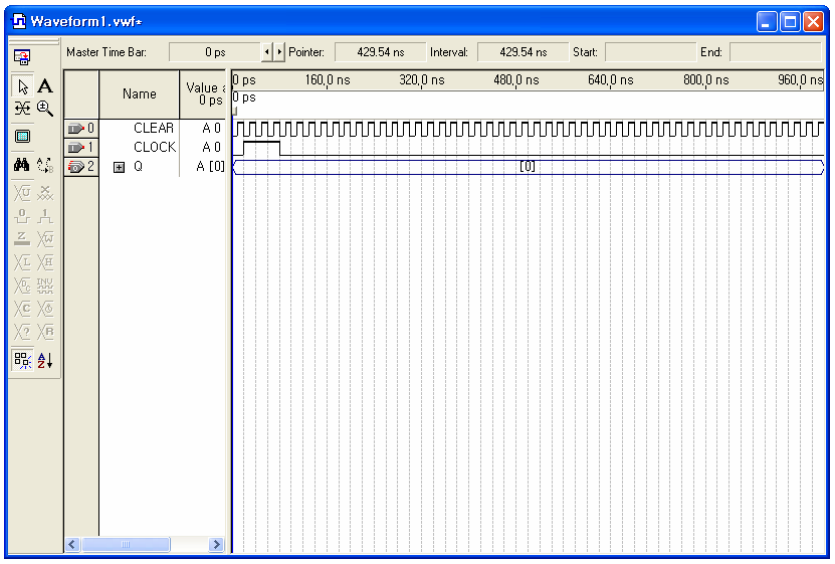
[그림 4-43]의 Node Finder 버튼을 사용하여 [그림 4-44]의 Node Finder 창이 활성화 됩니다. 이 창에서는 입,출력 포트에서 보고자 하는 포트를 선택하는 곳으로 Filter을 all로 설정해 준 다음 검색을 하여야 합니다. 검색을 위해서는 List 버튼을 통해 그림과 같이 설계한 프로젝트의 입,출력 포트가 나타나게 됩니다. 이 포트들 중에 확인하려는 포트를 찾아서 선택을 하면 됩니다.



[그림 4-44] Node Finder

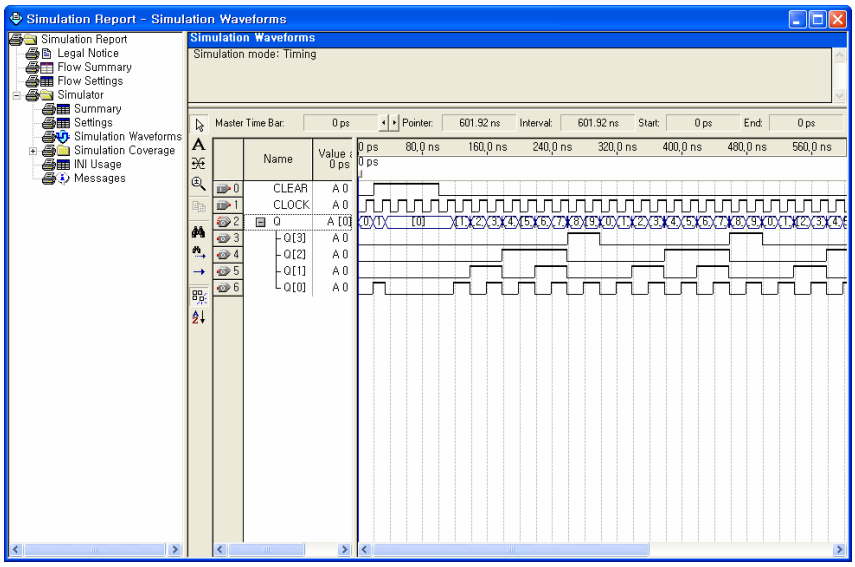
이상으로 포트 선택을 완료하면 OK 버튼을 눌러 마치면, 시뮬레이션 창에 포트 이름이 보입니다. Waveform 창에서 불러들인 포트들을 마우스로 블록화해서 파형을 그려주면 됩니다. 방법으로는 마우스로 파형을 입력하려는 포트의 영역을 블록을 지정하고, File -> Value 에 있는 메뉴를 통해 블록으로 지정된 곳에 입력 파형을 줄 수 있습니다. 여기에서는 0, 1값부터, High Impedance, Unknown, Clock등의 값을 줄 수 있습니다.

그림 [그림 4-45]은 입력 파형에 대한 정의를 마친 모습을 보여주고 있습니다. CLOCK은 주기적인 클럭을 CLEAR에 대해서는 일정 구간을 1 로 할당한 모습입니다.



[그림 4-45] Waveform Editor 파형 정의

이렇게 파형 정의를 마치면 현재 작업한 것을 프로젝트에 저장하고, Processing -〉 Start Simulation을 통해 시뮬레이션 결과 파형을 볼 수 있습니다. [그림 4-46]는 이러한 결과 파형을 보여 주고 있습니다. 여기에서는 초기의 파형 입력 창과 달리 별도의 창이 활성화 되어 결과 파형을 분석해 주고 있습니다. 왼쪽의 창의 리포트 창을 통해 시뮬레이션에서 발생하는 결과값을 문서로 표시해 주고 있습니다.



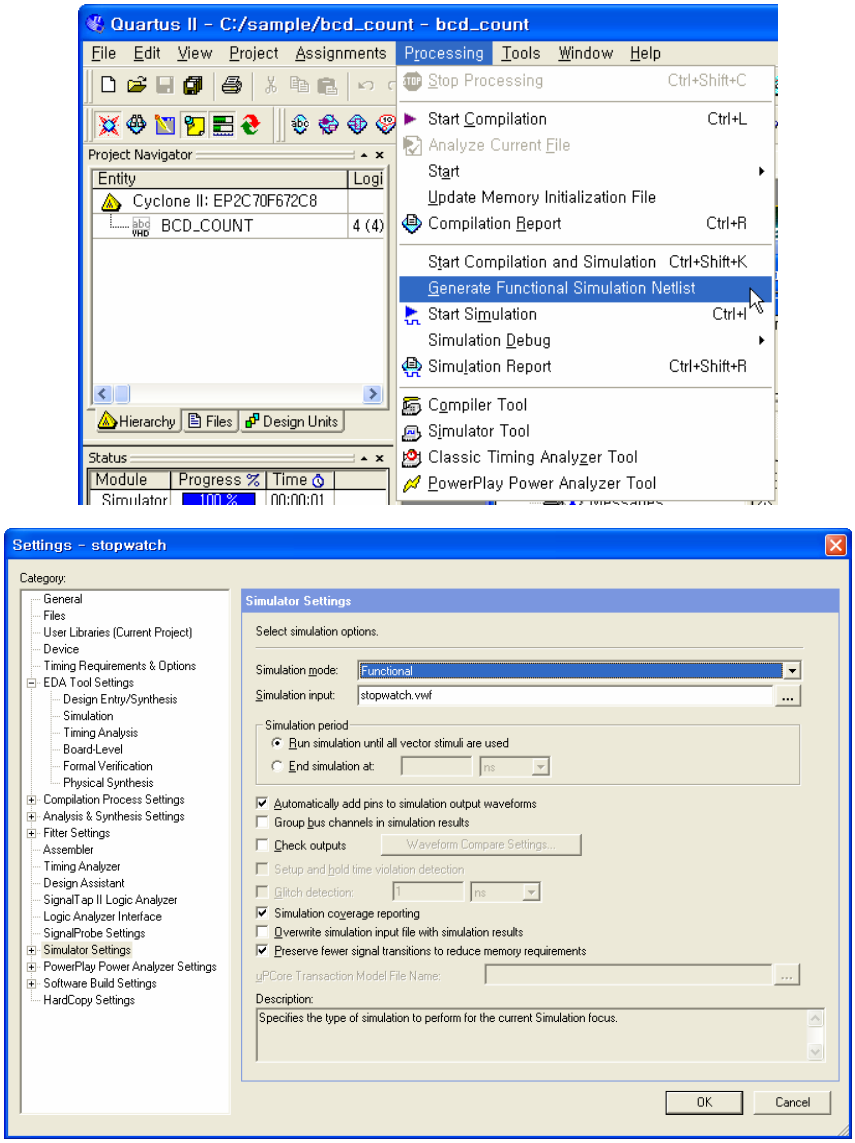
[그림 4-46] Simulation Waveforms

시뮬레이션에서는 이러한 입력파형에 대한 값과 설계 파일의 정보를 보고 결과 값을 보여주게 됩니다. [그림 4-46]에서는 파형의 입력 창과는 달리 Report 구간에 대한 영역이 나타나 있습니다.

따라서 파형적인 분석과 달리 문서적으로 결과에 대한 리포트를 해 주고 있습니다. 또한 이러한 시뮬레이션은 앞서 설명한 것과 같이 타이밍 시뮬레이션을 기본으로 하고 있습니다. 그러므로 사용자가 아무런 설정이 없으면, 디바이스에 대한 delay 시간을 고려한 시뮬레이션 과정을 수행하게 됩니다.

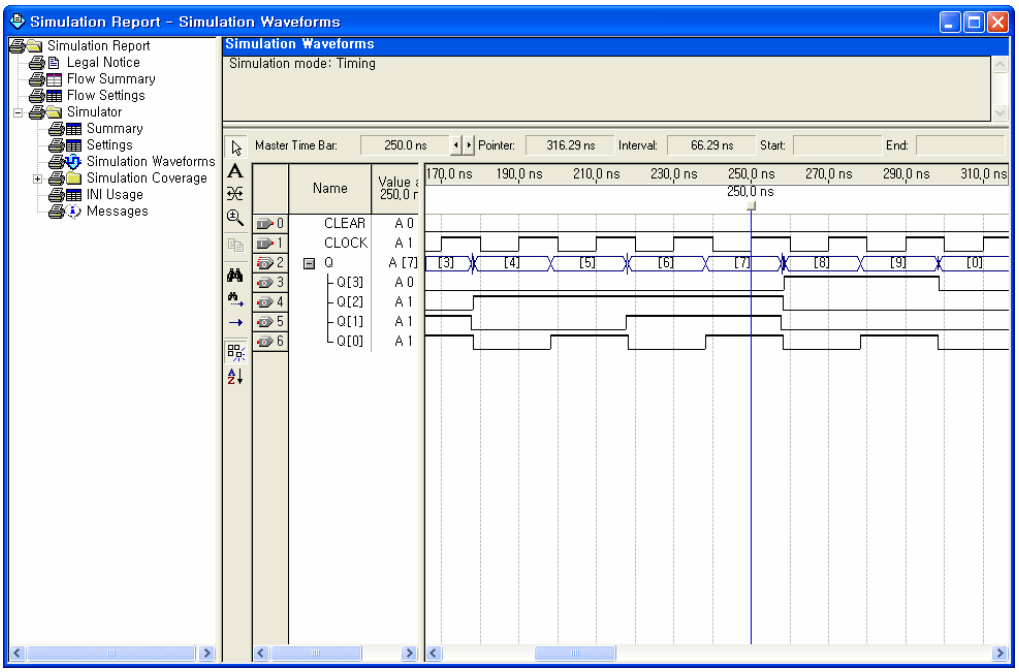
Function 시뮬레이션 작업을 위해서는 [그림 4-47]과 같이 메뉴의 Processing -〉 Generate Functional Simulation Netlist 작업을 먼저 해 주어야 합니다. 이 작업을 통해 만들어진 데이터 값을 보고 기능적인 시뮬레이션 작업을 수행하게 되는 겁니다. 그리고 Assignments -〉 Settings에서 왼쪽 Category를 Simulator Setting로 선택해 주고, 나타나는 시뮬레이션 창의 Simulation mode를 Functional로 선택해 서 시뮬레이션을 실행 시키면 됩니다.

만약 Generate Functional Simulation Netlist 작업이 수행되지 않고 Setting 창에서 Functional만 바꾸어서 실행을 시키면, 시뮬레이션 과정에서 Netlist 파일에 대한 정보가 없어서 에러가 발생하고 시뮬레이션 과정이 중단되게 됩니다. 따라서 위의 두 과정을 꼭 진행해 주어야 합니다.

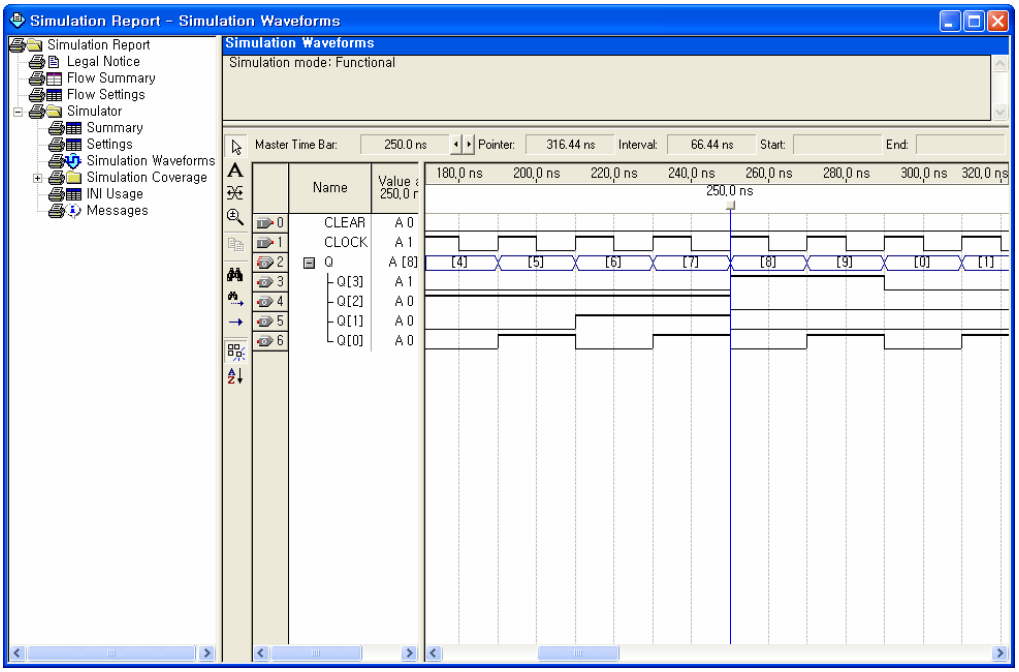


[그림 4-47] Functional Simulation Settings

다음의 [그림 4-48]과 [그림 4-49]는 두 가지의 시뮬레이션 결과를 비교해서 보여주고 있습니다. 타이밍 시뮬레이션에서는 디바이스에서 발생하는 delay 시간을 고려한 시뮬레이션 결과를 보여주고 있어 클럭의 상승 구간에 바로 동작하지 않고 디바이스 지연 시간을 고려한 시뮬레이션을 볼 수 있습니다.



[그림 4-48] Timing Simulation



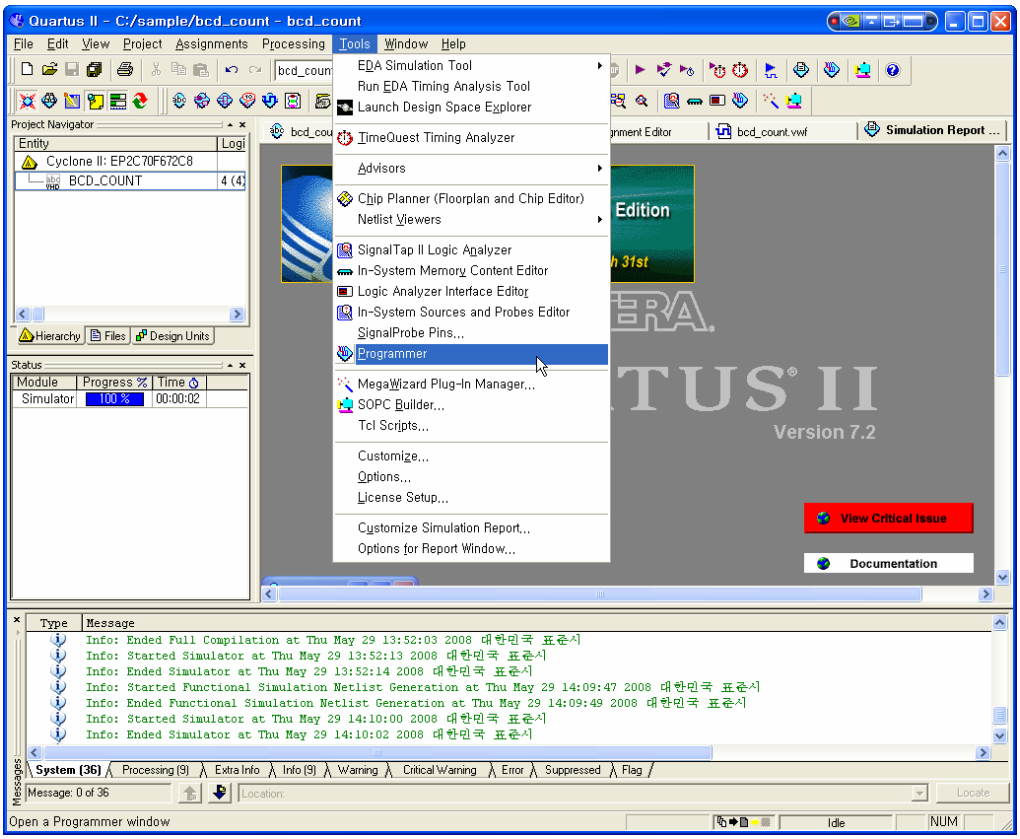
[그림 4-49] Function Simulation

7) Programming

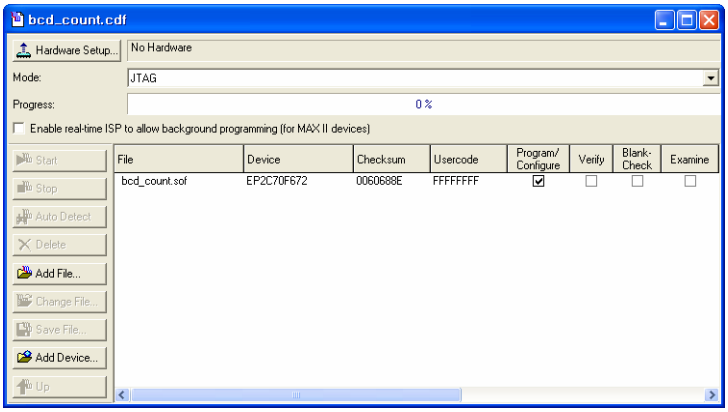
위에서 설계 및 검증이 끝난 논리 회로를 디바이스에 프로그래밍 부분입니다.

① Programmer 활성화

[그림 4-50]과 같이 Quartus II 메뉴에서 Tool 〉 Programmer에서 Programmer 창을 활성화해서 사용할 수 있습니다. 이 메뉴를 선택하면 [그림 4-51]과 같이 Programmer창이 활성화 되고 여기에서 장비에 직접 다운할 수 있습니다.



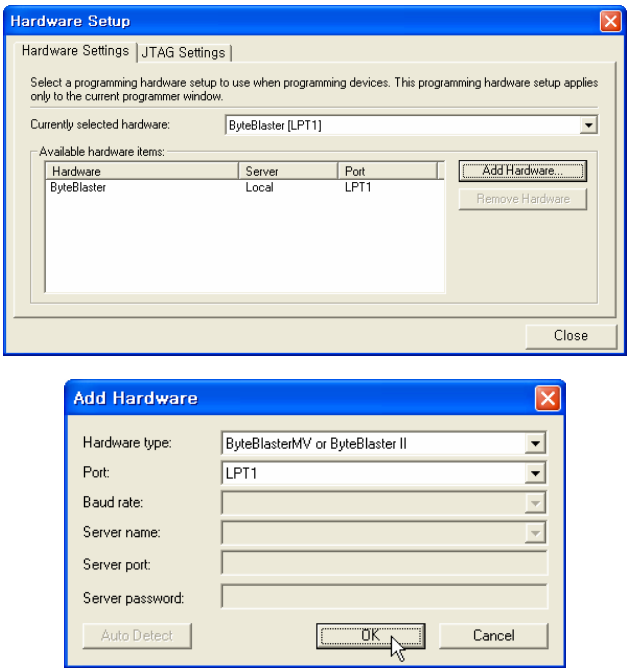
[그림 4-50]Quartus II 메뉴에서 Programmer 항목 선택



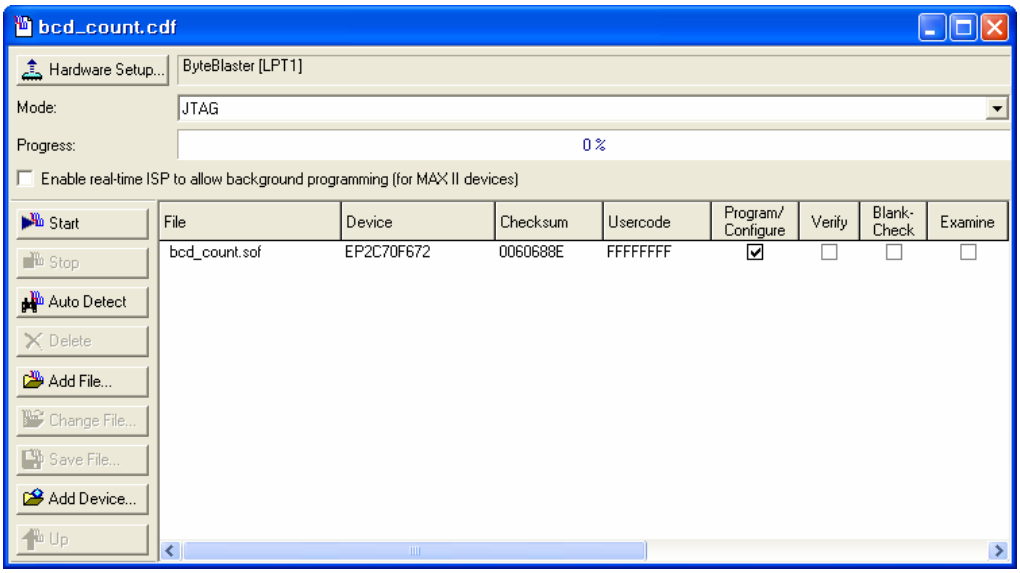
[그림 4-51] Programmer 창 활성화

② 하드웨어 설치

처음에 Programmer 창을 활성화 하면 Hardware Setup 창에 No Hardware라는 것이 표시가 됩니다. 여기에서는 ByteBlaster라는 하드웨어를 가지고 Altera device를 프로그램 하기 때문에 이러한 하드웨어를 등록을 해 주어야 합니다. Programmer 창 위에 보이는 Hardware Setup를 클릭하면 하드웨어를 등록해 주는 창이 활성화 됩니다. [그림 4-52]와 같이 Hardware Setup 창에서 ByteBlaster를 등록해 주면, [그림 4-53]과 같이 ByteBlaster의 등록된 모습을 볼 수 있습니다.



[그림 4-52] Hardware Setup

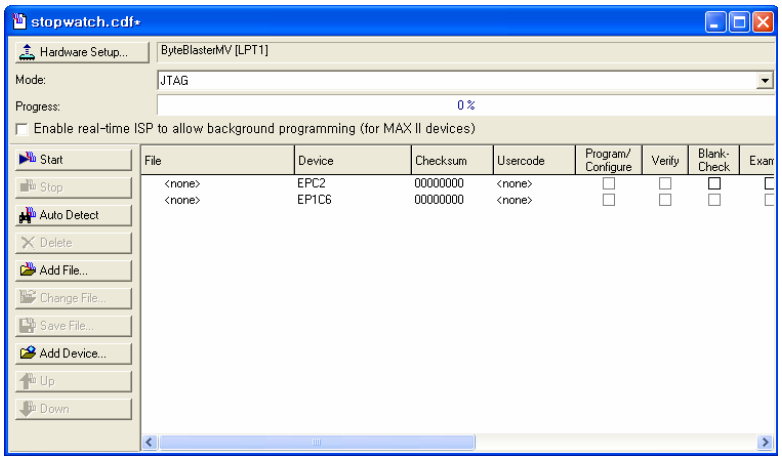


[그림 4-53] Hardware Setup 완료

③ 프로그래밍

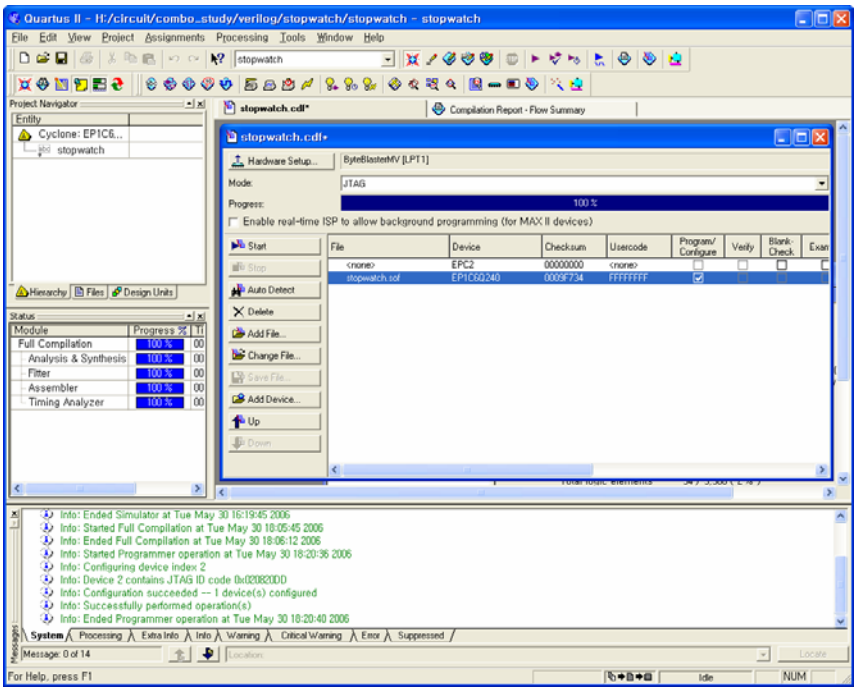
하나의 프로젝트가 선언되고 컴파일을 통해 프로그래밍 파일이 만들어 졌을 때, Programmer 창을 활성을 하면, File에 현재 프로젝트에서 프로그램 할 수 있는 .sof 파일과 device정보 등이 보이게 됩니다. 여기에서 Program/Configure에 대한 부분을 체크하고 Start 버튼을 클릭하여 다운 하시면 됩니다.

만약 동작이 되지 않을 때, Programmer 창에서 활성화 되는 부분의 버튼인 Auto Detect 버튼을 눌러 디바이스와 PC간의 연결이 제대로 되었는지 확인해 보기 바랍니다. 따라서 [그림 4-54]와 같이 화면상에 두 개의 디바이스가 나타나는지 확인해 보기 바랍니다. 만약 이 화면과 다르게 표시되거나 Unable to scan device chain. Can t scan JTAG chain. 이라는 메시지가 표시되면 연결 케이블에 대한 확인과 디바이스 전원에 대한 확인을 해 주고, 이 사항에 대한 문제가 없을 시 회사로 문의 해 주기 바랍니다.



[그림 4-54] Auto Detect

위 사항에서 아무런 이상이 없이 진행이 되면, 다운로드 케이블을 PC의 Parallel port와 보드의 다운로드 port에 연결한 후 Start 버튼을 누르면 디바이스에 다운이 됩니다. [그림 4-55]에는 다운이 완료된 모습을 보여주고 있습니다. 이렇게 다운이 완료되었을 때, 메시지 창에 Configuration succeeded 메시지가 표시되며 보드에서 설계한 회로가 바로 동작이 진행되게 됩니다.



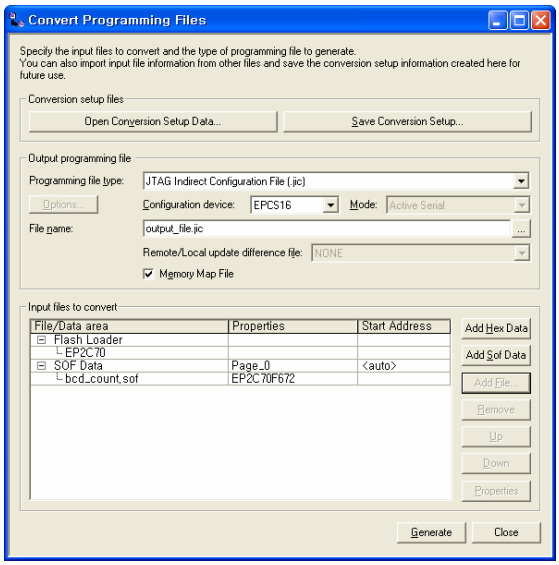
[그림 4-55] Configuration Ended

④ Configuration ROM Programming

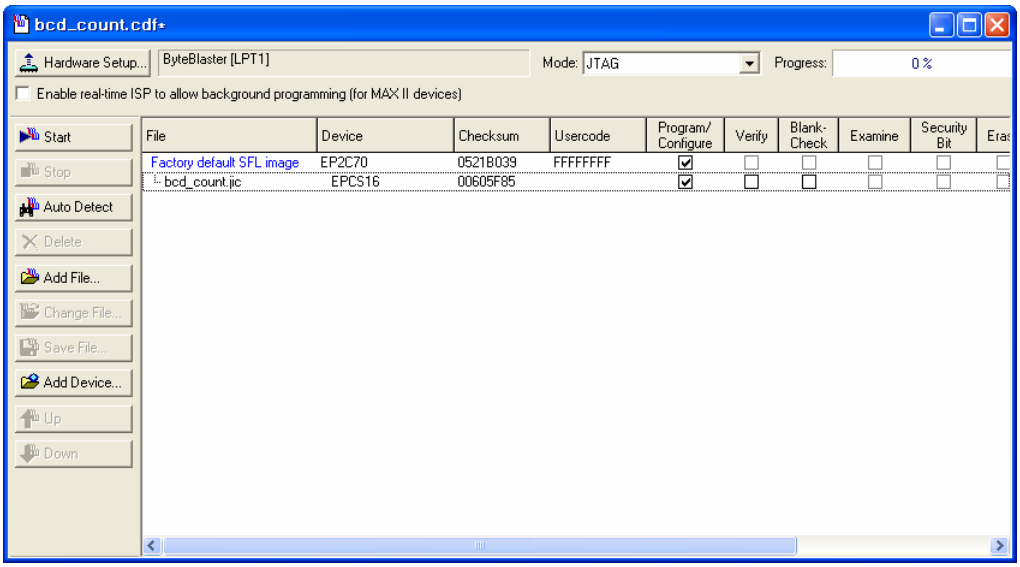
HBE-Combo III 보드에는 Cyclone II device가 RAM 타입의 디바이스이기 때문에 별도의 ROM 타입의 디바이스인 EPCS16 사용하고 있습니다. 따라서 전원의 ON/OFF시마다 FPGA 디바이스의 데이터가 사라져 다운을 다시 해야 되는 작업을 막기 위해 별도의 ROM을 두어서 전원의 ON/OFF시 마다 데이터를 ROM에서 FPGA디바이스로 전달하여 보드가 작동이 되도록 구성하고 있습니다.

Configuration ROM에 writing하는 방법은 기존의 프로젝트에서 설계된 .sof 파일을 변환하여 사용해 주면 됩니다. 따라서 Quartus II의 메뉴의 File -> Convert Programming Files에서 이 작업을 수행해 주면 됩니다. 여기에서는 Programming file type는 JTAG Indirect Configuration File (.jic) 로 변경해 주시고, Configuration device는 보드에서 사용하고 있는 EPCS16을 선택해 주셔야 합니다.

또한 File name는 사용자가 만들려는 적당한 이름을 주면 됩니다. 여기서 선택한 File name이 현재 작업하는 프로젝트 폴더에 생성되게 됩니다. 확장자는 .jic로 생성이 됩니다. 또한 Flash Loader를 선택하여 오른쪽에 활성화 되는 Add Device 버튼을 눌러 현재 사용하는 FPGA 디바이스인 EP2C70를 선택해 주고, SOF Data는 똑 같은 방법으로 Add File를 통해 현재 프로젝트에서 설계한 결과 파일인 .sof 파일을 넣어 주면 됩니다. 이상으로 모든 것에 대한 설정이 마쳤으면, Generate 버튼을 눌러 EPCS16에 프로그램 할 수 있는 파일을 생성하게 됩니다. [그림 4-56]에는 파이변환에 관한 창을 보여주고 있고, [그림 4-57]에는 이 결과 파일을 가지고 EPCS16 디바이스에 프로그램하는 과정을 보여주고 있습니다.



[그림 4-56] Convert Programming Files



[그림 4-57] Configuration ROM Programmer

4.2.2 ISE를 이용한 설계

1) 역할

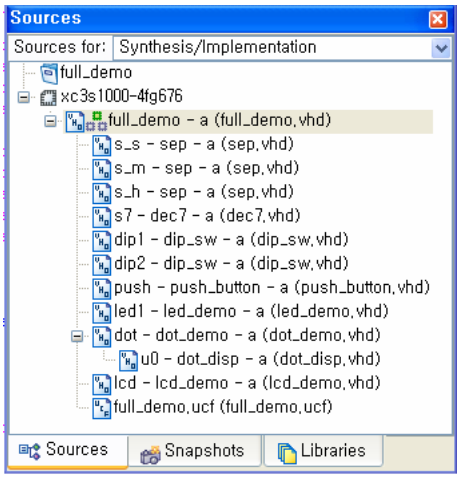
Xilinx ISE 프로그램도 Quartus II와 마찬가지로 프로젝트를 선언하여 프로젝트 중심으로 관리를 하도록 되어 있습니다. 따라서 프로젝트에 속한 설계 파일, 컴파일 관련 파일, 시뮬레이션 파일, 핀 정보 등의 일련의 파일들이 하나의 프로젝트에 속해 관리를 하게 됩니다. 이렇게 구성이 되어 있어, 하나의 프로젝트 파일을 오픈 하였을 때, 프로젝트에 관련된 모든 파일을 불러와서 정보를 보고 사용할 수 있도록 구성을 하고 있습니다.

[그림 4-58]은 장비의 Full demo 예제의 VHDL로 작성한 소스 코드를 보여주고 있으며, [그림 4-59]에서는 프로젝트에서 관련된 소스파일들을 보여주고 있습니다.

full_demo.vhd

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity full_demo is
5  port(
6     clk         : in std_logic;
7     button_sw   : in std_logic_vector(15 downto 0);
8     dip_switch  : in std_logic_vector(15 downto 0);
9
10    led         : out std_logic_vector(15 downto 0);
11    seg_data : out std_logic_vector(7 downto 0);
12    seg_com     : out std_logic_vector(11 downto 0);
13    lcd_data : out std_logic_vector(7 downto 0);
14    lcd_e    : out std_logic;
15    lcd_rs      : out std_logic;
16    lcd_rw      : out std_logic;
17    dot_data : out std_logic_vector(20 downto 0);
18    dot_scan : out std_logic_vector(14 downto 0);
19    step     : out std_logic_vector(3 downto 0);
20    piezo    : out std_logic);
21 end full_demo;
22
23 architecture a of full_demo is
24
25 component led_demo
26 port(
27    clk   : in std_logic;
28    led   : out std_logic_vector(15 downto 0));
29 end component;
30
31 component dot_demo
32 port(
33    clk      : in std_logic;
34    dot_d : out std_logic_vector(20 downto 0);
35    dot_scan: out std_logic_vector(14 downto 0));
36 end component;
37
38 component lcd_demo
```

[그림 4-58] full demo 예제

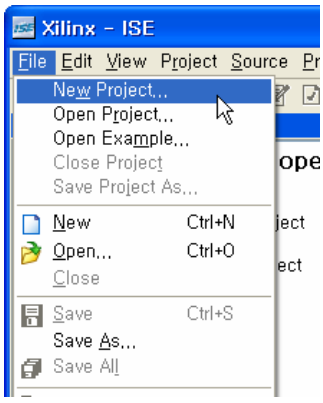


[그림 4-59] Source Files

2) Project 선언

Xilinx ISE도 Quartus II에서 설명한 예제를 그대로 설명 하도록 하겠습니다.

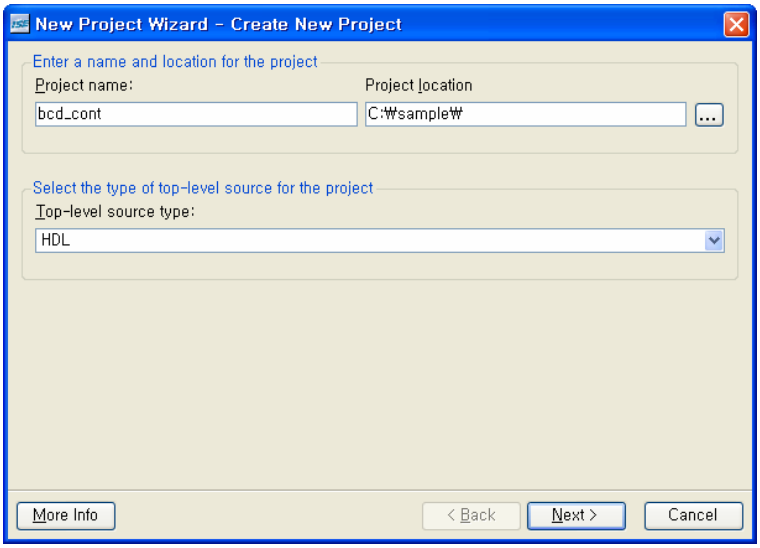
[그림 4-58]와 같이 ISE 프로그램은 File -> New Project 메뉴를 선택하여 프로젝트를 선언할 수 있습니다. 프로젝트 선언은 프로젝트 명 선언에서부터 디바이스 선택에서 파일 추가까지 총 5단계로 이루어져 있는데, 이 과정을 거쳐야 실제 코딩을 위한 프로젝트 선언 과정이 완료됩니다.



[그림 4-60] New Project

각 단계에서 다음 단계로 진행하기 위해서는 Next 버튼을 클릭해야 합니다. 프로젝트 선언을 시작하면 프로젝트에 대한 전반적인 진행 사항을 설명하는 창이 활성화되고 Next 버튼에 의해 다음 프로젝트 선언을 시작하게 됩니다.

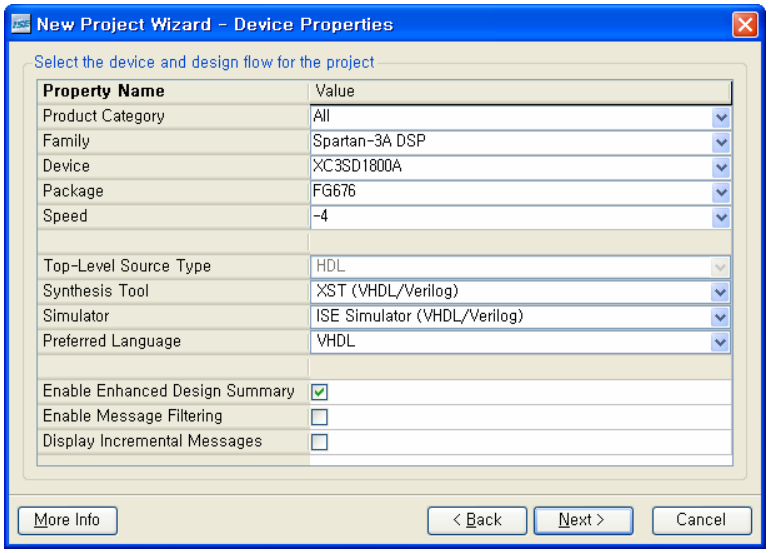
첫 단계로 프로젝트 관련 파일들의 Project Name, Project Location, Top-Level Source Type을 설정 해 주어야 합니다. Project Name을 선택할 때는 Project Location에도 같은 이름의 폴더가 자동으로 생성하여 그 폴더 내에서 관리 하도록 하고 있습니다. Top-Level Source Type는 현재 프로젝트의 top 파일의 타입을 설정해 주는 것입니다. 여기에는 HDL, Schematic, EDIF, NGC/NGO의 파일 형태를 선택해 주면 됩니다. [그림 4-61]에서는 프로젝트의 이름을 bcd_cont로 설정해 주었고, 이에 관련된 파일들은 C:₩sample에 저장되도록 설정했습니다. 또한 top 파일을 VHDL파일 형태로 설계를 할 것이기 때문에 Top-Level Source Type을 HDL로 설정한 것입니다.



[그림 4-61] Create New Project

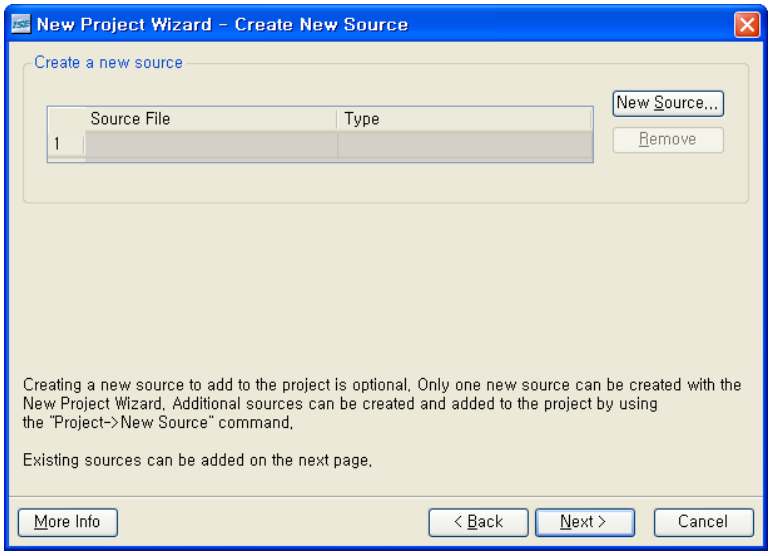
다음은 프로젝트 설정 단계에서 디바이스 설정 및 EDA 툴 설정을 하는 단계입니다. 따라서 현재 사용하는 디바이스를 선택하고 합성과 시뮬레이션 툴을 선택하여 설정해 주는 작업을 하는 단계라고 볼 수 있습니다.

[그림 4-62]에서는 현재 작업을 설정한 모습을 보여주고 있습니다. Spartan-3A DSP Family의 180만 게이트 676핀 사양을 가진 디바이스를 선택한 모습을 보여주고 있으며, 합성과 시뮬레이션 툴은 ISE에서 지원해 주는 것을 사용하고 있습니다. 아래 그림에서 이와 같은 설정 모습을 볼 수 있습니다.



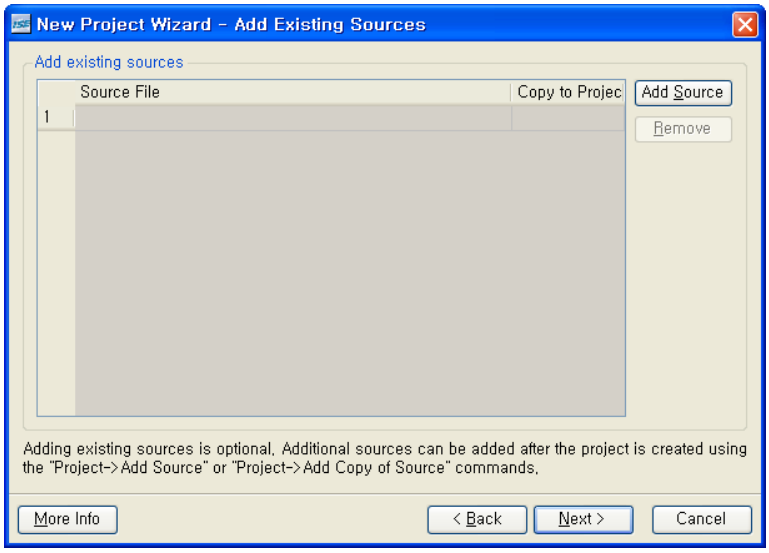
[그림 4-62] Device Properties

다음 과정은 프로젝트 설정 단계에서 프로젝트에 추가 할 파일 중 새로 생성할 파일의 이름과 타입을 적어주는 단계입니다. 따라서 설계자가 어떤 파일을 생성하여 설계를 할지 미리 생각하고 프로젝트를 선언하고 있다면 이 단계에서 새로 생성할 파일을 적어 주면 됩니다. 새로 생성할 파일을 현 단계에서 넣어 주어도 되고, 프로젝트 선언이 끝난 다음에서도 추가가 가능합니다. [그림 4-63]에서는 Create New Source단계의 창을 보여주고 있습니다.



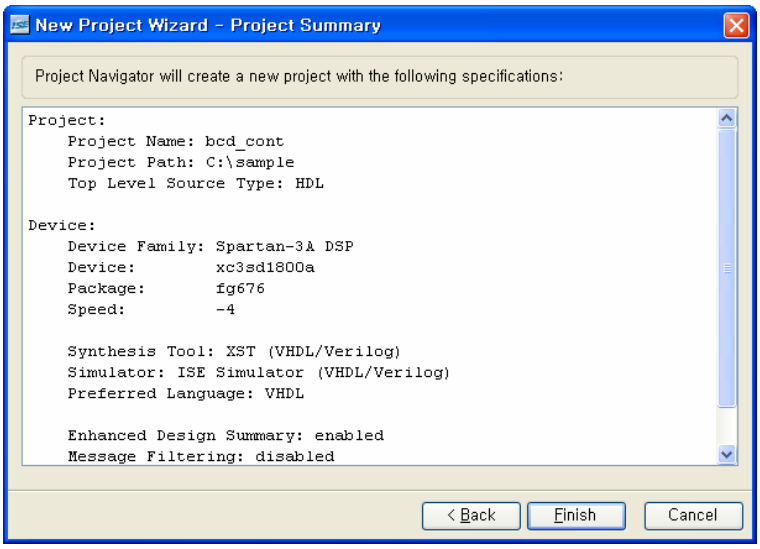
[그림 4-63] Create New Source

현 단계는 이전 단계와 비슷한 소스 파일을 추가하는 단계이지만 다른 점은 미리 설계 된 소스 파일을 추가해 주는 단계입니다. 따라서 사용자가 미리 설계된 상위 또는 하위 파일들이 있을 시 이것을 현재 생성하려는 프로젝트에 추가해 주는 부분입니다. 이 부분도 프로젝트를 생성한 후에도 설정이 가능합니다. [그림 4-64]에는 Add Existing Sources 단계를 보여주고 있습니다.



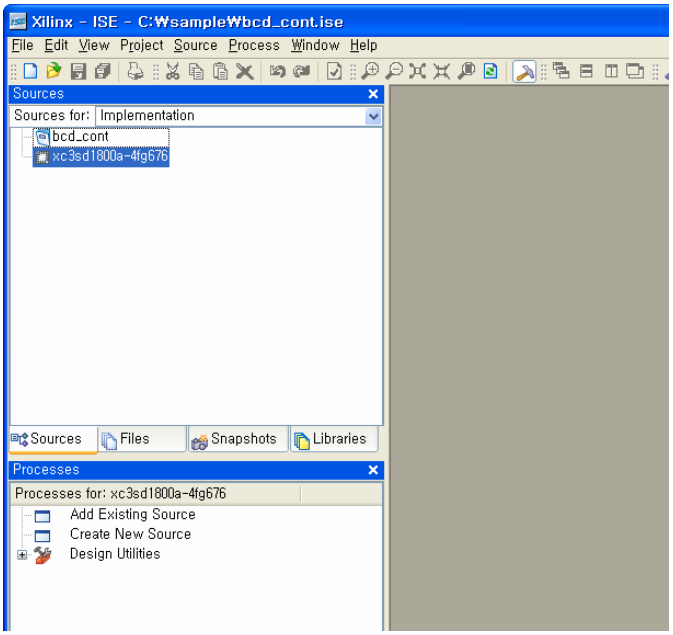
[그림 4-64] Add Existing Sources

이 단계는 프로젝트 설정의 마지막 단계로써 프로젝트 설정 단계에 대한 Summary를 보여주는 창입니다. 따라서 이전 단계에 프로젝트 설정 정보를 다시 확인해 보면서 잘 못된 부분이 없는지 다시 확인하는 단계입니다. 현 단계에서 잘 못 설정된 부분을 있으면 Back 버튼에 의해 이전 단계로 이동하여 수정을 하고, 이상이 없으면 Finish 버튼을 눌러 프로젝트 선언을 마치면 되겠습니다.



[그림 4-65] Project Summary

이전까지의 작업으로 현 작업에 대한 프로젝트 선언 작업을 마쳤습니다. 따라서 이 후에 만들어지는 모든 파일들은 현재의 프로젝트 폴더 내에서 되고 관리가 될 것입니다. [그림 4-64]에서는 프로젝트 설정을 마치고 ISE창에 설정된 모습을 보여주고 있습니다. Sources창에서 현재 설정된 프로젝트인 watch와 디바이스인 Spartan3 계열의 디바이스가 보여지고 있습니다.



[그림 4-66] 타이틀에 표시된 프로젝트

3) Design Entry

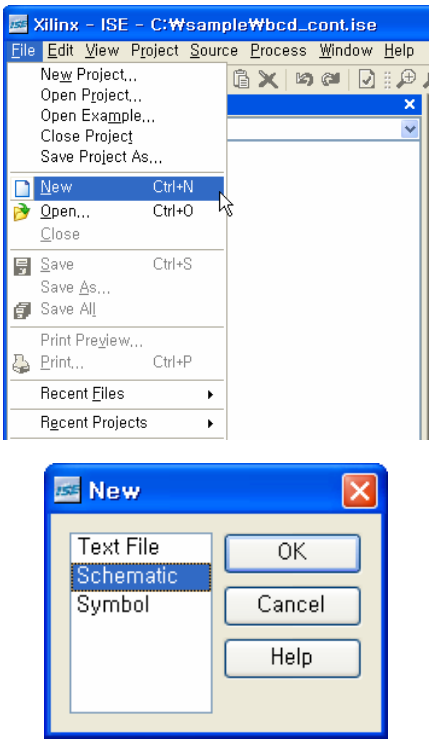
❑ Schematic Editor

① 특징

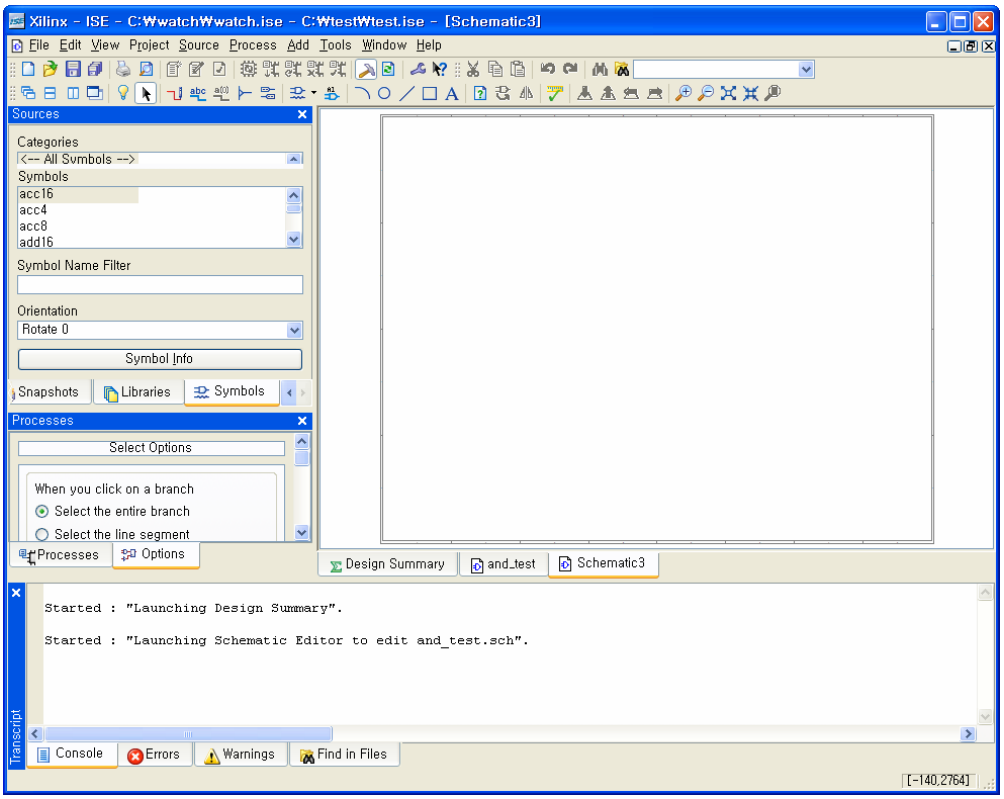
특정 기능을 가진 심볼 라이브러리를 생성하여, 그 심볼 라이브러리의 입 출력 데이터 라인을 선(wire) 등으로 각 심볼 등의 입 출력 데이터 신호를 연결하여 논리 회로를 설계하는 방법입니다.

② Schematic 선택하기

Schematic을 이용해 하나의 논리 회로를 설계하기 위해서는 Schematic Design 창을 활성화 시켜야 하는데, [그림 4-67]과 같이 File -> New 메뉴를 선택하여 New 창을 활성화 한 후 Schematic이라는 항목을 선택하여 OK 버튼을 누르면 [그림 4-68]과 같이 Schematic Editor 창이 활성화 됩니다



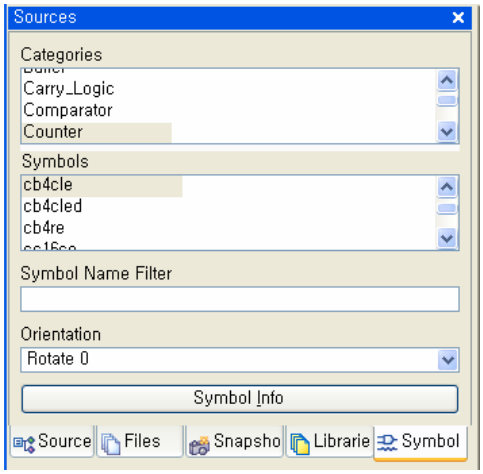
[그림 4-67] 설계 메뉴에서의 Schematic 선택



[그림 4-68] Schematic Editor

③ 심볼 라이브러리

Schematic 으로 작업할 수 있는 환경이 되었으면 이곳에 symbol을 삽입해 주어야 합니다. 따라서 Sources 창에서 아래쪽에 Symbols를 선택하면 아래 [그리 4-69]와 같은 창이 활성화 되고, Symbol Name Filter에서 찾고자 하는 Symbol을 검색하여 찾으면 됩니다. 그림에서는 cb4cle라는 심블을 선택하는 부분을 보여주고 있습니다. 또한 이 창에서는 Symbol을 회전할 수 있는 Orientation 설정 부분과, 현재 선택한 Symbol에 대한 정보를 확인 할 수 있는 Symbol Info 버튼이 있습니다.

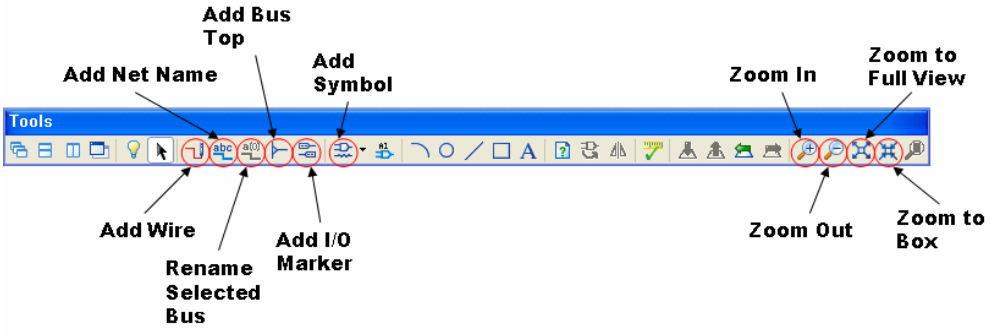


[그림 4-69] Symbols

사용자가 심볼의 이름을 알고 있을 때 직접 Symbol의 이름으로 검색하여 불러와 사용이 가능하고 Symbols에서 직접 심볼을 검색하여 찾아 설계 소스에 추가해 주면 됩니다.

④ 각 심볼의 연결

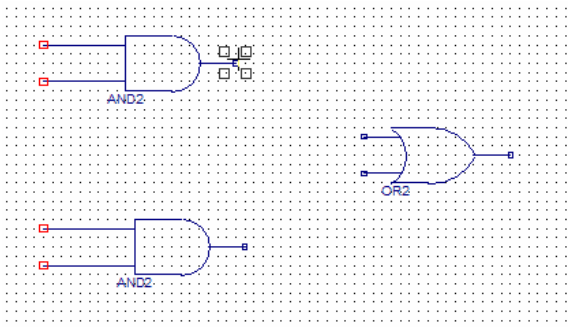
이전의 작업으로 심볼을 불러오면 심볼간의 연결을 해 주어야 합니다. 또한 심볼을 디바이스의 I/O 핀과 연결할 수 있는 포트를 연결해 주어야 합니다. [그림 4-70]은 심볼을 이용하여 회로를 설계하는데 사용하는 아이콘들이 나와 있습니다.



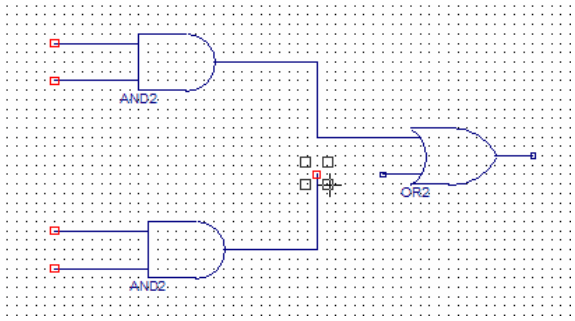
[그림 4-70] Tools

[그림 4-70]과 같이 자주 사용하는 아이콘에서는 Wire를 추가해 주는 부분과, 각 Wire간의 이름을 주는 아이콘, 심볼을 추가하는 아이콘, 화면에 대한 확대, 축소하는 아이콘들이 포함되어 있습니다. 따라서 설계자가 이러한 아이콘을 통해 Schematic으로 설계를 하는데 쉽게 할 수 있습니다.

[그림 4-71]에서는 심볼들간의 연결하는 모습을 볼 수 있습니다. 왼쪽 상단의 AND2에 마우스를 가져가면, 그림과 같이 사각형 모양의 그림이 4개가 생기게 됩니다. 이것은 심볼의 핀과 연결하여 Wire을 생성하겠다는 표시입니다. 따라서 이 상태로 Wire을 생성하면서 연결하려는 심볼의 핀까지 마우스로 드래그 해 주면 됩니다. 만약 마우스를 커서의 모양이 바뀌지 않으면, 아이콘에서 Add Wire에 대한 것을 선택하였는지 다시 한번 확인해 보시기 바랍니다.

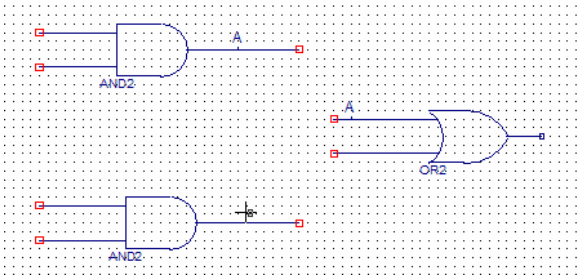


[그림 4-71] 마우스 포인터의 변화



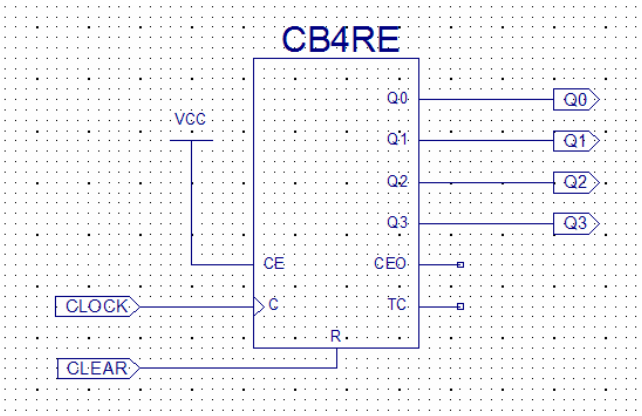
[그림 4-72] Wire 연결

이름으로 연결할 경우에는 연결하고자 하는 Node를 [그림 4-73]과 같이 Symbol에서 wire을 일정 부분 연결한 후 마우스로 이름을 부여하는 모습을 보여주고 있습니다. 메뉴의 Add -> Net Name을 선택한 후에 ISE의 Processes 창에서 Name의 이름을 적어주면 화면의 마우스 커서 옆에 이름이 표시되게 됩니다. 이 상태에서 미리 그려 둔 Wire에 마우스로 클릭하면 Wire에 이름이 표시되게 됩니다.



[그림 4-73] 이름으로 연결

다음 그림에는 현재 예제 소스로 사용하기 위한 4bit count의 회로 설계 모습을 보여주고 있습니다. CB4RE, VCC 심블을 이용하였고, 심블들 간의 Wire를 통해 연결해 주었습니다. 또한 입, 출력 포트는 Add I/O Marker을 통해 설계를 해 주었습니다. I/O 포트의 이름은 마우스로 선택하여 하나 씩 이름을 지정해 주었습니다.



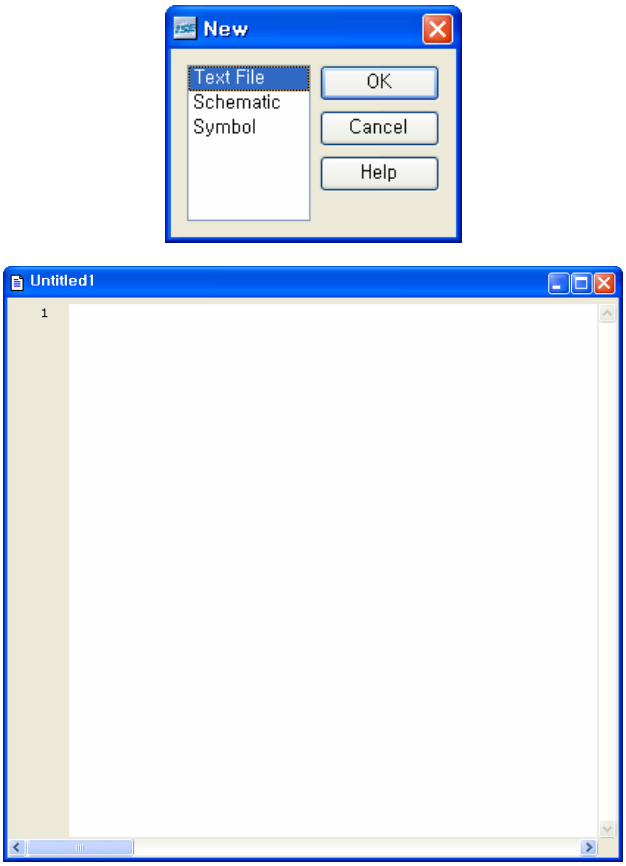
[그림 4-74] 4 bit Count

❑ Text Editor (HDL에 의한 설계)

HDL에 의한 설계의 기본 내용은 Quartus II에 설명이 되어 있으므로 ISE 설명에서는 생략하고 설계 방법으로 바로 넘어가도록 하겠습니다.

① Text File 선택하기

Text의 HDL을 이용하여 설계하기 위해서는 Text File 창을 활성화 시켜야 하는데, File -〉 New 메뉴를 선택하면, [그림 4-75]와 같이 어떠한 형태로 설계를 할 것인지 선택하게 된다. 여기에서 Text File를 선택하면 그림과 같이 설계를 할 수 있는 창이 활성화 됩니다. 여기에서 사용자가 HDL을 이용하여 설계를 하고 저장을 하면 됩니다.

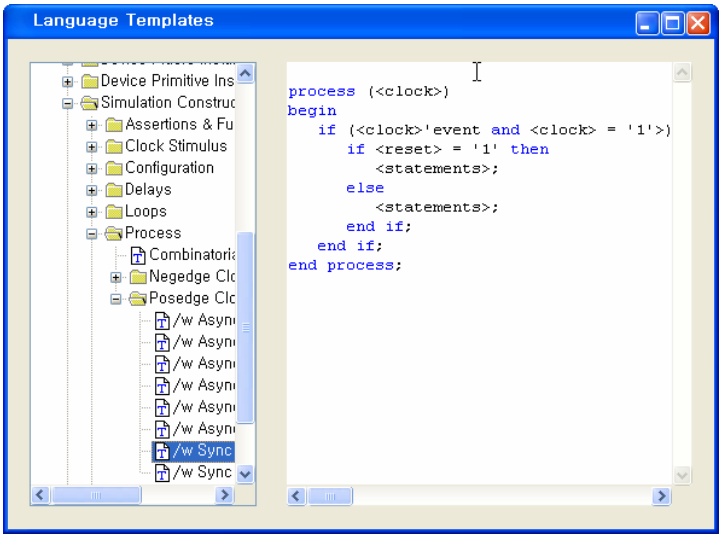


[그림 4-75] Text File

② 설계하기

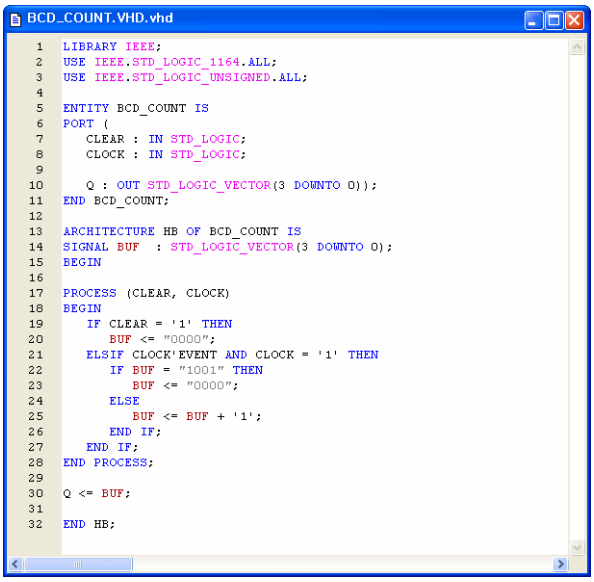
HDL을 이용한 설계 방법에는 VHDL, Verilog HDL에 의한 설계 방법이 있습니다. 이러한 예제는 Quartus II 툴 설명에서 나온 예제 그림과 같습니다. 저장하는 파일 이름도 Quartus II와 같은 VHDL 파일은 .vhd, Verilog HDL 파일은 .v의 확장자를 가지고 저장합니다.

여기에서도 Quartus II와 마찬가지로 메뉴의 Edit -〉 Language Templates가 존재하여 각 언어에 대한 기본적인 형태를 지원해 주고 있습니다. 따라서 사용자는 이러한 메뉴를 이용하여 어떠한 형태로 설계할 지 알 수 있고 간단하게 HDL을 작성할 수 있습니다. [그림 4-76]은 Language Templates를 보여주고 있습니다.



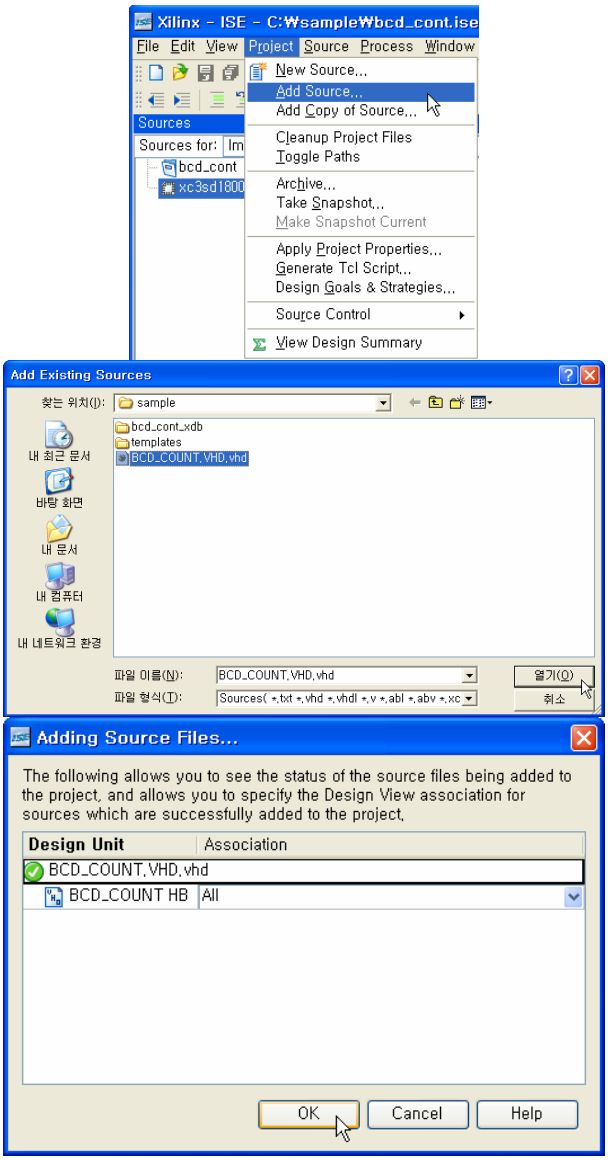
[그림 4-76] Language Templates

여기에서도 Quartus II에서 설명한 예제 소스인 BCD_COUNT를 이용하여 설명하도록 하겠습니다. 다음 그림에서 설계 소스를 보여주고 있습니다.



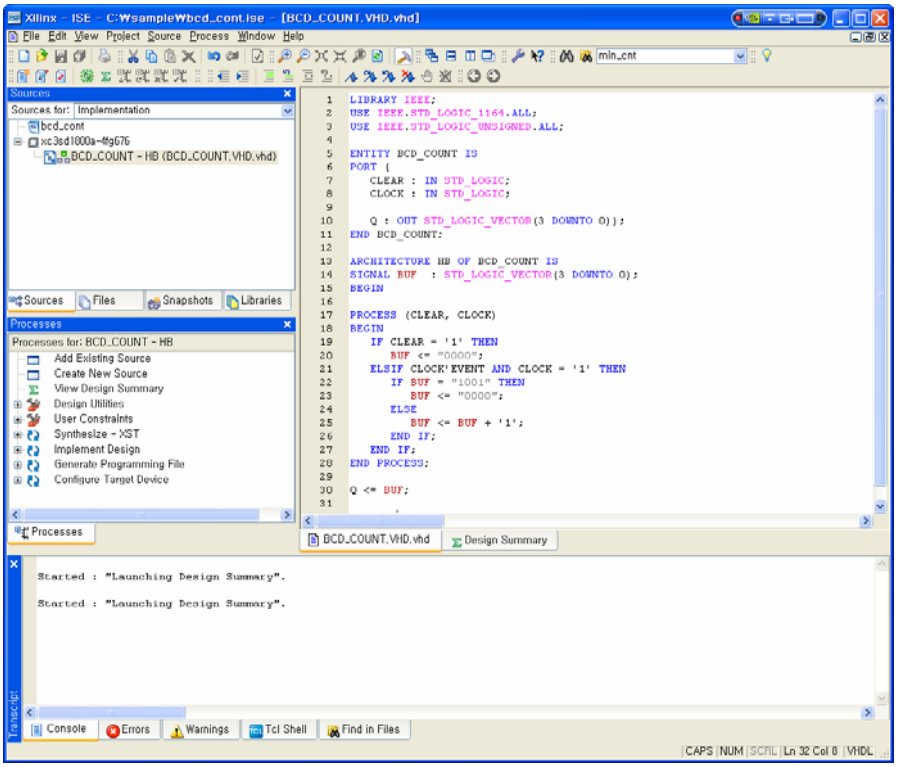
[그림 4-77] BCD_COUNT Source

소스 설계가 끝나고 파일을 저장했으면, 프로젝트에 현재 설계한 소스 파일을 추가해 주어야 합니다. 이 작업은 메뉴에의 Project -> Add Source를 통해 현재 설계한 파일을 추가할 수 있습니다. 다음의 그림에서 파일을 추가하는 모습을 보여주고 있습니다.



[그림 4-78] Adding Source

위의 그림에서 볼 수 있듯이 Add Source의 메뉴를 통해 3단계를 거치면서 현재 설계한 BCD_COUNT.VHD를 추가하면 다음의 그림과 같이 현재 현 프로젝트에 등록된 모습을 보여주고 있습니다.



[그림 4-79] Add Source

위의 그림과 같이 소스를 추가하면 Process창에 새로운 메뉴가 생깁니다. 이러한 메뉴는 컴파일 과정에서 사용하는 것으로 메뉴가 활성화가 되는지 꼭 확인하셔야 합니다.

4) Compile

① 역할

컴파일러는 과정은 Quartus II에서 설명한 같은 목적을 가지고 수행을 하게 됩니다. 여기에서는 Xilinx ISE에서의 컴파일 과정을 살펴보기로 하겠습니다.

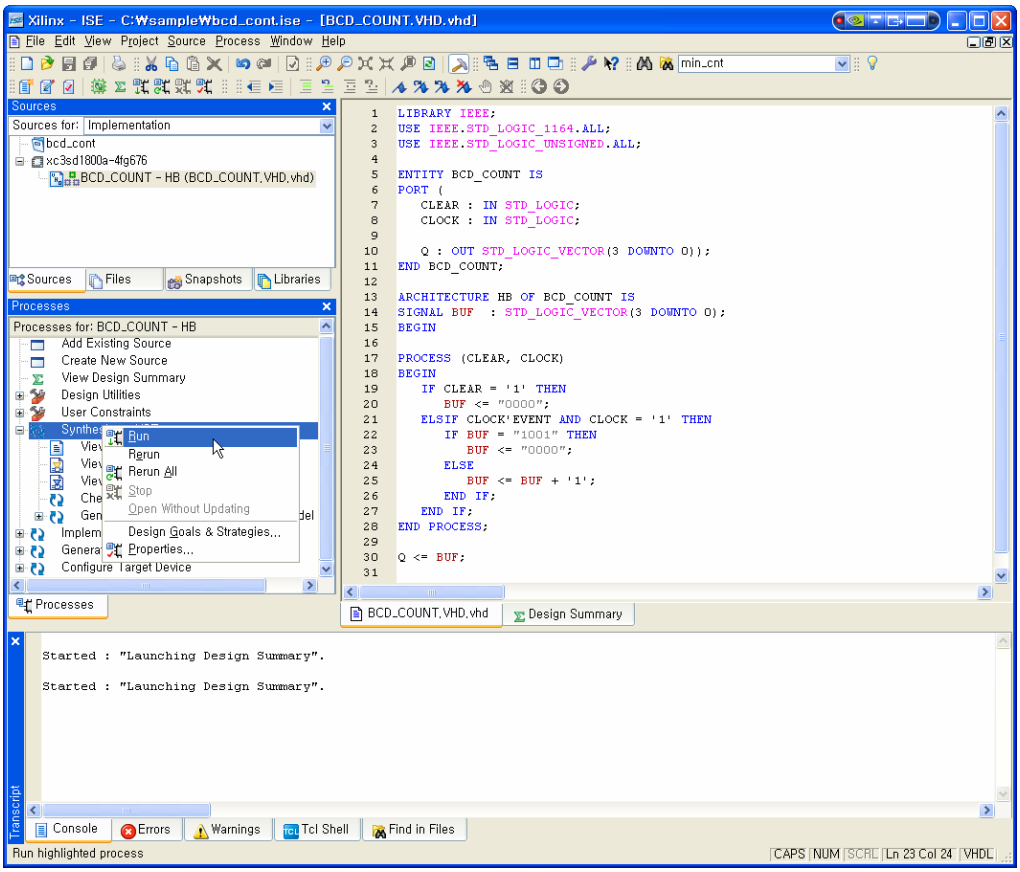
② Compiler

ISE에서는 컴파일 과정을 크게 4부분으로 나누고 있습니다. ISE Processes창에서 볼 수 있듯이 Synthesize, Implement Design, Generate Programming File, Configure Target Device로 나뉘고 있습니다. Synthesize는 현재 설계한 회로에 대한 부분을 확인하는 작업을 하게 됩니다. 따라서 사용자의 설계 파일을 보고 컴파일을 하는 부분입니다. 설계 소스를 보고 문법적인 오류에 대한 검사를 하고 이에 대한 합성을 하는 단계 입니다. Implement Design은 디바이스를 보고 컴파일 하는 과정입니다.

따라서 디바이스에 대한 디바이스의 로직에 이전에 컴파일 한 소스를 배치하는 단계

입니다. 또한 디바이스의 핀에서 어떠한 핀을 사용하여 제어를 할 지 결정하는 단계입니다. 여기에서 설계 소스에서 사용하는 핀을 지정하여 사용할 수 있도록 활성화 시켜주는 작업을 하게 됩니다. Generate Programming File은 프로그램에 관련된 작업을 하게 됩니다. 따라서 프로그램 할 수 있는 파일을 생성하는 작업을 하게 됩니다. 또한 Configure Target Device는 타깃 디바이스에 프로그램 하는 과정입니다. 이 과정에서 FPGA 디바이스 이외의 Configuration ROM에 대한 프로그램 파일도 생성해 줄 수 있습니다.

이렇게 나눠져 있는 컴파일 수행 작업은 메뉴의 Process 또는 툴의 Processes 창에서 실행을 할 수 있습니다. 위에 있는 각각의 작업에 마우스 오른쪽 키를 누르면 팝업 창이 활성화되는데, 여기에서 run을 클릭하여 각각의 컴파일 작업을 수행하게 됩니다. [그림 4-80]에서는 이러한 컴파일 작업을 보여주고 있습니다. 여기에서는 Synthesis 작업에 대한 컴파일 수행 작업을 보여주고 있습니다. 나머지도 이 작업과 똑같이 수행해 주면 됩니다.



[그림 4-80] Compiler

5) Assignments

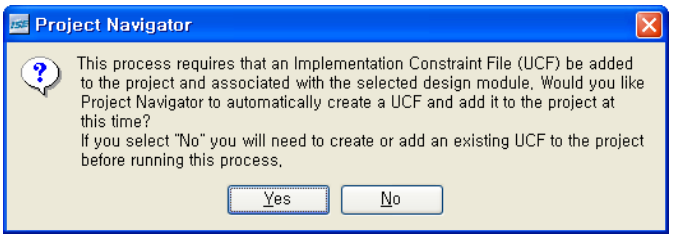
① 역할

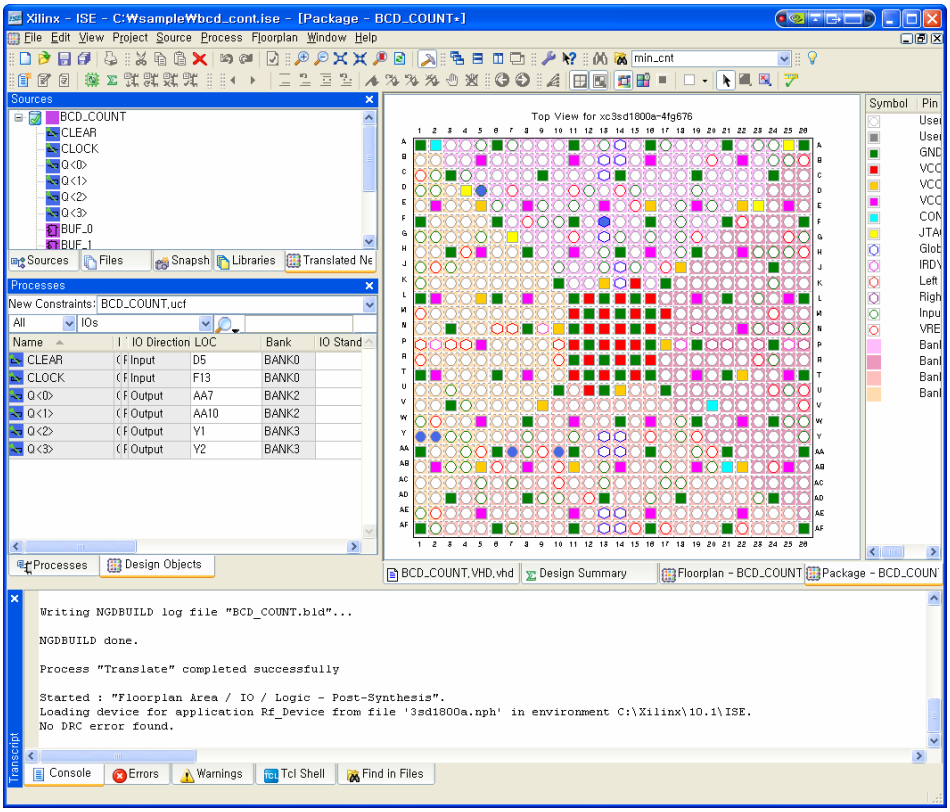
이 작업은 디바이스의 I/O핀 설정을 위한 작업입니다. Quartus II에 설명을 참고해 보기 바랍니다.

② Assign Package Pins

프로젝트 설정 단계에서 디바이스 설정을 미리 해 두었기 때문에 여기에서는 핀에 대한 설정만 해 주면 됩니다. 컴파일 과정에서 컴파일러가 포트에 대한 정보를 가지게 됩니다. 따라서 핀 설정 과정에서 사용하려는 핀을 불러와서 타겟으로 사용하는 디바이스에 Assign 하게 되는 것입니다.

이 작업은 Processes 창에서 할 수 있습니다. Processes 메뉴에서 User Constraints -〉 Floorplan Area / IO / Logic Post-Sunthesis에서 이러한 작업을 수행하게 됩니다. 이 메뉴를 마우스로 클릭하면 확장자가 .ucf 파일을 생성하면서 핀을 할당하는 새로운 창이 활성화 되는데 이 창에서 Yes 를 클릭하시면 됩니다. 그러면 Process에 Design Objects창이 활성화 되고, 여기에서 핀을 할당을 할 수 있습니다. 이 창에서는 Design Object List에 이전에 설계한 회로의 I/O Name이 미리 지정이 되어 있습니다. 따라서 사용자는 해당되는 I/O Name에 핀을 할당해 주면 됩니다. Design Object List에서 LOC에 핀 번호를 적어서 할당해 주면 됩니다. [그림 4-81]에서는 핀이 할당된 모습을 보여주고 있습니다. 핀 번호는 장비에서 메인 클럭과 버튼 스위치, LED를 통해 할당해 주었습니다.





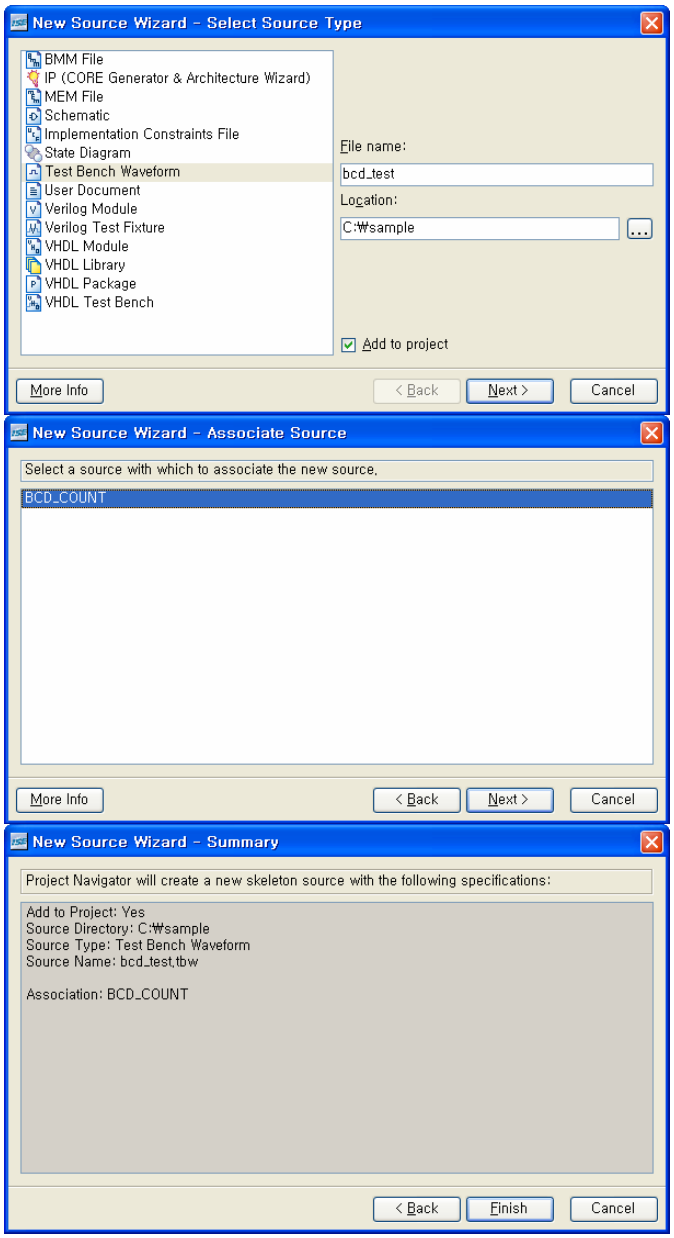
[그림 4-81] Assign Package Pins

6) Simulation

ISE 시뮬레이션도 툴 내부에서 지원해 주고 있습니다. 이것도 또한 장비의 설계 단계에서 사용자가 원하는 동작을 하는 지 파형적으로 검증해 보기 위해 시뮬레이션을 통해 분석하게 됩니다. 따라서 입력 따른 출력 파형이 사용자가 의도한 데로 동작을 하는지 이번 단계에서 검증하게 됩니다.

이번 단계에서는 시뮬레이션을 하기 위한 첫 번째 단계로 시뮬레이션 구동을 위해 입력 파형을 정의해 주어야 합니다. 따라서 이러한 파형을 프로젝트에 적용해 주기 위한 파일을 생성해 주어야 합니다. 따라서 메뉴의 Project -〉 New Source를 선택합니다.

다음 그림에서 Test Bench Waveform을 선택하고 오른쪽에 File name에서 생성할 파일 이름을 기입합니다.

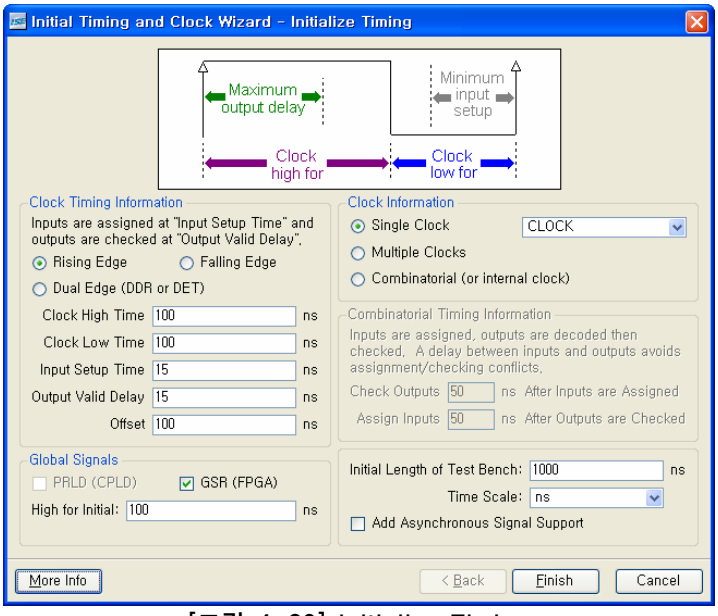


[그림 4-82] Test Bench Waveform

Test Bench Waveform 파일을 생성하기 위해서는 총 3단계로 작업이 이루어 집니다. 첫 번째 단계는 앞서 설명한 것과 같이 파일 이름 및 저장될 폴더를 지정해 주고, 다음 단계로 현재 시뮬레이션이 이루어질 설계 파일을 지정한다. 마지막 창에 이러한 설정을 정리해서 보여주게 됩니다.

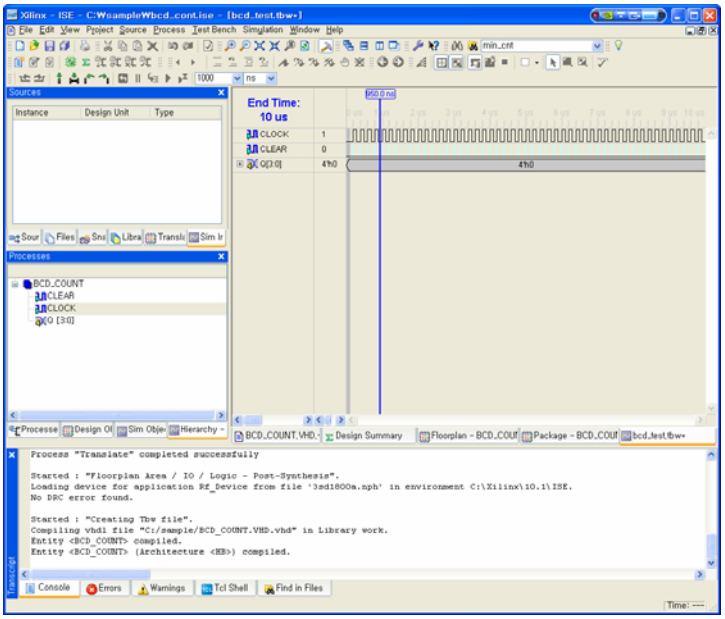
Finish 버튼으로 파일을 생성하면 다음 창에서는 입력 클럭에 대한 정의를 해 주게 됩

니다. 여기에서는 클럭에 따른 동작 시점과 클럭의 주파수 정보를 이 창에서 설정 해 줄 수 있습니다.



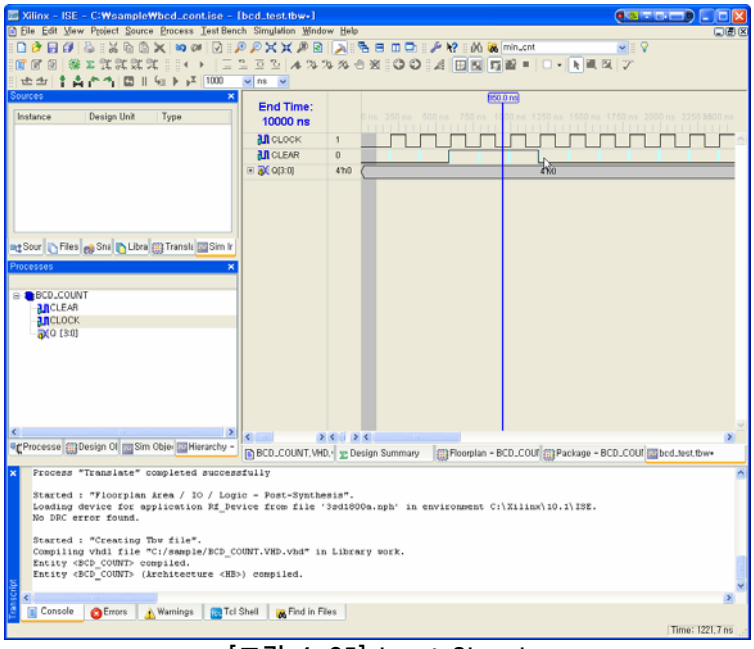
[그림 4-83] Initialize Timing

위와 같이 설정이 끝나면 다음 그림과 같은 창이 보여지게 됩니다.



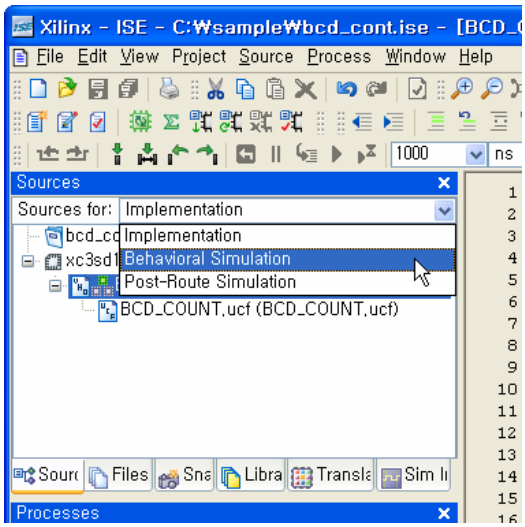
[그림 4-84] Create Waveform

이렇게 입력 클럭에 대한 정의를 마치면 CLEAR에 대한 파형도 정의해 주어야 합니다. 이를 위해서는 마우스로 CLEAR의 파형이 있는 쪽을 마우스로 클릭하면 파형이 반전됩니다. 이러한 방식으로 기본 0 의 값을 가지고 있는 파형에 마우스를 이용하여 입력 파형을 정의해 줍니다. 다음 그림에서 입력 파형의 정의가 완성된 것을 확인할 수 있습니다.



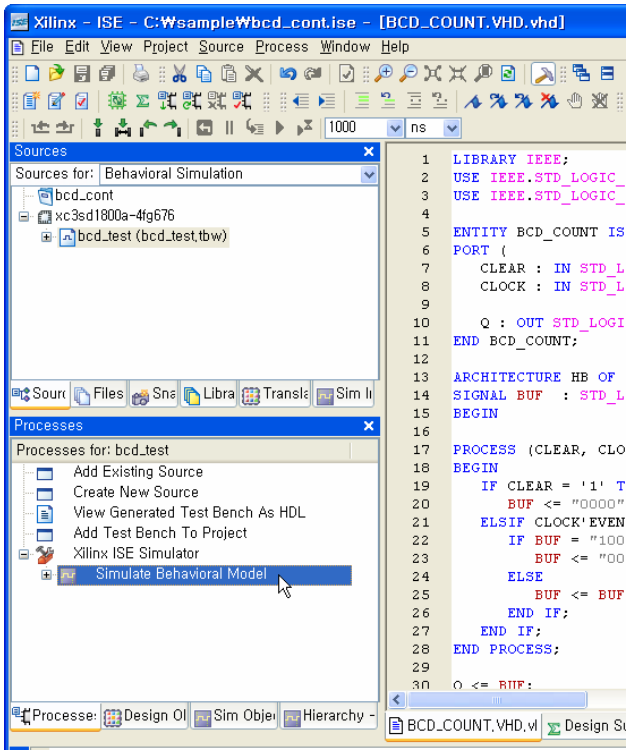
[그림 4-85] Input Signal

이상으로 시뮬레이션을 위한 준비 작업을 마쳤습니다. 이제부터 시뮬레이션을 시작하도록 하겠습니다. 시뮬레이션을 위해 먼저 Source 창에 있는 Source for의 Implementation을 Behavioral Simulation으로 변경해 주어야 합니다. 다음 창에 이러한 작업이 나와 있습니다.



[그림 4-86] Behavioral Simulation

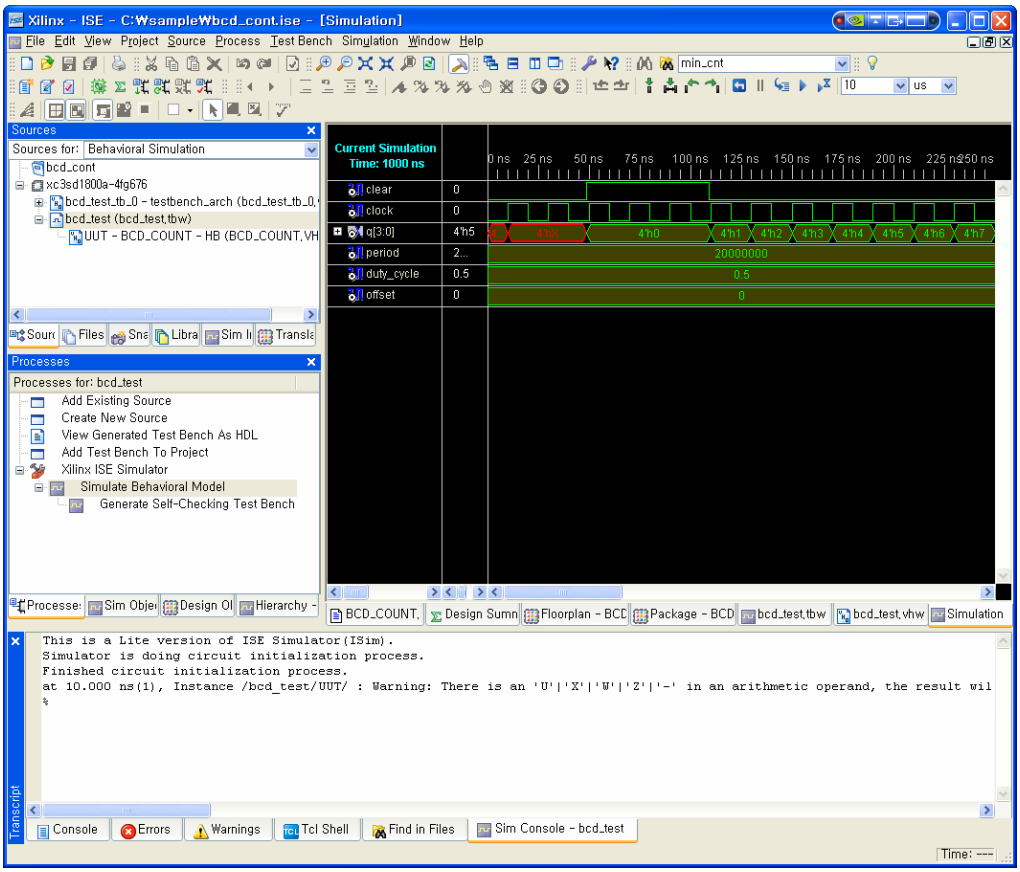
이렇게 설정하면 아래 쪽의 Processes창도 변하게 되는데, Xilinx ISE Simulator의 하위 메뉴인 Simulate Behavioral Model을 클릭하여 시뮬레이션을 실행하시면 됩니다.



[그림 4-87] Start Simulation

이와 같이 시뮬레이션을 동작 시키면 다음 그림과 같이 설계된 파형의 시뮬레이션 결

과 같이 보여집니다. BCD COUNTER를 설계한 것이기 때문에 CLEAR이 1 인 구간에서는 0 으로 출력 되고, 나머지 구간에서는 클럭이 상승 구간에서 출력이 1 씩 증가하게 됩니다. 다음 파형에서 이러한 결과값을 확인해 보도록 합니다.



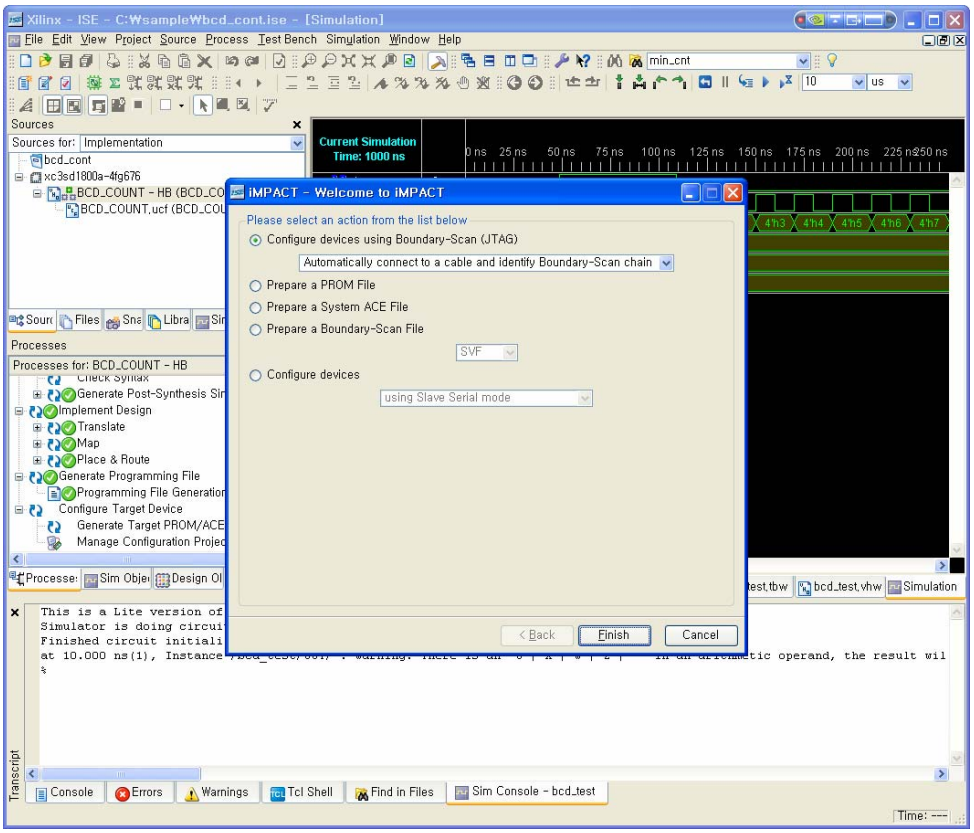
[그림 4-88] Simulation End

7) Configure Device (iMPACT)

Configure Device 작업은 현재까지의 툴에서의 작업을 직접 디바이스에 다운하여 디바이스를 직접 동작해 보는 작업입니다. 따라서 이전까지의 툴에서의 설계 작업을 통한 결과 파일을 디바이스에 직접 다운하여 동작을 시켜 보는 작업이라 할 수 있습니다. ISE에서는 iMPACT를 통해서 이러한 작업을 할 수 있습니다.

이전까지의 설계와 컴파일 작업이 끝난 상태에서 Generate Programming File을 실행하면 프로그램 할 수 있는 파일이 생성됩니다. 그리고 Configure Target Device -〉 Manage Configuration Project(iMPACT)를 마우스 오른쪽 키를 이용하여 생성되는 팝업 메뉴에서 RUN을 실행하면 프로그램을 위한 작업 창이 활성화 됩니다. 다음 [그림 4-89]에서는 iMPACT을 실행 시 초기 화면을 보여주고 있습니다.

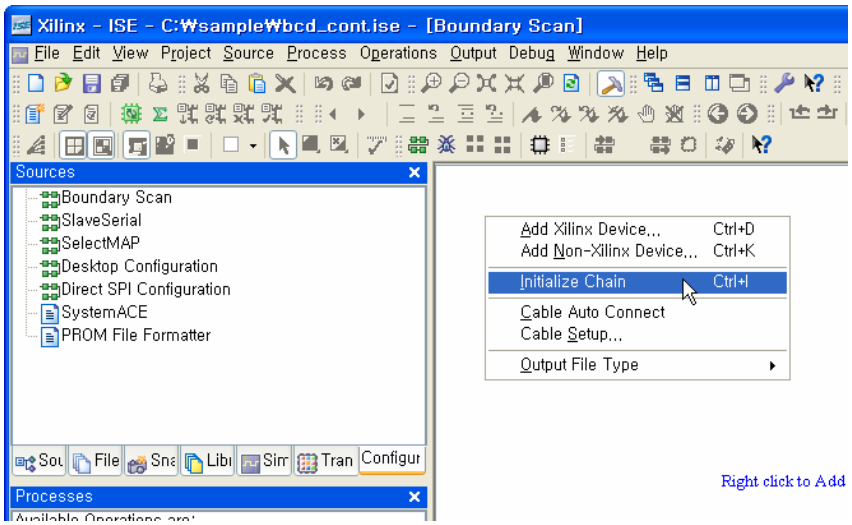
만약 실행 시 이전 단계의 컴파일 단계 중 빠진 것이 있으면, 앞 단계의 컴파일 과정이 모두 끝난 뒤 iMPACT이 실행 됩니다.



[그림 4-89] iMPACT

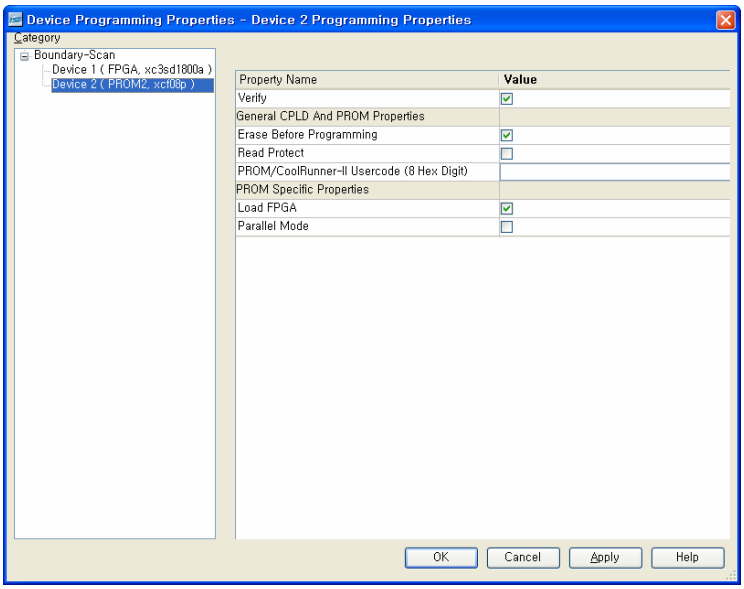
여기에서는 다운하려는 디바이스에 대한 다운 방식을 선택해 주는 작업입니다. 모드와 사용하려는 디바이스가 어떠한 것인지 선택을 해 주어야 합니다. 현재 장비에서는 Spartan-3A DSP군의 디바이스를 JTAG 방식으로 다운하는 모드를 취하고 있으므로, 처음에 있는 Configure devices using Boundary-Scan(JTAG)을 선택해서 Finish 버튼을 눌러 주면 됩니다.

이렇게 하면 다음 그림과 같은 창이 활성화 되는데 이때 오른 쪽 메인 창에서 마우스 오른 쪽 키를 이용하여 팝업 메뉴를 활성화 합니다. 이 메뉴에서 Initialize Chain을 선택합니다. 이 메뉴를 선택하기 이전에 PC와 장비간의 다운로드 케이블을 통해 연결되어 있고, 장비의 전원은 On 상태로 해 주셔야 합니다. 만약 이렇게 되어 있지 않으면 PC에서 다운로드 할 디바이스를 검색하지 못해 에러 메시지를 띄우게 됩니다.



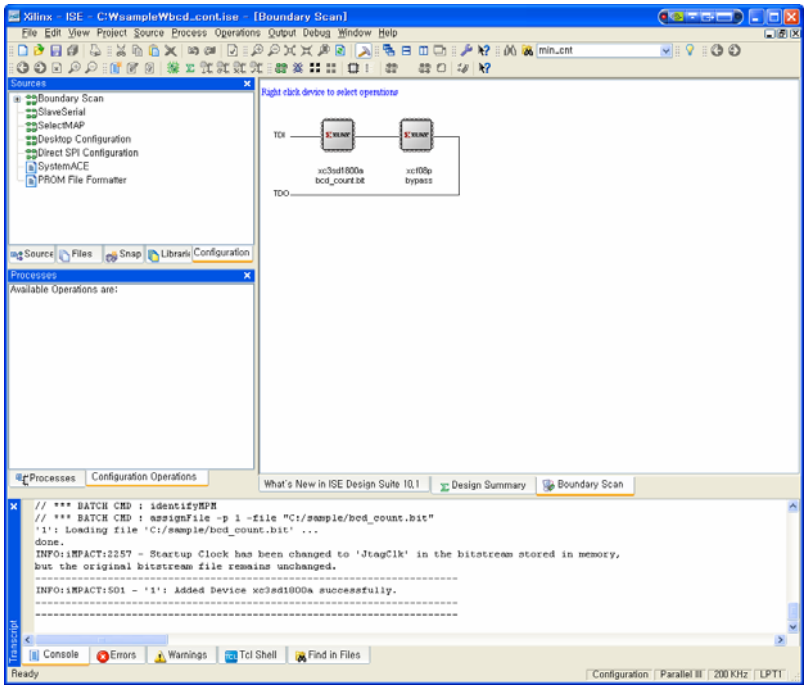
[그림 4-90] Initialize Chain

현재 장비에서 사용하고 있는 디바이스는 XC3SD1800AFGG676의 FPGA 디바이스와 XCF08의 PROM을 사용하고 있습니다. 따라서 디바이스 검색에서 2개의 디바이스가 검색이 되고, 검색된 디바이스에 어떠한 프로그램 파일을 프로그램 할지 검색할 수 있는 탐색 창이 나타나게 됩니다. 따라서 디바이스 별로 탐색 창이 생성되어 파일을 검색하여 지정해 주면 됩니다. 파일 설정이 끝나면 프로그램 설정을 해 주는 옵션 창이 활성화 되는데 여기에서 각 디바이스 별 다운로드 시 동작에 대한 설정을 해 주어야 합니다. 왼쪽 Category 창에서 FPGA와 PROM2를 선택할 수 있으며, 선택된 디바이스에 따른 설정을 오른쪽 메인 창에서 작업 하게 됩니다. FPGA 디바이스는 따로 체크할 필요가 없으며 PROM에 대한 다운로드 작업을 위해서는 다음의 그림과 같은 옵션을 체크해 주셔야 합니다. 이 옵션 작업은 PROM에 대한 설정 작업이면 PROM에 대한 프로그램 작업을 하지 않을 시 체크를 안하고 OK 버튼을 눌려 주셔도 됩니다.



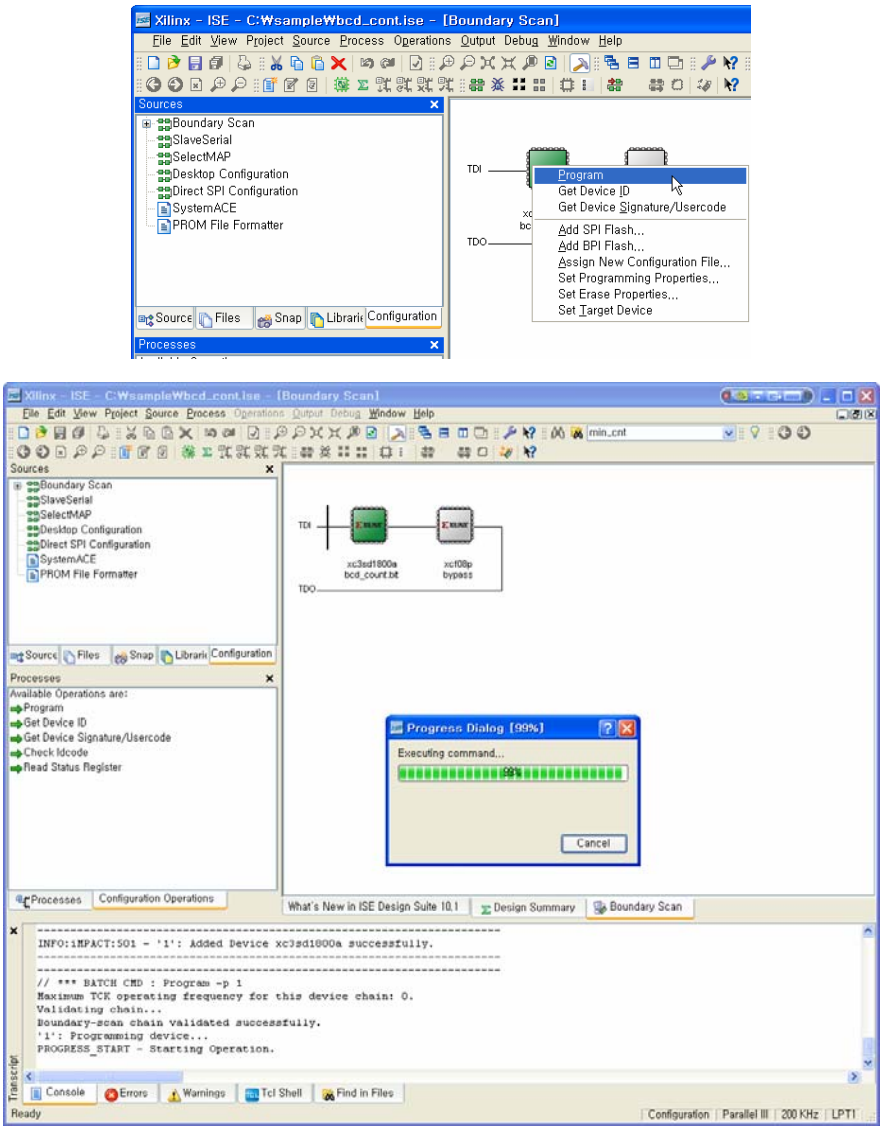
[그림 4-91] Programming Properties

[그림 4-92]에서는 JTAG를 통해 검색된 디바이스와 XC3SD1800A 디바이스에 프로그램 파일인 bcd_count.bit 파일이 지정된 모습을 보여주고 있습니다.



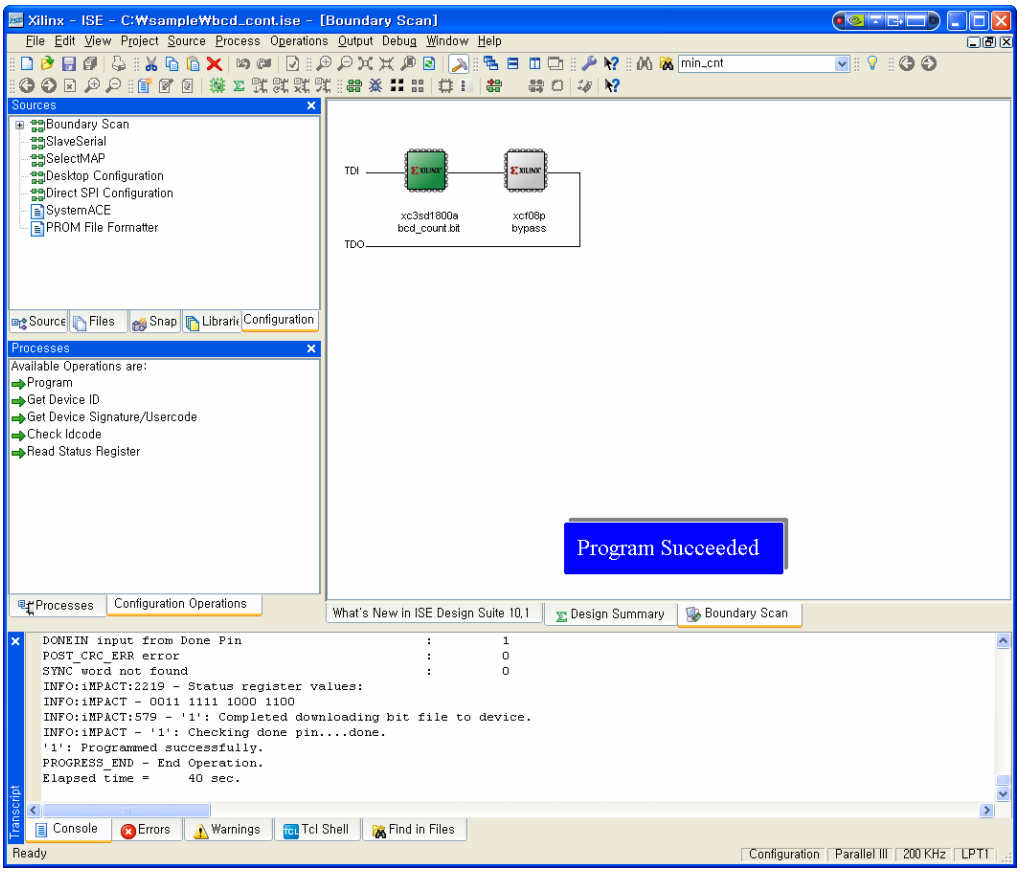
[그림 4-92] iMPACT

이렇게 파일을 지정했으면, 지정된 디바이스에 마우스 오른쪽 키를 눌러 Program 메뉴를 클릭해서 디바이스에 프로그램 하면 됩니다. [그림 4-93]에서 이러한 모습을 보여주고 있습니다.



[그림 4-93] Programming

디바이스에 프로그램이 완료 되면 프로그램 완료 창이 생성 됩니다. 또한 프로그램 다운로드 시 command가 99%에서 완료 메시지 창이 뜨기 까지의 시간이 수십 초가 걸리게 됩니다. 따라서 이 시간동안 케이블의 연결이나 전원을 OFF 시키는 일이 없어야 합니다. 다음의 그림에 다운로드 완료가 되었을 시 모습을 보여주고 있습니다.

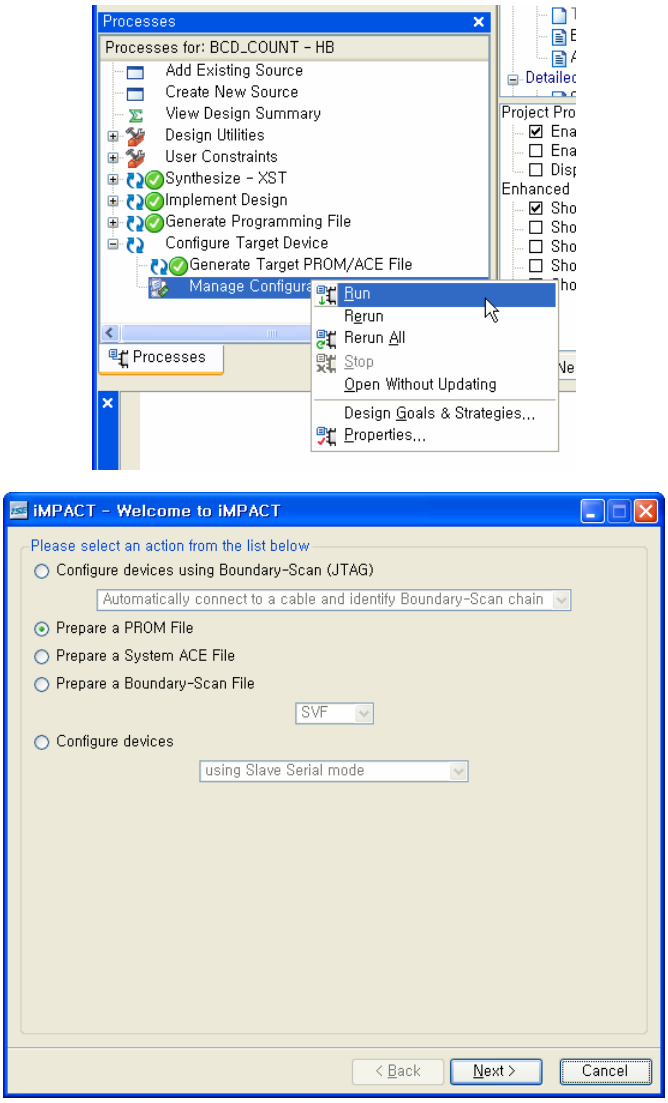


[그림 4-94] Program Succeeded

8) Configuration PROM

Xilinx 메인 디바이스도 Altera 디바이스와 같이 RAM 타입의 디바이스이기 때문에 별도의 ROM 타입의 디바이스를 같이 사용하여 이러한 점을 보완해 주고 있습니다. 여기에서 사용하는 PROM은 Xilinx사에서 나온 전용 Configuration PROM인 XCF08PVO48을 사용하고 있습니다. 따라서 여기도 마찬가지로 Xilinx사의 tool인 ISE를 가지고 파일의 변환 작업을 수행 하여야 합니다. Spartan-3A DSP를 가지고 컴파일 된 최종 파일을 이용하여 변환 작업을 수행합니다.

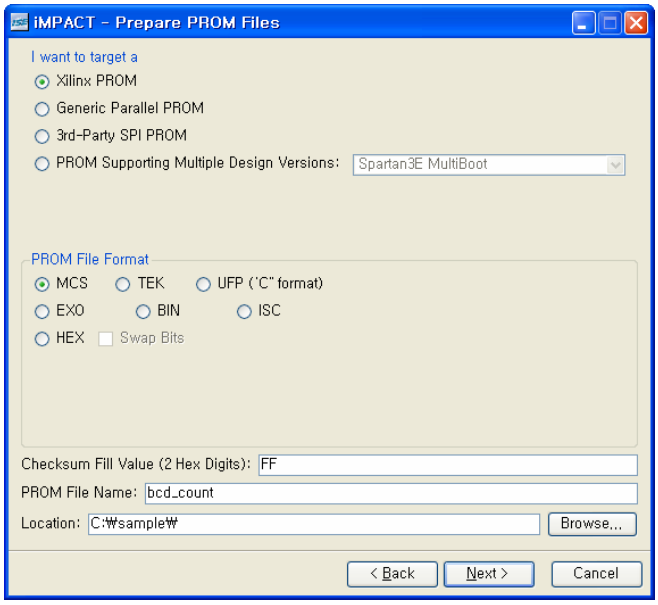
다음의 과정을 통해 이러한 변환 작업을 수행하는 방법이 나와 있습니다. [그림 A-95]은 .bit 파일을 Configuration PROM에 사용할 수 있는 파일로 변환을 위한 작업을 보여주고 있습니다. 이 작업 또한 iMPACT에서 할 수 있습니다. 따라서 ISE 왼쪽 창에서 있는 Configure Target Device에서 Manage Configuration Project(iMPACT)를 실행합니다. 다음의 그림에서 iMPACT를 실행하는 모습을 보여주고 있습니다. 이렇게 실행된 iMPACT창에서 Prepare a PROM File를 체크한 뒤 다음 단계로 진행합니다.



[그림 4-95] Create PROM File

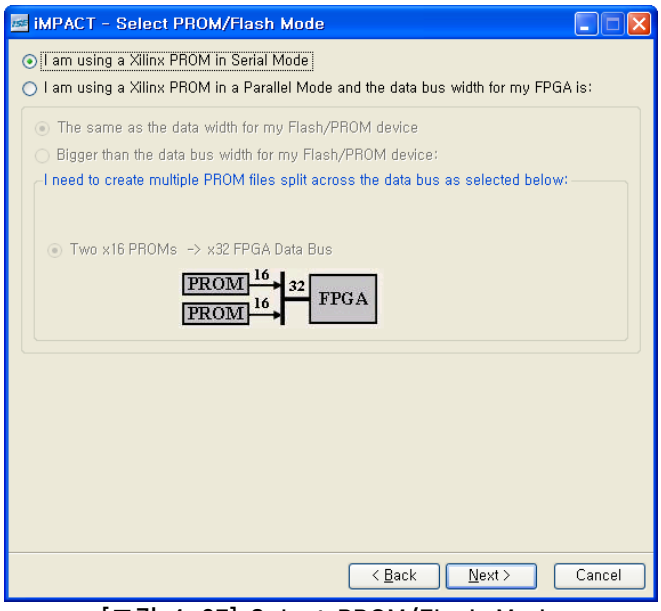
[그림 4-97]은 생성할 파일에 관한 설정을 해 주는 창입니다. 창에서 윗 부분에 잇는 I want to targer a 부분에서 어떠한 종류의 PROM을 사용하고 있는지 선택을 해 줍니다. 여기에서는 현재 장비에 장착되어 있는 Xilinx PROM 을 선택해 줍니다. 다음으로 PROM File Format 은 일반적인 PROM 프로그램 파일 형태인 MCS 를 선택해 줍니다. Checksum Fill Value (2 Hex Digits)는 Default로 설정되어 있는 FF 로 두고 다음에 있는 파일의 이름과 저장되는 폴더는 사용자가 직접 설정을 해 줍니다. 설정 전의 PROM File Name는 Untitled, Location은 프로젝트가 선언된 폴더로 지정이 되어 있습니다. 다음의 그림에서는 이러한 작업을 수행한 그림이 나와 있습

니다.



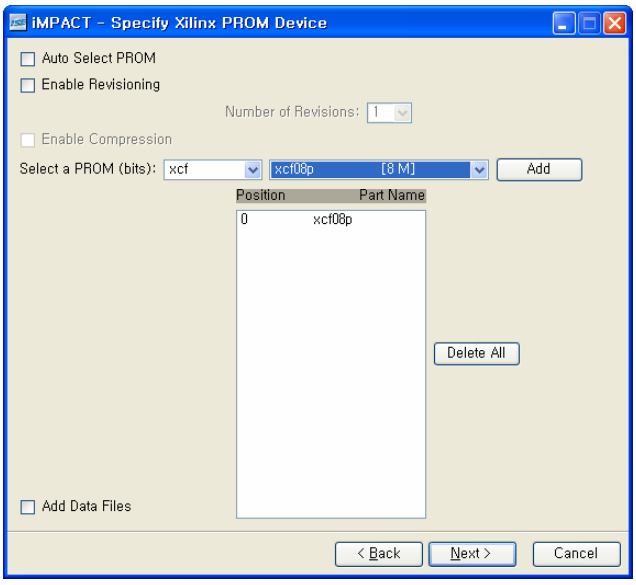
[그림 4-96] Prepare PROM Files

다음의 그림은 현재 장비에서 PROM과 FPGA의 연결 상태를 체크하는 창입니다. 현재 장비에서는 두 디바이스간의 데이터 전송 방식이 Serial Mode로 구성되어 있습니다. 따라서 이 창에서 I am using a Xilinx PROM in Serial Mode 를 체크하신 다음에 다음 단계로 진행하시면 됩니다.



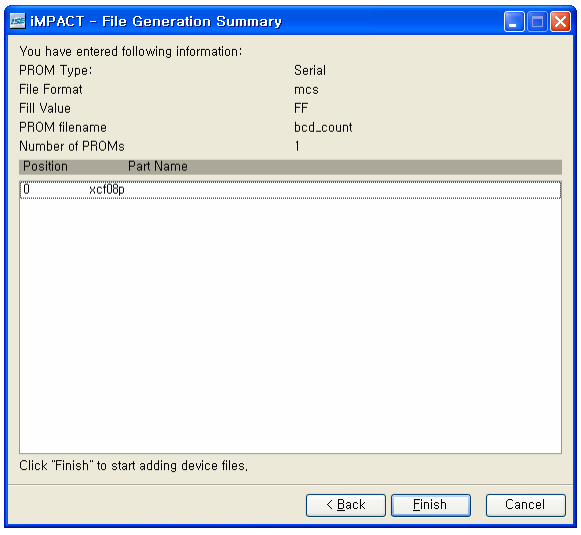
[그림 4-97] Select PROM/Flash Mode

[그림 4-98]은 현재 장비에서 사용하고 있는 Xilinx PROM 디바이스를 선택하는 작업을 합니다. 현재 타겟 장비에서 사용하고 있는 PROM의 종류를 선택해 주고 이러한 PROM이 몇 개 사용하는지를 이 창에서 결정해 주는 것 입니다. 따라서 현재 창에서는 Select a PROM 에서 Xilinx Device Module 에서 사용하고 있는 PROM인 XCF08PVO48를 선택하게 됩니다. 이러한 디바이스를 선택하고 현재 모듈에서 몇 개를 사용하는지를 Add 버튼을 이용하여 넣어 주면 됩니다. 여기에서는 현재 디바이스 모듈을 확인해 사용하는 디바이스를 넣어주고 가운데 창을 통해 디바이스 첨가해서 확인을 해 주면 됩니다.



[그림 4-98] Specify PROM Device

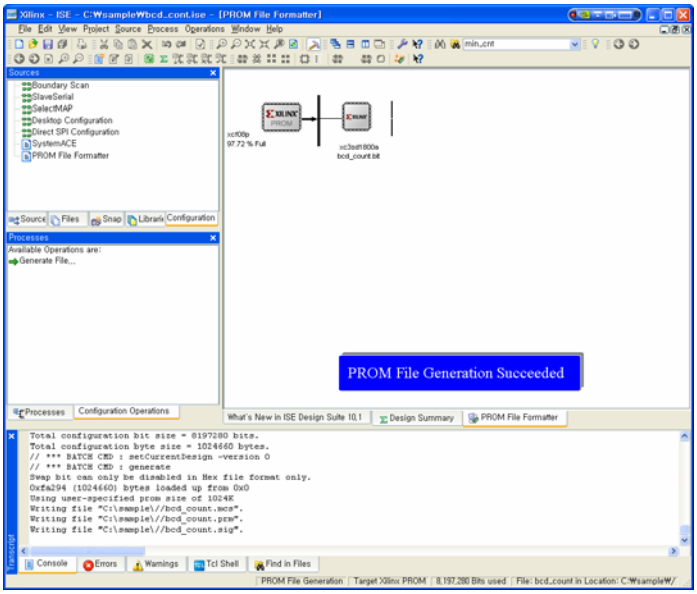
[그림 A-99]은 현재까지의 작업에 대한 Summary 형태로 보여주고 있습니다. 따라서 이전에서 작업한 것에 대한 내용을 이 창을 통해 최종적으로 확인해 볼 수 있습니다. 여기에서 확인한 내용이 이상이 없으면 Finish 버튼을 눌러 마치고, 수정 사항이 이상이 있을 시 Back 버튼을 통해 이 전 단계로 가서 다시 수정을 해 주면 됩니다.



[그림 4-99] File Generation Summary

이상의 작업을 마치고 아래의 Xilinx PROM에 적용을 해줄 파일 변환에 대한 창이 활성화 됩니다. 따라서 여기에서 변환할 .bit 파일을 현재의 작업 폴더에서 찾아서 넣어

주면 됩니다. 또한 bit 파일을 선택하면 더 추가할 파일이 있는 지 묻게 되는데 이 때 No를 클릭하여 창을 빠져 나오면 됩니다. 이렇게 하고 창에서 마우스 오른쪽 버튼을 이용해 Generate File... 을 실행하면 파일 변환을 수행하면 됩니다. 아래에서 이러한 작업의 완료된 모습을 보여주고 있습니다.



[그림 4-100] PROM File Generation Succeeded

현재까지 두 메인 디바이스에 대한 PROM 파일 변환 작업을 수행하였습니다. 사전에 위와 같이 파일 변환 작업을 하고 Xilinx 디바이스의 전용 다운로드 케이블을 이용하여 프로그램 수행을 하면 됩니다. 프로그램 하는 방법은 이 전의 FPGA 디바이스를 프로그램 할 때 PROM의 파일을 검색해서 포함해 주고, PROM도 같은 방법으로 프로그램 해 주면 됩니다.

부 록

5. 부록

5.1 장비 사용시 주의 사항

장비의 구동을 위해서는 상황에 따라 여러 개의 툴을 이용하게 됩니다. 또한 FPGA 디바이스를 제어하기 위해서도 각 회사 별로 Quartus II 및 ISE를 이용하여 설계해 주셔야 합니다.

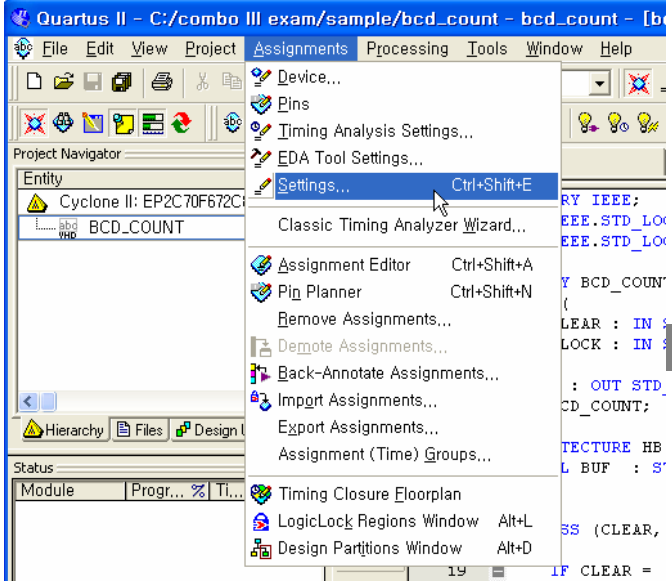
현재 장비에서는 FPGA Module 및 Process 모듈이 장착되어 있고 이 모듈들 간의 AMBA bus를 통해 연결 되어 있습니다. 따라서 FPGA 설계 시 이러한 AMBA bus에 대한 핀 정의를 해 주어야 합니다. 만약 FPGA 설계 시 이러한 핀을 사용하고 있으면 상관이 없지만 사용하지 않을 시 FPGA 툴에서는 핀을 사용하지 않은 핀을 GND나 HIGH로 동작을 하게 합니다. 따라서 FPGA 툴을 설계 시 사용하지 않는 핀에 대한 정의를 꼭 해 주셔야 합니다.

만약 다음에 나오는 핀 설정 작업이 없으면 FPGA 라인에 의한 Processor 모듈의 메인 디바이스가 손상을 입을 수 있기 때문에 이러한 작업을 꼭 해 주시기 당부 드립니다.

5.1.1 Quartus II에서의 Unused Pin 설정

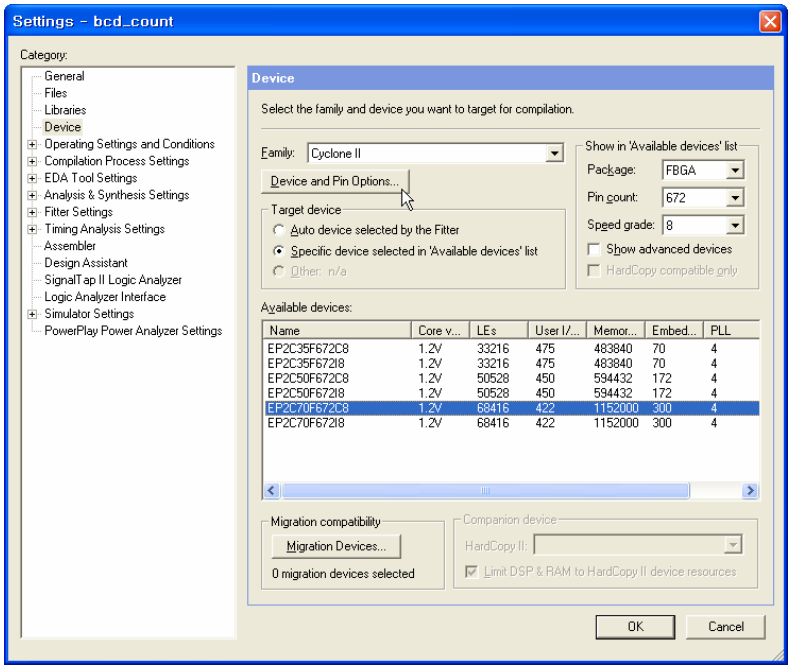
Quartus II의 설계 과정에서의 설정 작업은 메뉴의 Assignments를 통해 할 수 있습니다. 따라서 핀 작업뿐만 아니라 다른 설정 작업도 이 메뉴를 이용하여 설정 할 수 있도록 구성되어 있습니다.

먼저 메뉴의 Assignments -> Settings를 선택하여 프로젝트 설정 창을 활성화 합니다.



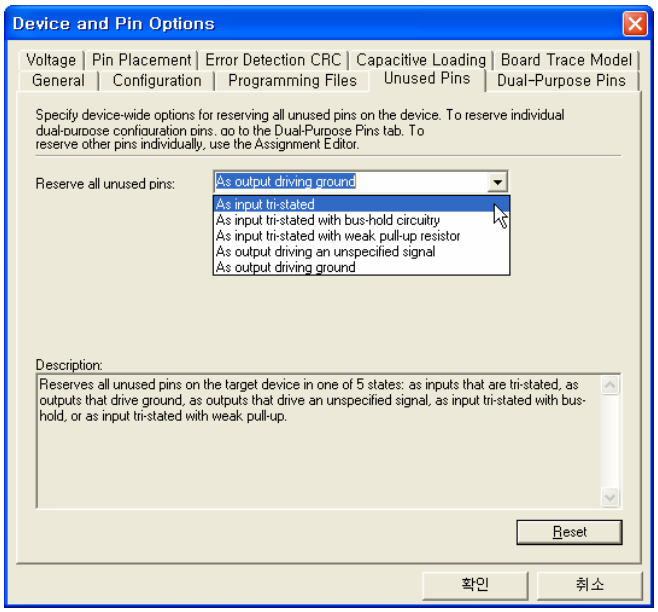
[그림 5-1] Assignments -> Settings

Setting 창에서 왼쪽 **Category**에서 **Device**를 선택하면 오른 쪽 설정 그림이 바뀌게 됩니다. 이 디바이스 설정 부분에서 Device and Pin Options 버튼을 클릭하여 다음 단계로 진행합니다.



[그림 5-2] Setting Device

버튼에 의해 생성되는 Device and Pin Options 창에서 위의 메뉴를 Unused Pins 로 선택하고, 활성화 되는 옵션에서 Reserve all unused pins를 As input tri-stated 를 선택해 줍니다.

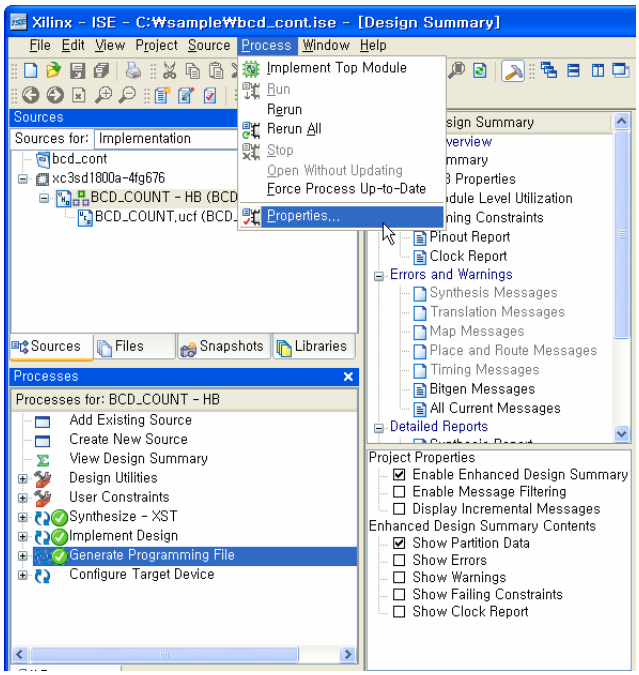


[그림 5-3] Device and Pin Options

이상의 설정을 마치고 확인을 눌러 설정을 마치고 다시 컴파일 작업을 수행해 주어야 현재 설정한 부분에 대해 프로젝트에 적용할 수 있습니다. 이상으로 Altera Quartus II에 대한 핀 설정 작업을 수행하였습니다.

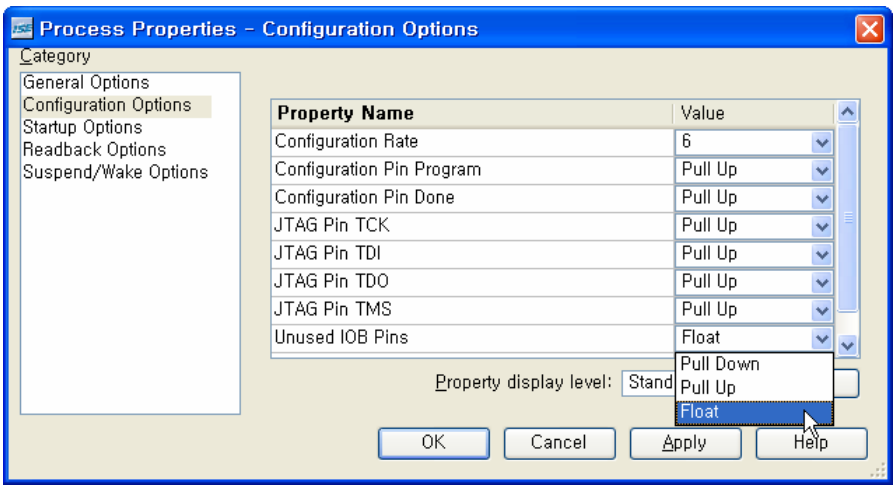
5.1.2 ISE에서의 Unused Pin 설정

Xilinx ISE에서도 사용하지 않는 핀에 대한 설정을 위해 메뉴의 Process -> Properties를 선택합니다.



[그림 5-4] Process -〉 Properties

이 창의 왼쪽 Category 에서 Configuration Options 를 선택하면 오른쪽 설정 창이 바뀌게 됩니다. 이창에서 Unused IOB Pins의 설정 부분을 Float 로 바꿔주시면 됩니다.



[그림 5-5] Configuration Options

설정이 바뀌면 OK 버튼을 통해 설정을 종료 하시면 되고, 완료된 작업을 프로젝트에 적용해 주기 위해 컴파일을 다시 해 주셔야 합니다. 이때 컴파일 작업은 Generate Programming File에 대한 컴파일 작업만 해 주시면 됩니다.